A Formalization of Happy Ending Problem

Divakaran D
Azim Premji University
divakaran.d@apu.edu.in

Syed Mohammad Meesum, T V H Prathamesh
Rishi Vyas
Krea University
{meesum.syed, prathamesh.turaga, rishi.vyas}@krea.edu.in

June 13, 2026

Contents

theory *Definitions*
imports *Main*

HOL−Library.Product-Lexorder
HOL−Analysis.Cartesian-Euclidean-Space
HOL−Library.Sublist

begin

type-synonym *R2* = *real* × *real*

definition *slope* :: *R2* ⇒ *R2* ⇒ *real* **where**
slope a b = (*snd b* − *snd a*) / (*fst b* − *fst a*)

abbreviation
sortedStrict ≡ *sorted-wrt* (<)

abbreviation *sdistinct* :: *R2 list* ⇒ *bool* **where**
sdistinct xs ≡ *distinct* (*map fst xs*) ∧ *sorted xs*

definition *nsubset*::'*a set* ⇒ *nat* ⇒ ('*a set*) *set* (**infix** ~ *76*) **where**
nsubset S k = {*X*. *X* ⊆ *S* ∧ *card X* = *k*}

definition *general-pos*::((*real*×*real*) *set*) ⇒ *bool* **where**
general-pos S ≡ (∀ *P3* ∈ *S*~*3*. ¬ *affine-dependent P3*)

definition *convex-pos*::('*a*::*euclidean-space set*) ⇒ *bool* **where**
convex-pos S ≡ (∀ *s* ∈ *S*. *convex hull S* ≠ *convex hull* (*S* − {*s*}))

abbreviation *convex-poly* :: *nat* ⇒ *R2 list* ⇒ *bool* **where**
convex-poly n xs ≡ *length xs* = *n* ∧ *convex-pos* (*set xs*)

definition *cross3* :: *R2* ⇒ *R2* ⇒ *R2* ⇒ *real* **where**
cross3 a b c = (*fst b* − *fst a*) ∗ (*snd c* − *snd b*) − (*fst c* − *fst b*) ∗ (*snd b* − *snd a*)

definition *cup3* :: - ⇒ - ⇒ - ⇒ *bool* **where**
cup3 a b c ≡ *cross3 a b c* > *0*

definition *cap3* :: - ⇒ - ⇒ - ⇒ *bool* **where**
cap3 a b c ≡ *cross3 a b c* < *0*

definition *collinear3* :: - ⇒ - ⇒ - ⇒ *bool* **where**
collinear3 a b c ≡ *cross3 a b c* = *0*

fun *list-check* :: (- ⇒ - ⇒ - ⇒ *bool*) ⇒ - *list* ⇒ *bool* **where**
list-check f [] = *True*
| *list-check f* (*a*#[]) = *True*
| *list-check f* (*a*#*b*#[]) = (*a* ≠ *b*)
| *list-check f* (*a*#*b*#*c*#*rest*) = (*f a b c* ∧ *list-check f* (*b*#*c*#*rest*))

definition *cap* :: *nat* ⇒ (*real*×*real*) *list* ⇒ *bool* **where**
cap k L ≡ (*k* = *length L*) ∧ (*sdistinct L*) ∧ (*list-check cap3 L*)

definition *cup* :: *nat* ⇒ (*real* × *real*) *list* ⇒ *bool* **where**
cup k L ≡ (*k* = *length L*) ∧ (*sdistinct L*) ∧ (*list-check cup3 L*)

definition *min-conv* :: *nat* ⇒ *nat* ⇒ *nat* **where**
min-conv k l =
Inf {*n* :: *nat*. (∀ *S* :: *R2 set*. (*card S* ≥ *n* ∧ *general-pos S* ∧ *sdistinct*(*sorted-list-of-set S*))
⟶ (∃ *xs*. *set xs* ⊆ *S* ∧ *sdistinct xs* ∧ (*cap k xs* ∨ *cup l xs*)))}

definition *min-conv-set* :: *nat* ⇒ *nat* ⇒ *nat set* **where**
min-conv-set k l =
{*n*. (∀ *S* :: *R2 set*. (*card S* ≥ *n* ∧ *general-pos S* ∧ *sdistinct*(*sorted-list-of-set*

S))
⟶ (∃ *xs*. *set xs* ⊆ *S* ∧ *sdistinct xs* ∧ (*cap k xs* ∨ *cup l xs*)))}

Miss Klein suggested the following problem. Can we find for a given number n a number N(n) such that from any set containing at least N(n) points it is possible to select n points forming a convex polygon? The number EZ(n) is defined to be the smallest possible value of N(n) for a given n.

definition *EZ* :: *nat* ⇒ *nat* **where**
EZ n = *Inf* {*N* :: *nat*.
(∀ *S* :: *R2 set*. (*card S* ≥ *N* ∧ *general-pos S* ∧ *sdistinct*(*sorted-list-of-set S*))
⟶ (∃ *xs*. *set xs* ⊆ *S* ∧ *sdistinct xs* ∧ *convex-poly n xs*))}

end
theory *Four-Convex*
imports *Definitions*
begin

lemma *convex-pos-inherit'*:
assumes *convex-pos S*
shows ∀ *A* ⊆ *S*. *convex-pos* (*S* − *A*)
unfolding *convex-pos-def*
proof(*rule ccontr*)
assume ¬ (∀ *A*⊆*S*. ∀ *s*∈*S* − *A*. *convex hull* (*S* − *A*) ≠ *convex hull* (*S* − *A* − {*s*}))
then obtain *A s* **where** *es*:*A*⊆*S s*∈*S*−*A convex hull* (*S*−*A*) = *convex hull* (*S*−*A*−{*s*})
by *blast*
hence *convex hull S* = *convex hull* (*S*−{*s*})
by (*smt* (*verit*, *ccfv-SIG*) *Diff-mono Diff-subset hull-inc hull-mono hull-redundant insert-Diff*
insert-mono rel-interior-empty rel-interior-subset subset-iff)
thus *False* **using** *assms convex-pos-def es*(*2*) **by** *blast*
qed

lemma *convex-pos-inherit0*:
assumes *convex-pos S*
shows ∀ *e* ∈ *S*. *convex-pos* (*S* − {*e*})
unfolding *convex-pos-def*
proof(*rule ccontr*)
assume ¬ (∀ *e*∈*S*. ∀ *s*∈*S* − {*e*}. *convex hull* (*S* − {*e*}) ≠ *convex hull* (*S* − {*e*} − {*s*}))
then obtain *e s* **where** *es*:*e*∈*S s*∈*S*−{*e*} *convex hull* (*S*−{*e*}) = *convex hull* (*S*−{*e*}−{*s*})
by *blast*
hence *convex hull S* = *convex hull* (*S*−{*s*})
by (*metis Diff-insert hull-insert insert-Diff-single insert-absorb insert-commute member-remove*
remove-def)

thus *False* **using** *assms convex-pos-def es(2)* **by** *blast*
qed

lemma *convex-pos-inherit*:
assumes *convex-pos S*
shows $\forall\ X \subseteq S$. *convex-pos X*
proof(*rule ccontr*)
assume $\neg\ (\forall\ X \subseteq S$. *convex-pos X*)
then obtain *X* **where** *X*:$X \subseteq S \neg$ (*convex-pos X*) **by** *auto*
then obtain *x* **where** *x*:$x \in X$ *convex hull* $X =$ *convex hull* $(X - \{x\})$
unfolding *convex-pos-def* **by** *blast*
then have $x \in$ *convex hull* $(X - \{x\})$
by (*metis hull-inc*)
then have $x \in$ *convex hull* $((X - \{x\}) \cup (S - X))$
by (*meson hull-mono in-mono sup.cobounded1*)
moreover have $((X - \{x\}) \cup (S - X)) = (S - \{x\})$
using *X(1) x(1)* **by** *fast*
ultimately have $x \in$ *convex hull* $(S - \{x\})$ **by** *simp*
hence *convex hull S* = *convex hull* $(S - \{x\})$ **using** *x(1) X(1)*
by (*metis hull-redundant in-mono insert-Diff*)
thus *False* **using** *x(1) X(1) assms*
using *convex-pos-def* **by** *auto*
qed

lemma *fourconvex*:
assumes $\forall X \subseteq (S::(real \times real)\ set).$ *card* $X \leq 4 \longrightarrow$ *convex-pos X*
shows *convex-pos S*
proof(*rule ccontr*)
assume *asm*:$\neg$ *convex-pos S*
then obtain *p* **where** *p*:$p \in S$ *convex hull* $S =$ *convex hull* $(S - \{p\})$
using *asm* **unfolding** *convex-pos-def* **by** *blast*
then have $p \in$ *convex hull* $(S - \{p\})$
using *hull-inc* **by** *fastforce*
then obtain *T* **where** *t*:*finite T* $T \subseteq S - \{p\}$ *card* $T \leq DIM(real \times real) + 1$
$p \in$ *convex hull T*
using *caratheodory*[*of* $S - \{p\}$] **by** *blast*
hence *c1*:$\neg$ *convex-pos* $(T \cup \{p\})$ **unfolding** *convex-pos-def*
by (*simp add*: *hull-insert insert-absorb subset-Diff-insert*)
have *c2*:*card* $(T \cup \{p\}) \leq 4$ **using** *t*
by (*simp add*: *subset-Diff-insert*)
have $T \cup \{p\} \subseteq S$ **using** *t(2) p(1)* **by** *auto*
thus *False* **using** *c1 c2*
by (*simp add*: *assms*)
qed

lemma *convexfour*:
assumes *convex-pos S*

shows $\forall X \subseteq (S::(real \times real)\ set).\ card\ X \leq 4 \longrightarrow convex\text{-}pos\ X$
using *assms convex-pos-inherit* **by** *auto*

theorem *fourConvexfour*:
$convex\text{-}pos\ S = (\forall X \subseteq (S::(real \times real)\ set).\ card\ X \leq 4 \longrightarrow convex\text{-}pos\ X)$
using *convexfour fourconvex* **by** *blast*
end
theory *Prelims*
imports *Definitions*

begin

lemma *subseq-rev*:
$subseq\ xs\ ys \Longrightarrow subseq\ (rev\ xs)\ (rev\ ys)$
proof(*induction xs arbitrary*: *ys*)
case *Nil*
then show *?case* **by** *simp*
next
case (*Cons a xs*)
then obtain *ysp yss* **where** *a-ys*: $ys = ysp\ @\ a\ \#\ yss\ subseq\ xs\ yss$
by (*metis* (*mono-tags*, *lifting*) *list-emb-ConsD*)
hence *subseq* (*rev xs*) (*rev yss*) **using** *Cons*(*1*) **by** *simp*
hence *subseq* ((*rev xs*)@[*a*]) ((*rev yss*)@[*a*]) **by** *simp*
then show *?case* **using** *a-ys*
by (*metis list-emb-prefix rev.simps*(*2*) *rev-append*)
qed

lemma *inf-subset-bounds*:
assumes $X \neq \{\}$ **and** $(X :: nat\ set) \subseteq (Y :: nat\ set)$
shows $Inf\ Y \leq Inf\ X$
by (*simp add*: *assms*(*1*,*2*) *cInf-superset-mono*)

lemma *inf-upper*:
$x \in (S::nat\ set) \longrightarrow Inf\ S \leq x$
by (*simp add*: *wellorder-Inf-le1*)

lemma *dst-set-card2-ne*:
fixes $S :: R2\ set$
assumes $card\ S \geq 2$
shows $\{(u,v) | u\ v.\ u \in S \wedge v \in S \wedge u < v\} \neq \{\}$
(**is** *?Spr* $\neq \{\}$)
proof−
obtain *s1 s2* **where** *s1*$\in$*S s2*$\in$*S s1*$<$*s2* **using** *assms*
by (*meson card-2-iff' ex-card linorder-less-linear subset-iff*)
hence (*s1*, *s2*) $\in$ *?Spr* **by** *blast*
thus *?Spr* $\neq \{\}$ **by** *force*
qed

lemma *width-non-zero*:

fixes S :: *real set*
assumes $S \neq \{\}$ **and** *finite* S
shows *Max* S − *Min* $S \geq 0$
using *assms Min-in* **by** *fastforce*

lemma *slope-div-ord*:
fixes *n1 n2 d1 d2* :: *real*
assumes $n1 \leq n2$ **and** $n1 > 0$
and $d1 \geq d2$ **and** $d2 > 0$
shows $n1 / d1 \leq n2 / d2$
using *assms* **by** (*simp add*: *divide-mono*)

lemma *finite-set-pr*:
assumes *finite S*
shows *finite* $\{(u,v) | u\ v.\ u \in S \wedge v \in S \wedge u < v\}$
(**is** *finite ?Spr*)
proof−
have *finite* $(S \times S)$ **using** *assms* **by** *blast*
moreover have $?Spr \subseteq S \times S$ **by** *blast*
ultimately show *?thesis* **by** (*meson rev-finite-subset*)
qed

lemma *distinctFst-distinct*:
distinct (*map fst xs*) ⟶ *distinct xs*
by (*simp add*: *distinct-map*)

value *sortedStrict* [(*1*,*2*)::*R2*, (*1*,*3*)]
value *sdistinct* [(*1*,*2*)::*R2*, (*1*,*3*)]
value *sdistinct* [(*1*,*2*), (*2*,*3*)]

lemma *sdistinct-sortedStrict*:
assumes *sdistinct xs*
shows *sortedStrict xs*
using *assms distinctFst-distinct strict-sorted-iff* **by** *auto*

lemma *append-first-sortedStrict*:
assumes *sorted-wrt* (<) (*B* :: *R2 list*) **and** (*b* :: *R2*) < (*hd B*)
shows *sorted-wrt* (<) (*b*#*B*)
proof−
have *1*: *sorted B distinct B* **using** *assms strict-sorted-iff* **by** *auto*
have *2*: *sorted* (*b*#*B*) **using** *assms*(*2*) *1*(*1*)
by (*metis list.sel*(*1*) *neq-Nil-conv nless-le sorted1 sorted2*)
have *3*: *distinct* (*b*#*B*) **using** *assms*(*2*) *1 2*
by (*metis* (*no-types*, *lifting*) *distinct.simps*(*2*) *list.sel*(*1*) *list.set-cases nless-le not-less-iff-gr-or-eq*

sorted-simps(*2*))
show *?thesis* **using** *2 3 strict-sorted-iff* **by** *blast*
qed

lemma *sorted-wrt-reverse-append*:
assumes *sorted-wrt* (>) (*B* :: *R2 list*) **and** (*b* :: *R2*) > (*hd B*)
shows *sorted-wrt* (>) (*b*#*B*)
proof−
have *1*: *distinct B* **using** *assms*
using *distinct-rev sorted-wrt-rev strict-sorted-iff* **by** *blast*
have *2*: *sorted-wrt* (>) (*b*#*B*) **using** *assms*(*2*) *1*
using *assms*(*1*) *distinct-rev sorted-wrt-rev strict-sorted-iff*
by (*smt* (*verit*, *best*) *list.sel*(*1*) *nless-le order-less-le-trans set-ConsD sorted-wrt.elims*(*2*) *sorted-wrt.simps*(*2*)
sorted-wrt1)
have *3*: *distinct* (*b*#*B*) **using** *assms 1 2*
by (*metis 1 assms*(*1*,*2*) *distinct.simps*(*2*) *distinct-singleton list.sel*(*1*) *not-less-iff-gr-or-eq set-ConsD*
sorted-wrt.elims(*2*))
show *?thesis* **using** *2 3 strict-sorted-iff* **by** *blast*
qed

lemma *append-last-sortedStrict*:
assumes *sorted-wrt* (<) (*B* :: *R2 list*)
and (*b* :: *R2*) > (*last B*)
shows *sorted-wrt* (<) (*B*@[*b*])
proof−
have *1*:*sorted-wrt* (>) (*rev B*) **using** *sorted-wrt-rev*[*of* (<)] **using** *assms*(*1*) **by** *force*
have *2*:*b* > *hd* (*rev B*)
by (*simp add*: *assms*(*2*) *hd-rev*)
have *3*:*B*@[*b*] = *rev* (*b*#(*rev B*)) **using** *rev-def* **by** *simp*
show *?thesis* **using** *sorted-wrt-rev 1 2 3 assms sorted-wrt-reverse-append*
by *metis*
qed

lemma *ind-seq-subseq*:
assumes *a* < *b b* < *c c* < *length xs* **and** *sdistinct xs*
shows *sdistinct* [*xs*!*a*, *xs*!*b*, *xs*!*c*]
proof
show *distinct* (*map fst* [*xs* ! *a*, *xs* ! *b*, *xs* ! *c*]) **using** *assms*(*1*,*2*,*3*,*4*)
by (*smt* (*verit*, *ccfv-threshold*) *distinct-singleton dual-order.trans nth-map*
le-eq-less-or-eq distinct-conv-nth distinct-length-2-or-more list.simps(*8*,*9*)
length-map verit-comp-simplify1(*3*))
next
show *sorted* [*xs* ! *a*, *xs* ! *b*, *xs* ! *c*]
by (*meson assms*(*1*,*2*,*3*,*4*) *less-or-eq-imp-le order.strict-trans1 sorted1 sorted2 sorted-nth-mono*)

qed

lemma *subseq-index*:
fixes *xs* :: *R2 list*
assumes *subseq ys xs*
shows ∃ *idx*.
set idx ⊆ {..<*length xs*} ∧
ys = *map* (λ *id*. *xs*!*id*) *idx* ∧
length idx = *length ys* ∧
sorted-wrt (<) *idx*
using *assms*
proof(*induction ys arbitrary*: *xs*)
case *Nil*
then show *?case* **by** *simp*
next
case (*Cons a ys*)
hence *subseq ys xs* **by** (*metis subseq-Cons'*)
obtain *xsp xss* **where** *a-xs*:*xs* = *xsp*@*a*#*xss subseq ys xss* **using** *Cons*(*2*) *list-emb-ConsD*
by (*metis* (*full-types*, *opaque-lifting*))
have *xs* = (*xsp*@[*a*])@*xss* **using** *a-xs*(*1*) **by** *simp*
hence *id-tr*:∀ *z* < *length xss*. *xss*!*z* = *xs*!(*1* + *length xsp* + *z*) **using** *a-xs*(*1*)
by (*metis add-Suc-shift nth-Cons-Suc nth-append-length-plus plus-1-eq-Suc*)

then obtain *idx* **where** *idx*:
set idx ⊆ {..<*length xss*}
ys = *map* ((!) *xss*) *idx*
length idx = *length ys*
sorted-wrt (<) *idx*
using *Cons.IH a-xs* **by** *blast*

define *idxsa* **where** *idxsa*: *idxsa* = (*map* (λ*n*. *1* + *length xsp* + *n*) *idx*)
have *idxsa-s*:*set idxsa* ⊆ {..<(*1* + *length xsp* + *length xss*)}
using *idxsa a-xs*(*1*) *idx*(*1*) **by** *auto*
have *sorted-wrt* (<) *idxsa* **using** *idx*(*4*) *idxsa* **by** (*simp add*: *sorted-wrt-iff-nth-less*)
hence *4*:*sorted-wrt* (<) ((*length xsp*)#*idxsa*) **using** *idxsa* **by** *simp*
hence *ys-idxs*: *ys* = *map* ((!) *xs*) *idxsa*
using *idx*(*2*) *idxsa* **by** (*simp add*: *a-xs*(*1*) *nth-append*)
hence *aind*:*xs*!*length xsp* = *a* **using** *a-xs* **by** *simp*
hence *2*:(*a*#*ys*) = *map* ((!) *xs*) ((*length xsp*)#*idxsa*) **using** *ys-idxs* **by** *simp*
have *3*:*length* (*a*#*ys*) = *length* ((*length xsp*)#*idxsa*) **using** *idx*(*3*) *idxsa* **by** *simp*
have *1*:*set* ((*length xsp*)#*idxsa*) ⊆ {..<*length xs*} **using** *idxsa-s a-xs*(*1*) **by** *auto*
then show *?case* **using** *1 2 3 4* **by** *metis*
qed

lemma *subseq-index3*:
fixes *xs* :: *R2 list*
assumes *subseq* [*a*,*b*,*c*] *xs*
shows ∃ *i j k*. *i*<*j* ∧ *j*<*k* ∧ *k*<*length xs* ∧ *a* = *xs*!*i* ∧ *b* = *xs*!*j* ∧ *c* = *xs*!*k*
proof−

```
  obtain idx where idx:
    set idx ⊆ {..<length xs}
    [a,b,c] = map (λ id. xs!id) idx
    sorted-wrt (<) idx
    length idx = 3
    using assms subseq-index
    by (metis length-Cons list.size(3) numeral-3-eq-3)
  obtain a b c where ijk: idx = Cons a (Cons b (Cons c Nil))
    using idx(4) numeral-3-eq-3 length-0-conv Suc-length-conv length-Suc-conv by
smt
  hence 1:c < length xs using idx ijk
    by (meson lessThan-iff list.set-intros(1) set-subset-Cons subset-code(1))
  thus ?thesis using idx(2,3) 1 ijk by auto
qed

lemma sdistinct-subseq:
  fixes   xs ys :: R2 list
  assumes subseq xs ys and sdistinct ys
  shows   sdistinct xs
  using assms
proof(induction ys)
  case (list-emb-Nil ys)
  then show ?case by simp
next
  case (list-emb-Cons xs ys y)
  then show ?case by simp
next
  case (list-emb-Cons2 x y xs ys)
  hence F1: sdistinct xs by simp
  then show ?case
  proof
    assume asm: distinct (map fst xs) sorted xs
    have fst x ∉ set (map fst ys)
      using list-emb-Cons2.hyps(1)
        list-emb-Cons2.prems
      by auto
    hence fst x ∉ set (map fst xs)
      by (metis (full-types, opaque-lifting)
          list-emb-Cons2.hyps(2)
          list-emb-set
          subseq-map)
    hence 1:distinct (map fst (x#xs))
      by (simp add: asm(1))
    have ((∀ y ∈ set ys. x≤y) ∧ sorted ys)
      using list-emb-Cons2.hyps(1)
        list-emb-Cons2.prems
        sorted-simps(2)
      by blast
    hence ((∀ y ∈ set xs. x≤y) ∧ sorted xs)
```

```
    using asm(2)
      list-emb-Cons2.hyps(2)
      list-emb-set
    by fastforce
  hence sorted (x#xs)
    by simp
  thus ?thesis using 1 by simp
 qed
qed


corollary sdistinct-subl:
 fixes  X Y :: R2 list
 assumes sublist X Y and sdistinct Y
 shows  sdistinct X
 using assms sdistinct-subseq by blast

lemma sdistinct-sub:
 fixes  A B :: R2 set
 assumes finite B and A ⊆ B and sdistinct(sorted-list-of-set B)
 shows  sdistinct(sorted-list-of-set A)
proof−
 have subseq (sorted-list-of-set A) (sorted-list-of-set B) using assms(1,2)
 proof(induction sorted-list-of-set B arbitrary: A B)
  case Nil
  hence B = {}
   by (metis sorted-list-of-set.sorted-key-list-of-set-eq-Nil-iff)
  hence A = {} using Nil by simp
  then show ?case by simp
 next
  case (Cons a x)
  define Ba where Ba ≡ B − {a}
  hence 1:x = sorted-list-of-set(Ba) using Cons(2,3,4)
   by (metis remove1.simps(2) sorted-list-of-set.sorted-key-list-of-set-remove)
  have 2: finite Ba using Cons(3) Ba-def by blast
  hence 3: A − {a} ⊆ Ba using Ba-def Cons(4) by blast
  hence 4:a ∈ A ⟹ subseq (sorted-list-of-set (A−{a})) (sorted-list-of-set Ba)
   using Cons 1 2 by simp
  have 5:a ∈ A ⟹ a#(sorted-list-of-set (A−{a})) = sorted-list-of-set A
   by (smt (verit, best) Cons.hyps(2) Cons.prems(1,2) Min-in Min-le empty-iff
finite-has-minimal2
      finite-subset list.inject sorted-list-of-set-nonempty subset-iff)

  hence 6:a ∈ A ⟹ subseq (sorted-list-of-set A) (sorted-list-of-set B)
   by (metis 1 4 Cons.hyps(2) subseq-Cons2)
  have a∉A ⟹ ?case using Cons 1 2 3
   by (metis Diff-empty Diff-insert0 list-emb-Cons)
  thus ?case using 6 by auto
 qed
```

thus *sdistinct* ((*sorted-list-of-set A*))
using *sdistinct-subseq assms* **by** *simp*
qed

lemma *subseq-sorted-subset*:
assumes *distinct xs* **and** *sorted xs* **and** $Y \subseteq$ *set xs*
shows *subseq* (*sorted-list-of-set Y*) *xs*
using *assms*
proof(*induction xs arbitrary*: *Y*)
case *Nil*
then show *?case* **by** *simp*
next
case (*Cons a xs*)
hence *1*:*distinct xs*
by (*metis distinct.simps*(*2*))
have *2*:*sorted xs* **using** *Cons.prems*(*2*) *sorted-simps*(*2*) **by** *blast*
have *3*:$Y - \{a\} \subseteq$ *set xs* **using** *Cons*(*4*) **by** *auto*
have *4*:*subseq* (*sorted-list-of-set* (*Y*−{*a*})) *xs* **using** *Cons*(*1*) *1 2 3* **by** *simp*
show *?case*
proof(*cases a* ∈ *Y*)
case *True*
have *T0*:*a* = *min-list* (*a*#*xs*) **using** *Cons*(*2*,*3*)
by (*metis List.finite-set Min-insert2 list.distinct*(*1*)
list.simps(*15*) *min-list-Min sorted-simps*(*2*))
hence *sorted-list-of-set Y* = *a* # *sorted-list-of-set* (*Y*−{*a*}) **using** *Cons.prems*(*3*)
True
by (*metis List.finite-set Min-antimono list.distinct*(*1*) *min-list-Min finite-subset*
Min-le dual-order.eq-iff empty-iff sorted-list-of-set-nonempty)
then show *?thesis* **using** *1 2 3 4* **by** (*metis subseq-Cons2*)
next
case *False*
then show *?thesis* **using** *Cons.prems*(*3*) *Cons.IH* **by** (*auto simp add*: *1 2*)
qed
qed

lemma *ex-equiv-minconv*:
(∃ *xs*. *set xs* ⊆ *S* ∧ (*sortedStrict xs*) ∧ (*cap k xs* ∨ *cup l xs*)) ⟶
(∃ *xs*. (*set xs* ⊆ *S* ∧ (*sortedStrict xs*)) ⟶ (*cap k xs* ∨ *cup l xs*))
by *blast*

lemma *cross3-commutation-12*[*simp*]: *cross3 x y z* = *0* ⟶ *cross3 y x z* = *0*
unfolding *cross3-def* **by** *argo*

lemma *cross3-commutation-23*[*simp*]: *cross3 x y z* = *0* ⟶ *cross3 x z y* = *0*
unfolding *cross3-def* **by** *argo*

lemma *cross3-non0-distinct*: *cross3 a b c* ≠ *0* ⟶ *distinct* [*a*,*b*,*c*]

unfolding *cross3-def*
by (*metis cross3-commutation-23 cross3-def diff-self distinct-length-2-or-more distinct-singleton mult-zero-left mult-zero-right*)

lemma *cap-reduct*:
assumes *cap* (*k*+*1*) (*a*#*xs*)
shows *cap k xs*
using *assms add-left-cancel length-Cons list.distinct*(*1*) *list.inject list-check.elims*(*2*)
unfolding *cap-def* **by** *fastforce*

lemma *cup-reduct*:
assumes *cup* (*k*+*1*) (*a*#*xs*)
shows *cup k xs*
using *assms add-left-cancel length-Cons list.distinct*(*1*) *list.inject list-check.elims*(*2*)
unfolding *cup-def* **by** *fastforce*

lemma *card-of-s*:
assumes *set xs* ⊆ *S cap k xs distinct xs finite S*
shows *card S* ≥ *k*
by (*metis assms cap-def card-mono distinct-card*)

lemma assumes (*X*::*nat set*) ≠ {}
shows ∃ *S* ∈ *X*. *Inf X* = *S*
using *assms*
by (*simp add*: *Inf-nat-def1*)

lemma fixes *S*
assumes *finite S*
shows ∃ *xs*. *length xs* = *card S* ∧ *set xs* = *S*
using *assms*
by (*metis distinct-card finite-distinct-list*)

lemma *set2list*:
assumes *finite* (*S* :: *R2 set*)
shows ∃!*xs*. *set xs* = *S* ∧ *sortedStrict xs* ∧ *distinct xs* ∧ *length xs* = *card S*
using *finite-sorted-distinct-unique*[*of S*] *strict-sorted-iff distinct-card assms*
by *metis*

lemma *exactly-one-true*:
assumes *A* = *collinear3 a b c* **and** *B* = *cap3 a b c* **and** *C* = *cup3 a b c*
shows (*A* ∨ *B* ∨ *C*) ∧ ((¬*A* ∧ ¬*B*) ∨ (¬*B* ∧ ¬*C*) ∨ (¬*C* ∧ ¬*A*))
using *assms* **unfolding** *collinear3-def cap3-def cup3-def* **by** *linarith*

lemma *lcheck-len2-T*:
assumes *length P* ≤ *2* **and** *sdistinct P* **shows** *list-check f P*
proof(*cases length P* = *2*)
case *True*
hence ∃ *x y*. *P*=[*x*,*y*] ∧ *sdistinct* [*x*,*y*] **using** *assms*(*2*)
by (*metis* (*no-types*, *lifting*) *One-nat-def Suc-1 length-0-conv length-Suc-conv*)

then show *?thesis* **using** *list-check.simps(3)* **by** *fastforce*
next
case *False*
hence $(P = []) \vee (\exists x.\ P{=}[x])$
by (*metis assms(1) length-0-conv length-Suc-conv less-2-cases-iff nat-less-le*)
then show *?thesis* **by** *fastforce*
qed

lemma *bad3-from-bad*:
assumes *sdistinct P* **and** ¬ *list-check cc P*
shows $\exists a\ b\ c.$ (*sublist* [*a*,*b*,*c*] *P*) ∧ ¬ *cc a b c*
using *assms*
proof(*induction length P arbitrary*: *P*)
case (*Suc x*)
then show *?case*
proof(*cases length P* ≤ *2*)
case *False*
hence $\exists a\ u\ v\ rest.\ P = (a\#u\#v\#rest)$ **using** *Suc(2) Suc.prems(2) assms(1) list.size(3)*
by (*metis One-nat-def Suc-1 bot-nat-0.extremum length-Cons list.exhaust not-less-eq-eq*)
then obtain *a u v rest* **where** *aP*: $P = (a\#u\#v\#rest)$ **by** *blast*
hence ¬ *cc a u v* ∨ ¬ *list-check cc* (*u*#*v*#*rest*) **using** *Suc* **by** *simp*
then show *?thesis*
proof
assume ¬ *cc a u v*
thus $\exists a\ b\ c.$ *sublist* [*a*, *b*, *c*] *P* ∧ ¬ *cc a b c* **using** *aP*
by (*metis append-Cons append-Nil sublist-append-rightI*)
next
assume ¬ *list-check cc* (*u*#*v*#*rest*)
moreover have *length* (*u*#*v*#*rest*) = *x* **using** *aP Suc(2)* **by** *simp*
moreover have *sdistinct* (*u*#*v*#*rest*) **using** *Suc(3) aP* **by** *simp*
ultimately have $\exists a\ b\ c.$ *sublist* [*a*, *b*, *c*] (*u*#*v*#*rest*) ∧ ¬ *cc a b c* **using** *Suc(1)* **by** *blast*
thus $\exists a\ b\ c.$ *sublist* [*a*, *b*, *c*] *P* ∧ ¬ *cc a b c* **using** *aP Suc* **by** (*meson sublist-Cons-right*)
qed
qed(*metis lcheck-len2-T*)
qed(*auto*)

lemma *get-cup-from-bad-cap*:
assumes *sdistinct P* **and** ¬ *cap* (*length P*) *P*
shows $\exists a\ b\ c.$ (*sublist* [*a*,*b*,*c*] *P*) ∧ ¬ *cap3 a b c*
using *bad3-from-bad assms(1,2) cap-def*
by *blast*

lemma *get-cap-from-bad-cup*:
assumes *sdistinct P* **and** ¬ *cup* (*length P*) *P*
shows $\exists a\ b\ c.$ (*sublist* [*a*,*b*,*c*] *P*) ∧ ¬ *cup3 a b c*

```
  using bad3-from-bad assms(1,2) cup-def strict-sorted-iff by blast

lemma list-cap-tail:
  assumes list-check cap3 (xs@[c,b])
    and cross3  c b a < 0
  shows list-check cap3 (xs@[c,b,a])
  using assms
proof(induction xs)
  case Nil
  hence distinct [c,b,a] using cross3-def
    using cross3-non0-distinct not-less-iff-gr-or-eq by blast
  hence list-check cap3 [b,a] unfolding list-check.simps(3) by simp
  then show ?case using list-check.simps(4)[of cap3 c b a []] unfolding cap3-def
    by (metis append-Nil assms(2))
next
  case (Cons x xs)
  hence Cons1: list-check cap3 ((x # xs) @ [c, b]) by argo
  have cross3: cross3 c b a < 0 using Cons by argo
  have list-check cap3 (xs@[c,b]) using Cons(2) list-check.simps
    using list-check.elims(3) by fastforce
  hence s1:list-check cap3 (xs@[c,b,a]) using Cons by argo
  then show ?case
  proof(cases xs)
    case Nil
    hence cross3 x c b < 0 using Cons(2) list-check.simps(4)[of cap3 x c b []]
      unfolding cap3-def
      by (metis (no-types, lifting) append-eq-Cons-conv)
    then show ?thesis using Nil list-check.simps(4)
      by (metis s1 append-Cons append-Nil cap3-def)
  next
    case (Cons y ys)
    hence xs-is:xs = y#ys by argo
    then show ?thesis
    proof(cases ys)
      case Nil
      hence cap3 x y c using Cons Cons1 by simp
      then show ?thesis using Cons Cons1 cross3 unfolding cap3-def
        by (metis ‹list-check cap3 (xs @ [c, b, a])› append-Cons append-Nil
            list-check.simps(3,4) local.Nil)
    next
      case (Cons z zs)
      hence cap3 x y z using xs-is Cons Cons1 by simp
      then show ?thesis using xs-is Cons s1 by auto
    qed
  qed
qed

lemma list-cup-tail:
  assumes list-check cup3 (xs@[c,b])
```

```
    and cross3  c b a > 0
  shows list-check cup3 (xs@[c,b,a])
  using assms
proof(induction xs)
  case Nil
  hence distinct [c,b,a] using cross3-def
    using cross3-non0-distinct not-less-iff-gr-or-eq
    by (metis assms(2))
  hence list-check cup3 [b,a] unfolding list-check.simps(3) by simp
  then show ?case using list-check.simps(4)[of cup3 c b a []] unfolding cup3-def
    by (metis append.left-neutral assms(2))
next
  case (Cons x xs)
  hence Cons1: list-check cup3 ((x # xs) @ [c, b])  by argo
  have cross3: cross3 c b a > 0 using Cons by argo
  have list-check cup3 (xs@[c,b]) using Cons(2) list-check.simps
    using list-check.elims(3) by fastforce
  hence s1:list-check cup3 (xs@[c,b,a]) using Cons by argo
  then show ?case
  proof(cases xs)
    case Nil
    hence cross3 x c b > 0 using Cons(2) list-check.simps(4)[of cup3 x c b []]
      unfolding cup3-def
      by (metis (no-types, lifting) append-eq-Cons-conv)
    then show ?thesis using Nil list-check.simps(4)
      by (metis s1 append-Cons append-Nil cup3-def)
  next
    case (Cons y ys)
    hence xs-is:xs = y#ys by argo
    then show ?thesis
    proof(cases ys)
      case Nil
      hence cup3 x y c using Cons Cons1 by simp
      then show ?thesis using Cons Cons1 cross3 unfolding cap3-def
        by (metis s1 append-Cons append-Nil list-check.simps(4) local.Nil)
    next
      case (Cons z zs)
      hence cup3 x y z using xs-is Cons Cons1 by simp
      then show ?thesis using xs-is Cons s1 by auto
    qed
  qed
qed

lemma extend-cap-cup:
  assumes sortedStrict (B@[b]) and list-check cc (B :: R2 list) and cc (B!(length
B−2)) (B!(length B−1)) b
  shows   list-check cc (B@[b])
using assms
proof(induction B)
```

```
  case (Cons a B)
  show ?case
  proof(cases length B = 0)
    case True
    then show ?thesis using Cons(2) by simp
  next
    case cF:False
    then show ?thesis
    proof(cases length B = 1)
      case True
      then obtain x where B = [x]
        by (metis One-nat-def Suc-length-conv length-0-conv)
      then show ?thesis using Cons(2) True
        using Cons.prems(3) by auto
    next
      case False
      then have blen: length B ≥ 2 using cF by linarith
      have 1:sortedStrict (B @ [b]) using Cons by simp
      have 2:list-check cc B using Cons
        by (smt (verit) list-check.elims(1) list-check.simps(4))
     have 3:cc (B ! (length B − 2)) (B ! (length B − 1)) b using blen Cons.prems
       by (metis Suc-1 cF diff-Suc-1 diff-Suc-eq-diff-pred length-Cons less-eq-Suc-le
nth-Cons' order-less-irrefl
           zero-less-diff)
      have 4:list-check cc (B @ [b]) using 3 1 2  Cons.IH by blast
      obtain c1 c2 crest where cp: B = c1#c2#crest using blen
      by (metis Suc-1 Suc-le-length-iff list.size(3) neq-Nil-conv not-one-le-zero)
      hence 5:cc a c1 c2 using Cons.prems(2) list-check.simps(4)[of cc] by simp
      have list-check cc (c1#c2#crest@[b]) using cp 4 by simp
     then show ?thesis using list-check.simps(4)[of cc a c1 c2 crest@[b]] 5 cp by
auto
    qed
  qed
qed(simp)

end
theory GeneralPosition3
  imports Definitions Prelims

begin
lemma general-pos-subs:
  assumes X ⊆ Y and general-pos Y
  shows general-pos X
proof(rule ccontr)
  assume ¬general-pos X
 then obtain S where S:S ∈ X^~ 3 affine-dependent S unfolding general-pos-def
by blast
  have S ∈ Y^~3 using assms(1) unfolding nsubset-def
    by (smt (verit) S mem-Collect-eq nsubset-def order-trans)
```

thus *False* **using** *S(2) assms(2)* **unfolding** *general-pos-def* **by** *simp*
qed

lemma *affine-comb-affine-dep*:
assumes $(u :: real) \geq 0$ **and** $x = (1 - u) *_R y + u *_R (z :: R2)$
and *distinct* $[x, y, z]$
shows *affine-dependent* $\{x, y, z\}$
proof−
have $x \in$ *affine hull* $\{y, z\}$ **using** *assms affine-def hull-def* **by** *force*
thus *?thesis* **using** *assms* **by** (*simp add*: *affine-dependent-def*)
qed

lemma *pair-scal*:$c *_R u = (c * fst\ u, c * snd\ u)$
by (*simp add*: *scale-def*)

lemma *lindep-cross3-0*:
assumes $(x :: R2) = u *_R y + (1 - (u :: real)) *_R z$
shows *cross3 x y z = 0*
proof−
have *1*: *fst y − fst x =* $(1 - u) * (fst\ y - fst\ z)$ **using** *assms* **by** (*simp*, *argo*)
have *2*: *snd z − snd x =* $- u * (snd\ y - snd\ z)$ **using** *assms* **by** (*simp*, *argo*)
have *3*: *fst z − fst x =* $- u * (fst\ y - fst\ z)$ **using** *1* **by** *argo*
have *snd y − snd x =* $(1 - u) * (snd\ y - snd\ z)$ **using** *2* **by** *argo*
thus *?thesis* **unfolding** *cross3-def* **using** *1 2 3*
by (*smt* (*verit*, *best*) *minus-mult-minus mult.commute*
vector-space-over-itself.scale-scale)
qed

lemma *in-hull3-iscollinear*:
assumes $(x :: R2) \in$ *affine hull* $\{y, z\}$
shows *cross3 x y z = 0*
proof−
have $\exists v{\geq}0. \exists u{\geq}0.\ (u+v = 1) \longrightarrow (x = u *_R y + v *_R z)$
using *assms* **unfolding** *hull-def affine-def* **by** *force*
then obtain *u v* **where** $u+v = 1\ x = u *_R y + v *_R z$
using *assms affine-hull-2* **by** *fastforce*
thus *?thesis* **using** *lindep-cross3-0 assms* **by** (*smt* (*verit*))
qed

lemma *distinct-snd-case-aux*:
assumes *distinct* $[snd\ x, snd\ y, snd\ z :: real]$
and *fst x = fst y* **and** *fst y = fst z*
shows *affine-dependent* $\{x, y, z :: R2\}$
proof−
have *1*: *snd z − snd x* $\neq 0$ **using** *assms(1)* **by** *force*

define u **where** *u-def*: $u = (snd\ y - snd\ x)\ /\ (snd\ z - snd\ x)$
have $snd\ y * (snd\ z - snd\ x) = (snd\ z - snd\ y) * snd\ x + (snd\ y - snd\ x) * snd\ z$ **by** *argo*
hence $snd\ y = ((snd\ z - snd\ y)\ /\ (snd\ z - snd\ x)) * snd\ x + \ u * snd\ z$ **unfolding** *u-def*
by (*smt* (*verit*, *ccfv-SIG*) *diff-divide-eq-iff times-divide-eq-left assms*(1) *1*)
hence $snd\ y = (1 - u) * snd\ x + u * snd\ z$ **unfolding** *u-def*
using *assms add-divide-distrib divide-eq-1-iff* **by** (*smt* (*verit*, *ccfv-SIG*) *1*)
hence $y = (1 - u) *_R x + u *_R z$ **using** *assms*
by (*smt* (*verit*, *best*) *add-diff-cancel-left' fst-diff fst-scaleR prod-eq-iff real-scaleR-def scaleR-collapse snd-add snd-scaleR*)
thus *?thesis* **using** *affine-comb-affine-dep assms*(1)
by (*smt* (*verit*, *best*) *add.commute distinct-length-2-or-more distinct-singleton insert-commute*)
qed

lemma *swap-coords*:
assumes $x = u *_R y + (1 - u) *_R (z :: R2)$
shows $prod.swap\ x = u *_R prod.swap\ y + (1 - u) *_R prod.swap\ z$
using *assms* **by** (*simp add*: *scaleR-prod-def*)

lemma *affine-swap*:
assumes *affine S*
shows *affine* (*prod.swap* ‘ *S*)
using *assms* **unfolding** *affine-def* **by** *force*

lemma *hull-swap-1*:
prod.swap ‘ (*affine hull s*) ⊆ *affine hull prod.swap* ‘ *s*
unfolding *hull-def*
proof
fix x **assume** $x \in prod.swap\ `\ \bigcap\ \{t.\ affine\ t \wedge s \subseteq t\}$
hence $x \in \bigcap \{prod.swap\ `\ t |\ t.\ affine\ t \wedge s \subseteq t\}$ **by** *force*
hence $\forall t.\ affine\ t \wedge s \subseteq t \longrightarrow x \in prod.swap\ `\ t$ **by** *blast*
hence $\forall t.\ affine\ (prod.swap\ `\ t) \wedge prod.swap\ `\ s \subseteq prod.swap\ `\ t \longrightarrow x \in prod.swap\ `\ t$
by (*metis* (*no-types*, *lifting*) *ext affine-swap image-ident image-image image-mono swap-swap*)
hence $\forall t.\ affine\ t \wedge prod.swap\ `\ s \subseteq t \longrightarrow x \in t$
by (*metis subset-imageE surj-swap top-greatest*)
hence $x \in \bigcap\ \{t.\ affine\ t \wedge prod.swap\ `\ s \subseteq t\}$ **by** *blast*
thus $x \in \bigcap\ \{t.\ affine\ t \wedge prod.swap\ `\ s \subseteq t\}$ **by** *blast*
qed

lemma *hull-swap-2*:
affine hull prod.swap ‘ *s* ⊆ *prod.swap* ‘ (*affine hull s*)
by (*simp add*: *affine-swap hull-minimal hull-subset image-mono*)

lemma *hull-swap*:

prod.swap ‘ (affine hull (s :: R2 set)) = affine hull prod.swap ‘ s
using *hull-swap-1 hull-swap-2* **by** *blast*

lemma *cross-affine-dep-aux3*:
assumes *affine-dependent (S :: R2 set)*
shows *affine-dependent (prod.swap ‘ S)*
using *swap-coords assms affine-swap hull-swap* **unfolding** *affine-dependent-def*
by (*smt* (*verit*) *Diff-insert-absorb image-iff image-insert mk-disjoint-insert swap-swap*)

lemma *cross-affine-dependent-aux1*:
assumes *(y1 − x1) ∗ (z2 − y2) − (z1 − y1) ∗ (y2 − x2) = 0*
and *distinct [(x1, x2), (y1, y2), (z1, z2)]*
and *y1 = x1*
shows *affine-dependent {(x1, x2), (y1, y2), (z1, z2) :: R2}*
proof−
have *(z1 − x1) ∗ (y2 − x2) = 0* **using** *assms* **by** *simp*
then have *1: z1 = x1 ∨ y2 = x2* **by** *simp*
then show *?thesis*
proof(*cases z1 = x1*)
case *True*
hence *distinct [x2, y2, z2]* **using** *assms distinct-singleton prod-eq-iff*
by (*smt* (*verit*, *del-insts*) *distinct-length-2-or-more vector-space-over-itself.scale-eq-0-iff*)
then show *?thesis* **using** *distinct-snd-case-aux*[*of (x1, x2) (y1, y2) (z1, z2)*]
True assms
by (*smt* (*verit*, *best*) *distinct-length-2-or-more distinct-snd-case-aux fst-conv*
insert-commute snd-conv)
next
case *False*
then have *(y1, y2) = (x1, x2)* **using** *1 assms prod-eq-iff* **by** *blast*
thus *?thesis* **using** *assms*(*2*) **by** *simp*
qed
qed

lemma *cross-affine-dependent*:
assumes *cross3 x y z = 0* **and** *distinct [x, y, z]*
shows *affine-dependent {x, y, z :: R2}*
proof(*cases fst y = fst x*)
case *True*
thus *?thesis* **using** *cross-affine-dependent-aux1 cross3-def assms*(*1*,*2*) *surjective-pairing*
by (*smt* (*verit*) *mult-eq-0-iff*)
next
case *False*
then show *?thesis*
proof(*cases snd y = snd x*)
case *True*
then show *?thesis*
using *cross-affine-dependent-aux1*[*of snd y snd x fst z fst y snd z fst x*]
assms(*1*,*2*)

cross3-def surjective-pairing cross-affine-dep-aux3[*of* (*prod.swap* ' {*x*, *y*, *z*})] *prod.swap-def*
by (*smt* (*verit*, *ccfv-SIG*) *distinct-length-2-or-more distinct-singleton image-empty*
image-insert mult-eq-0-iff swap-simp)
next
case *False*
assume *a1*: *fst y* ≠ *fst x*
define *u* **where** *u-def*: *u* = (*fst z* − *fst y*) / (*fst y* − *fst x*)
then have *fst z* = *fst y* − *u* ∗ *fst x* + *u* ∗ *fst y* **using** *False a1*
by (*simp add*: *nonzero-eq-divide-eq right-diff-distrib*)
then have *fst z* = (*1* + *u*) ∗ (*fst y*) − *u* ∗ *fst x*
by *argo*
then have *z1*:*fst z* = (− *u*) ∗ *fst x* + (*1* + *u*) ∗ (*fst y*)
by *simp*
have *u* = (*snd z* − *snd y*) / (*snd y* − *snd x*) **using** *u-def assms a1*
by (*simp add*: *False cross3-def frac-eq-eq mult.commute*)
then have *snd z* = *snd y* − *u* ∗ *snd x* + *u* ∗ *snd y* **using** *False a1*
by (*simp add*: *nonzero-eq-divide-eq right-diff-distrib*)
then have *snd z* = (−*u*) ∗ *snd x* + (*1* + *u*) ∗ *snd y*
by *argo*
then have *z* = (− *u*) ∗$_R$ *x* + (*1* + *u*) ∗$_R$ *y* **using** *assms a1 False z1*
by (*metis* (*mono-tags*, *lifting*) *add.commute add-left-cancel diff-add-cancel fst-diff*
fst-scaleR pair-scal real-scaleR-def scaleR-one snd-diff snd-scaleR)
then show *affine-dependent* {*x*, *y*, *z* :: *R2*}
by (*smt* (*verit*, *del-insts*) *add-diff-cancel-left' affine-comb-affine-dep assms*(*2*) *diff-add-cancel distinct-length-2-or-more insert-commute*)
qed
qed

lemma *affinedep-cases*:
assumes *affine-dependent* {*x*, *y*, *z* :: *real* × *real*}
shows *x* ∈ *affine hull* {*y*, *z*} ∨ *y* ∈ *affine hull* {*x*, *z*} ∨ *z* ∈ *affine hull* {*x*, *y*}
using *assms* **unfolding** *affine-dependent-def hull-redundant-eq*
by (*smt* (*verit*, *ccfv-SIG*) *Diff-insert-absorb insert-absorb insert-commute insert-not-empty*)

lemma *cross-affine-dependent-conv*:
assumes *affine-dependent* {*x*, *y*, *z* :: *real* × *real*}
shows *cross3 x y z* = *0*
using *affinedep-cases assms cross3-commutation-12*
cross3-commutation-23 in-hull3-iscollinear
by *blast*

lemma *genpos-cross0*:
assumes *general-pos S* **and** *x* ∈ *S* **and** *y* ∈ *S* **and** *z* ∈ *S* **and** *distinct* [*x*, *y*, *z*]
shows *cross3 x y z* ≠ *0*
using *assms nsubset-def general-pos-def cross-affine-dependent*

by (*smt* (*verit*, *del-insts*) *add.right-neutral add-Suc-right distinct-card empty-set empty-subsetI insert-subset list.simps*(15) *list.size*(3,4) *mem-Collect-eq numeral-3-eq-3*)

lemma *coll-is-affDep*:
distinct [*a*,*b*,*c*] ⟶ (*collinear3 a b c* ⟷ *affine-dependent* {*a*, *b*, *c*})
using *collinear3-def cross3-commutation-23 cross3-non0-distinct cross-affine-dependent cross-affine-dependent-conv* **by** *blast*

lemma *distinct-is-triple*:
a∈*S* ∧ *b*∈*S* ∧ *c*∈*S* ∧ *distinct* [*a*,*b*,*c*] ⟷ {*a*,*b*,*c*} ∈ *S* ~ 3
by (*smt* (*verit*, *del-insts*) *distinct.simps*(2) *distinct-card distinct-singleton empty-set empty-subsetI insert-absorb insert-subset length-Cons lessI list.set*(2) *list.size*(3) *mem-Collect-eq not-less-iff-gr-or-eq nsubset-def numeral-3-eq-3*)

definition
gpos S ≡ ∀ *a*∈*S*. ∀ *b*∈*S*. ∀ *c*∈*S*. *distinct* [*a*,*b*,*c*] ⟶ ¬ *collinear3 a b c*

lemma *gpos-generalpos*:
gpos S ⟷ *general-pos S*
using *coll-is-affDep distinct-is-triple nsubset-def*[*of S* 3] **unfolding** *gpos-def general-pos-def*
by (*smt* (*verit*, *ccfv-threshold*) *card-3-iff mem-Collect-eq*)

definition *v2* :: *real* ⇒ *R2* **where** *v2* ≡ λ*a*. (*a*, *a* ∗ *a*)
lemma *f-prop0*: *cross3* (*v2 a*) (*v2 b*) (*v2 c*) = (*a* − *b*) ∗ (*c* − *a*) ∗ (*b* − *c*)
unfolding *collinear3-def cross3-def v2-def* **by** (*simp*, *argo*)
lemma *f-prop1*: ∀ *x y*. *x* ≠ *y* ⟷ *v2 x* ≠ *v2 y* **using** *v2-def fst-conv* **by** *metis*
lemma *f-prop2*: *distinct* [*a*,*b*,*c*] ⟶ *distinct*[*v2 a*, *v2 b*, *v2 c*] **using** *f-prop1* **by** *auto*
lemma *f-prop3*: *distinct* [*a*,*b*,*c*] ⟶ ¬ *collinear3* (*v2 a*) (*v2 b*) (*v2 c*)
using *f-prop2 f-prop0* **unfolding** *collinear3-def cross3-def v2-def* **by** *auto*
lemma *f-prop4*: *sortedStrict* [*a*,*b*,*c*] ⟶ *cup3* (*v2 a*) (*v2 b*) (*v2 c*)
using *f-prop0 strict-sorted-iff* **unfolding** *cup3-def*
by (*smt* (*verit*, *ccfv-threshold*) *distinct-length-2-or-more mult-eq-0-iff sorted2 zero-less-mult-iff*)
lemma *f-prop5*: *sortedStrict* (*rev* [*a*,*b*,*c*]) ⟶ *cap3* (*v2 a*) (*v2 b*) (*v2 c*)
using *f-prop4 f-prop3 exactly-one-true*
by (*smt* (*verit*, *best*) *append.simps*(1,2) *cup3-def distinct-rev f-prop0 rev.simps*(2) *singleton-rev-conv sorted2 strict-sorted-iff zero-less-mult-iff*)
lemma *f-prop6*: *sortedStrict S* ⟶ *gpos* (*v2* ‘ *set S*)
by (*smt* (*verit*, *ccfv-SIG*) *collinear3-def distinct-length-2-or-more f-prop0 gpos-def image-iff mult-eq-0-iff*)
lemma *f-prop7*: (*a* :: *real*) > *0* ⟶ ((*a* < *b*) ⟶ (*a*∗*a* < *b*∗*b*)) **by** (*simp add*: *mult-strict-mono*)
lemma *f-prop8*: ((*a* > *0* ∧ (*b*::*real*) > *0*) ⟶ (*a*∗*a* < *b*∗*b*) ⟶ (*a* < *b*)) **by** (*metis antisym-conv3 f-prop7 order-less-asym*′)

lemma *f-prop9*: *sortedStrict*[*a*,*b*] ⟷ *sortedStrict*[*v2 a*, *v2 b*] **using** *f-prop7 v2-def strict-sorted-iff*
by (*metis* (*lifting*) *distinct-length-2-or-more less-prod-simp linorder-not-less not-less-iff-gr-or-eq sorted2 sorted-wrt1*)
lemma *f-prop10*: *sortedStrict* [*a*,*b*,*c*] ⟷ *sdistinct*[*v2 a*, *v2 b*, *v2 c*] **using** *f-prop9*
by (*smt* (*verit*, *best*) *distinct-length-2-or-more distinct-map fst-conv less-prod-simp linorder-not-less list.simps*(*8*,*9*) *sorted2 strict-sorted-iff v2-def*)
lemma *f-prop11*: *sortedStrict L* ⟶ *sdistinct* (*map v2 L*)
by (*smt* (*verit*, *del-insts*) *fst-conv imageE less-prod-simp list.set-map nless-le sorted-wrt-map-mono strict-sorted-iff v2-def*)
lemma *f-prop12*: *sdistinct* (*map v2 L*) ⟹ *cup* (*length* (*map v2 L*)) (*map v2 L*)
proof(*induction L*)
case (*Cons a L*)
then show *?case*
proof(*cases length* (*map v2* (*a*#*L*)) ≤ *2*)
case *False*
then obtain *x y z rest* **where** *aL*: *a*#*L* = *x*#*y*#*z*#*rest*
by (*metis One-nat-def Suc-1 Suc-le-length-iff le-SucE length-Cons length-map nle-le*)
hence *sdistinct* [*v2 x*,*v2 y*,*v2 z*]
using *Cons.prems* **by** *auto*
hence *cup3* (*v2 x*) (*v2 y*) (*v2 z*) **using** *f-prop4 f-prop10* **by** *presburger*
then show *?thesis* **using** *Cons aL cup-def* **by** *force*
qed(*metis lcheck-len2-T Cons*(*2*) *cup-def*)
qed(*simp add*: *cup-def*)

thm *card-le-inj*

lemma *real-set-ex*: ∃ (*S*:: *real set*). *card S* = *n*
using *infinite-UNIV-char-0 infinite-arbitrarily-large* **by** *blast*

lemma *genpos-ex*:
∃ *S*. *gpos S* ∧ *card S* = *n* ∧ (∃ *Sr* :: *real set*. *S* = (*v2* ‘ *Sr*)) ∧ *sdistinct*(*sorted-list-of-set S*)
proof−
obtain *X* **where** *rset*: *card* (*X* :: *real set*) = *n* **using** *real-set-ex* **by** *blast*
hence *1*:*card* (*v2* ‘ *X*) = *n* **by** (*metis card-image f-prop1 inj-onI*)
have *gpos* (*v2* ‘ *X*) **using** *f-prop3 gpos-def*
by (*smt* (*verit*, *best*) *distinct-length-2-or-more distinct-singleton image-iff*)
thus *?thesis* **using** *1 rset f-prop10*
by (*metis card-image distinct-map f-prop11 f-prop6 list.set-map*
sorted-list-of-set.idem-if-sorted-distinct sorted-list-of-set.length-sorted-key-list-of-set
sorted-list-of-set.sorted-sorted-key-list-of-set
sorted-list-of-set.strict-sorted-key-list-of-set)
qed

end
theory *Invariants*

imports *Definitions GeneralPosition3*

begin

abbreviation
reflect ≡ (λ *p*. −(*p* :: *R2*))

lemma *subseq-neg*:
subseq ys xs ⟹ *subseq* (*map reflect ys*) (*map reflect xs*)
using *subseq-map* **by** *blast*

lemma *cross3-neg*:
cross3 x y z = *cross3* (−*x*) (−*y*) (−*z*)
unfolding *cross3-def*
by (*simp add*: *left-diff-distrib′ right-diff-distrib′*)

lemma *slope-neg*:
slope x y < *slope y z* ⟹ *slope* (−*x*) (−*y*) < *slope* (−*y*) (−*z*)
unfolding *slope-def*
by (*metis* (*no-types*, *opaque-lifting*) *fst-uminus minus-diff-minus*
minus-divide-left minus-divide-right snd-uminus
verit-minus-simplify(*4*))

lemma *gpos-neg*:
assumes *gpos S*
shows *gpos* (*reflect* ‘ *S*)
proof−
{
fix *a b c*
assume *asm*: *a*∈*S b*∈*S c*∈*S distinct* [*a*,*b*,*c*]
hence *1*:¬ *collinear3* (*a*) (*b*) (*c*) **using** *gpos-def assms asm* **by** *simp*
have *2*: *distinct* [−*a*, −*b*, −*c*] **using** *asm* **by** *simp*
have *3*:¬*collinear3* (−*a*) (−*b*) (−*c*) **using** *1* **unfolding** *collinear3-def cross3-def*
by (*simp add*: *mult.commute vector-space-over-itself*.*scale-right-diff-distrib*)
have *distinct* [−*a*, −*b*, −*c*] ⟶ ¬*collinear3* (−*a*) (−*b*) (−*c*) **using** *2 3* **by** *simp*
}
thus *?thesis* **unfolding** *gpos-def* **by** *simp*
qed

corollary *general-pos-neg*:
general-pos S ⟶ *general-pos* (*reflect* ‘ *S*)
using *gpos-generalpos gpos-neg* **by** *blast*

corollary *general-pos-neg-neg*:
general-pos (*reflect* ‘ *S*) ⟶ *general-pos S* **using** *general-pos-neg*
by (*metis* (*no-types*, *opaque-lifting*) *general-pos-subs minus-equation-iff*
rev-image-eqI subset-eq)

lemma *card-neg*:

(*card S = n*) = (*card* (*reflect* ‘ (*S* :: *R2 set*)) = *n*) **using** *card-def*
by (*simp add*: *card-image*)

lemma *length-neg*:
length xs = *length* (*map reflect xs*) **by** *simp*

lemma *length-rev-neg*:
length xs = *length* (*rev* (*map reflect xs*)) **by** *simp*

lemma *neg-neg*:
reflect ∘ *reflect* = *id* **by** *simp*

lemma *rev-reflect*:
rev ∘ (*map reflect*) = (*map reflect*) ∘ *rev*
by (*simp add*: *comp-def rev-map*)

lemma *rev-reflect-inv*:
(*rev* ∘ (*map reflect*)) ∘ (*rev* ∘ (*map reflect*)) = (*id* :: *R2 list* ⇒ *R2 list*)
using *rev-reflect neg-neg*
by (*metis List.map.comp comp-assoc comp-id list.map-id0 rev-involution*)

lemma *implm-rev-reflect-inv*:*rev* (*map reflect* (*rev* (*map reflect xs*))) = *xs*
proof(*induction xs*)
case (*Cons a xs*)
have *rev* (*map reflect* (*a* # *xs*)) = (*rev* (*map reflect xs*))@[*reflect a*]
by *simp*
also have *map reflect* ((*rev* (*map reflect xs*))@[*reflect a*])
= (*map reflect* (*rev* (*map reflect xs*)))@ [*a*]
by *simp*
also have *rev* ((*map reflect* (*rev* (*map reflect xs*)))@[*a*]) = [*a*]@(*rev* (*map reflect* (*rev* (*map reflect xs*))))
by *simp*
ultimately show *?case* **using** *Cons* **by** *simp*
qed(*simp*)

lemma *distinct-neg*:
distinct xs = *distinct* (*rev* (*map reflect* (*xs* :: *R2 list*)))
by (*simp add*: *distinct-map*)

lemma *sorted-neg*:
sorted xs = *sorted* (*rev* (*map reflect* (*xs* :: *R2 list*)))
(**is** *?P xs* = *?Q xs*)
proof
assume *asm*:*?P xs*
hence *1*:*sorted-wrt* (≥) (*rev xs*)
by (*simp add*: *sorted-wrt-rev*)
have *Fact*:∀ *x y*. *x* ≤ *y* ⟶ *reflect x* ≥ *reflect y* **by** *simp*
hence *2*:*sorted-wrt* (≥) (*map reflect xs*)
by (*metis asm sorted-wrt-map-mono*)

```
  thus ?Q xs using asm 1 sorted-wrt-rev
    by blast
next
  have Fact:∀ x y. reflect x ≥ reflect y ⟶ x ≤ y  by simp
  assume asm:?Q xs
  thus ?P xs using sorted-wrt-rev sorted-wrt-map-mono Fact
    by (smt (verit, ccfv-SIG) sorted-wrt-map sorted-wrt-mono-rel)
qed

lemma sortedstrict-neg:
  sortedStrict xs = sortedStrict (rev (map reflect (xs :: R2 list)))
  using distinct-neg sorted-neg strict-sorted-iff by blast

lemma orig-refl-rev:
  cap3 x y z = cup3 (−z) (−y) (−x)
  unfolding cap3-def cup3-def cross3-def by fastforce

lemma list-check-cup3-imp-cap3:
  assumes list-check cup3 xs
  shows list-check cap3 (rev (map reflect xs)) using assms
proof(induction xs)
  case Nil
  then show ?case by auto
next
  case (Cons a xs)
  have lcc:list-check cup3 (a # xs) using Cons by argo
  then show ?case
  proof(cases xs)
    case Nil
    then show ?thesis by simp
  next
    case (Cons b ys)
    hence b-ys: xs = b#ys by argo
    then show ?thesis
    proof(cases ys)
      case Nil
      have a ≠ b using  lcc unfolding Nil Cons lcc list-check.simps .
      then show ?thesis unfolding Nil Cons lcc list-check.simps by simp
    next
      case (Cons c zs)
      have 1:(rev (map reflect (a # b # c # zs)))
              = (rev (map reflect zs))@([reflect c, reflect b, reflect a])
        by simp
      have cross3 a b c > 0 using lcc unfolding Cons b-ys lcc
        by (meson cup3-def list-check.simps(4))
      hence cross3 (reflect c) (reflect b) (reflect a) < 0
        by (simp add: cross3-def)

      then show ?thesis unfolding lcc  implm-rev-reflect-inv
```

```
      unfolding b-ys   Cons using
        list-cap-tail[of (rev (map reflect zs)) reflect c reflect b reflect a]
          1
       by (metis Cons.IH Cons.prems Cons-eq-append-conv b-ys list.simps(9)
list-check.simps(4)
        local.Cons rev.simps(2) rev-append rev-swap)
   qed
  qed
qed


lemma list-check-cap3-imp-cup3:
  assumes list-check cap3 xs
  shows list-check cup3 (rev (map reflect xs)) using assms
 using assms
proof(induction xs)
  case Nil
  then show ?case by auto
next
  case (Cons a xs)
  have lcc:list-check cap3 (a # xs) using Cons by argo
  then show ?case
  proof(cases xs)
    case Nil
    then show ?thesis by simp
  next
    case (Cons b ys)
    hence b-ys: xs = b#ys by argo
    then show ?thesis
    proof(cases ys)
      case Nil
      have a ≠ b using  lcc unfolding Nil Cons lcc list-check.simps .
      then show ?thesis unfolding Nil Cons lcc list-check.simps by simp
    next
      case (Cons c zs)
      have 1:(rev (map reflect (a # b # c # zs)))
              = (rev (map reflect zs))@([reflect c, reflect b, reflect a])
        by simp
      have cross3 a b c < 0 using lcc unfolding Cons b-ys lcc
        by (meson cap3-def list-check.simps(4))
      hence cross3 (reflect c) (reflect b) (reflect a) > 0
        by (simp add: cross3-def)
      then show ?thesis unfolding lcc  implm-rev-reflect-inv
        unfolding b-ys   Cons using
          list-cap-tail[of (rev (map reflect zs)) reflect c reflect b reflect a]
            1
         by (smt (verit, ccfv-threshold) Cons.IH Cons.prems(1) append.assoc ap-
pend-Cons append-Nil
          b-ys list.simps(9) list-check.simps(4) list-cup-tail local.Cons rev.simps(2))
```

```
    qed
  qed
qed

value map uminus [1,2,3::real]

lemma sdistinct-rev-map-refl1:
  assumes sdistinct L
  shows   sdistinct (rev (map reflect L))
proof−
  have 1:distinct (map fst L) using assms by simp
  have 2:distinct (map reflect L) using assms
    by (simp add: distinct-map)
  hence distinct (map fst (map reflect L)) using assms 1 2
    by (metis (no-types, lifting) ext comp-def distinct-map fst-uminus inj-uminus
list.map-comp)
  thus sdistinct (rev (map reflect L)) using assms
    by (metis distinct-map distinct-rev set-rev sorted-neg)
qed

lemma sdistinct-rev-map-refl:
  sdistinct L ⟷ sdistinct (rev (map reflect L))
  using sdistinct-rev-map-refl1
  by (metis implm-rev-reflect-inv)

lemma sdistinct-refl-set:
  sdistinct(sorted-list-of-set S) ⟷ sdistinct (sorted-list-of-set (reflect ‘ S))
  using sdistinct-rev-map-refl
proof −
  have 0 = card S ⟶ sdistinct (sorted-list-of-set S) = sdistinct (sorted-list-of-set
(reflect ‘ S)) ∨ distinct (map fst (sorted-list-of-set (reflect ‘ S))) ∧ distinct (map
fst (sorted-list-of-set S))
    by (metis bot-nat-0.extremum card-neg card-seteq empty-subsetI sorted-list-of-set.fold-insort-key.infinite
sorted-list-of-set.sorted-key-list-of-set-empty)
  then show ?thesis
    by (metis (no-types) card.infinite card-neg distinct-map list.set-map sdistinct-rev-map-refl
set-rev sorted-list-of-set.distinct-sorted-key-list-of-set sorted-list-of-set.set-sorted-key-list-of-set
sorted-list-of-set.sorted-sorted-key-list-of-set sortedstrict-neg strict-sorted-iff)
qed

lemma cap-orig-refl-rev:
  cap k xs = cup k (rev (map reflect (xs::R2 list)))
proof(induction xs arbitrary: k)
  case Nil
  then show ?case unfolding cap-def cup-def
    by simp
next
  case (Cons a xs)
  { assume asm:cap k (a # xs)
```

hence *sdistinct* (*a*#*xs*) **unfolding** *cap-def* **by** *argo*
hence *1*:*sdistinct* (*rev* (*map reflect* (*a* # *xs*))) **using** *sdistinct-rev-map-refl* **by** *blast*
have *2*:*length* (*rev* (*map reflect* (*a* # *xs*))) = *k* **using** *asm* **unfolding** *cap-def*
by *auto*
also have *list-check cap3* (*a*#*xs*) **using** *asm* **unfolding** *cap-def* **by** *argo*
hence *list-check cup3* (*rev* (*map reflect* (*a*#*xs*)))
using *list-check-cap3-imp-cup3* **by** *blast*
hence *cup k* (*rev* (*map reflect* (*a*#*xs*)))
using *1 2 asm* **unfolding** *cup-def cap-def* **by** *argo*} **note** *rs* = *this*
then have *s1*:*cap k* (*a*#*xs*) ⟶ *cup k* (*rev* (*map reflect* (*a*#*xs*))) **by** *argo*
have *cup k* (*rev* (*map reflect* (*a*#*xs*))) ⟶ (*cap k* (*a*#*xs*))
proof
assume *asm*:*cup k* (*rev* (*map reflect* (*a*#*xs*)))
hence *sdistinct* (*rev* (*map reflect* (*a*#*xs*))) **unfolding** *cup-def* **by** *argo*
hence *1*:*sdistinct* (*a* # *xs*) **using** *sdistinct-rev-map-refl* **by** *blast*
have *2*:*length* (*a* # *xs*) = *k* **using** *asm* **unfolding** *cup-def*
by *auto*
also have *3*:*list-check cup3* (*rev* (*map reflect* (*a*#*xs*)))
using *asm* **unfolding** *cup-def* **by** *argo*
hence *list-check cap3* (*a*#*xs*)
proof−
have *list-check cap3* (*rev* (*map reflect* (*rev* (*map reflect* (*a*#*xs*)))))
using *list-check-cup3-imp-cap3*[*OF 3*] **.**
also have (*rev* (*map reflect* (*rev* (*map reflect* (*a*#*xs*))))) = *a*#*xs*
using *implm-rev-reflect-inv*[*of a*#*xs*] **.**
ultimately show *?thesis* **by** *argo*
qed
thus *cap k* (*a*#*xs*)
using *1 2 asm* **unfolding** *cup-def cap-def* **by** *argo*
qed
then show *?case* **using** *s1* **by** *argo*
qed

lemma *min-conv-sets-eq*:
shows {*n* :: *nat*. (∀ *S* :: *R2 set*. (*card S* ≥ *n* ∧ *general-pos S* ∧ *sdistinct*(*sorted-list-of-set S*))
⟶ (∃ *xs*. *set xs* ⊆ *S* ∧ *sdistinct xs* ∧ (*cap k xs* ∨ *cup l xs*)))}
= {*n* :: *nat*. (∀ *S* :: *R2 set*. (*card S* ≥ *n* ∧ *general-pos S* ∧ *sdistinct*(*sorted-list-of-set S*))
⟶ (∃ *xs*. *set xs* ⊆ *S* ∧ *sdistinct xs* ∧ (*cap l xs* ∨ *cup k xs*)))}
(**is** *?MCS k l* = *?MCS l k*)
proof−

have *?MCS l k* ⊆ *?MCS k l* **for** *k l*
proof
fix *x*
assume *x-in*:*x* ∈ *?MCS l k*

hence $\forall S :: R2\ set.\ card\ S \geq x \wedge general\text{-}pos\ S \wedge sdistinct(sorted\text{-}list\text{-}of\text{-}set\ S)$
$\longrightarrow (\exists xs.\ set\ xs \subseteq (reflect\ `\ S) \wedge sdistinct\ xs \wedge (cap\ k\ xs \vee cup\ l\ xs))$
by (*smt* (*verit*, *best*) *cap-def cap-orig-refl-rev cup-def general-pos-neg-neg id-apply image-mono image-set mem-Collect-eq o-def rev-reflect-inv set-rev*)

hence $\forall S :: R2\ set.(\ card\ S \geq x \wedge general\text{-}pos\ S \wedge sdistinct(sorted\text{-}list\text{-}of\text{-}set\ S))$
$\longrightarrow (\exists xs.\ set\ xs \subseteq S \wedge sdistinct\ xs \wedge (cap\ k\ xs \vee cup\ l\ xs))$
using *card-neg general-pos-neg neg-neg sdistinct-refl-set*
by (*metis* (*no-types*, *lifting*) *eq-id-iff image-comp image-ident*)

thus $x \in ?MCS\ k\ l$ **by** *blast*
qed

thus *?thesis* **by** *blast*

qed

lemma *min-conv-arg-swap*: $min\text{-}conv\ k\ l = min\text{-}conv\ l\ k$
unfolding *min-conv-def* **using** *min-conv-sets-eq* **by** *simp*

lemma *convex-pos-rev*:
$convex\text{-}pos\ (set\ xs) \Longrightarrow convex\text{-}pos\ (set\ (rev\ xs))$
using *set-rev* **by** *simp*

lemma *hull-refl*:
assumes $\forall t.\ S\ t = S\ (reflect\ `\ t)$
shows $reflect\ `\ (S\ hull\ s) = S\ hull\ (reflect\ `\ s)$
proof−
have *1*:$(x \in S\ hull\ s) \longleftrightarrow (\forall t.\ S\ t \wedge s \subseteq t \longrightarrow x \in t)$ **for** x **unfolding** *hull-def* **by** *blast*
have *2*:$(\forall t.\ S\ t \wedge s \subseteq t \longrightarrow x \in t) \longleftrightarrow (\forall t.\ S\ t \wedge reflect\ `\ s \subseteq t \longrightarrow reflect\ x \in t)$ **for** x
by (*metis* (*no-types*, *opaque-lifting*) *add.inverse-inverse assms id-apply imageI image-comp*
image-id image-mono neg-neg)
have *3*:$(\forall t.\ S\ t \wedge reflect\ `\ s \subseteq t \longrightarrow reflect\ x \in t) \longleftrightarrow (x \in reflect\ `\ (S\ hull\ (reflect\ `\ s)))$ **for** x **unfolding** *hull-def*
by (*smt* (*verit*, *best*) *InterE InterI image-iff mem-Collect-eq minus-equation-iff*)
have $(S\ hull\ s) = reflect\ `\ (S\ hull\ (reflect\ `\ s))$ **using** *1 2 3* **by** *auto*
thus *?thesis*
by (*metis* (*no-types*, *lifting*) *eq-id-iff image-comp*
image-ident neg-neg)
qed

lemma *convex-pos-refl*:

assumes *convex-pos S*
shows *convex-pos* (*reflect* ‘ *S*)
using *assms* **unfolding** *convex-pos-def*
proof−
assume *asm*:∀ *s*∈*S*. *convex hull S* ≠ *convex hull* (*S* − {*s*})
have *1*:*s*∈*S* ⟶ *convex hull reflect* ‘ *S* = *reflect* ‘ (*convex hull S*) **for** *s* **using** *hull-refl*
by (*simp add*: *linear-uminus*)
have *2*:*s*∈*S* ⟶ *reflect* ‘ (*convex hull S*) ≠ (*reflect* ‘ (*convex hull* (*S* − {*s*}))) **for** *s*
using *asm* **by** (*metis* (*no-types*, *lifting*) *eq-id-iff image-comp image-ident neg-neg*)
have *3*:*s*∈*S* ⟶ (*reflect* ‘ (*convex hull* (*S* − {*s*}))) = (*convex hull* (*reflect* ‘ (*S* − {*s*}))) **for** *s*
using *hull-refl* **by** (*simp add*: *linear-uminus*)
have *4*:*s*∈*S* ⟶ (*convex hull* (*reflect* ‘ (*S* − {*s*}))) = *convex hull* (*reflect* ‘ *S* − {*reflect s*}) **for** *s*
by (*smt* (*verit*) *Diff-insert-absorb 1 2 3*
image-insert insert-Diff insert-absorb)
have *res*:*s*∈*S* ⟶ *convex hull reflect* ‘ *S* ≠ *convex hull* (*reflect* ‘ *S* − {*reflect s*}) **for** *s*
using *1 2 3 4*
by *metis*
have *s* ∈ *S* ⟶ *reflect s* ∈ *reflect* ‘ *S* **for** *s* **by** (*simp add*: *linear-uminus*)
hence *reflect s* ∈ *reflect* ‘ *S*
⟶ *convex hull reflect* ‘ *S* ≠ *convex hull* (*reflect* ‘ *S* − {*reflect s*}) **for** *s*
using *res* **by** (*simp add*: *image-iff*)
thus ∀ *s*∈*reflect* ‘ *S*. *convex hull reflect* ‘ *S* ≠ *convex hull* (*reflect* ‘ *S* − {*s*}) **by** *blast*
qed

lemma *convex-pos-list-refl*:
assumes *convex-pos* (*set xs*)
shows *convex-pos* (*set* (*map reflect xs*))
using *convex-pos-refl assms* **by** *force*

lemma *bij-tr*:
bij (λ *p*. *p* + (*t*::*R2*))
by (*simp add*: *bij-plus-right*)

lemma *tr-refl-inv*:
map (λ*p*. *p* + *reflect ta*) (*map* (λ*p*. *p* + *ta*) *ps*) = *ps*
by (*simp add*: *comp-def*)

lemma *prod-addright-le*:
fixes *a s t* :: *R2*
assumes *a* ≤ *s*

shows $a + t \leq s + t$
proof(*cases fst a < fst s*)
case *True*
hence *fst* $(a + t) <$ *fst* $(s + t)$ **by** *simp*
then show *?thesis*
by (*smt* (*verit*, *best*) *assms fst-add prod-le-def snd-add*)
next
case *False*
hence *I8*:*fst a* = *fst s* **using** *assms*
by (*simp add*: *prod-le-def*)
hence *snd a* ≤ *snd s* **using** *assms prod-le-def False* **by** *blast*
hence *snd* $(a + t) \leq$ *snd* $(s + t)$ **by** *simp*
then show *?thesis*
by (*auto simp add*: *I8 less-eq-prod-def*)
qed

lemma *distinct-fst-translated*:
fixes $L ::$ *R2 list* **and** $t ::$ *R2*
assumes *distinct* (*map fst L*) **and** $tr \equiv (\lambda p.\ p + t)$
shows *distinct* (*map fst* (*map tr L*))
using *assms*
proof(*induction L*)
case (*Cons a L*)
have *distinct* (*map fst L*)
using *Cons.hyps*(*2*) **by** *auto*
hence *distinct* (*map fst* (*map tr L*)) **using** *Cons.hyps* **by** *simp*
have *F1*:*fst a* ∉ *set* (*map fst L*) *distinct* (*map fst L*) **using** *Cons.hyps*(*2*) **by** *auto*
have *fst* (*a*+*t*) ∉ *set* (*map fst* (*map tr L*))
proof
assume *asm*: *fst* $(a + t) \in$ *set* (*map fst* (*map tr L*))
then obtain $x ::$ *R2* **where** *xp*:*fst* (*x*+*t*)∈*set*(*map fst*(*map tr L*)) *fst* (*x*+*t*)=*fst*(*a*+*t*)
by *force*
hence *fst x* = *fst a* **by** *simp*
hence *fst* (*a*+*t*) ∈ *set*(*map fst*(*map tr L*)) **using** *xp*(*1*) **by** *simp*
hence *fst a* ∈ *set*(*map fst L*)
by (*smt* (*verit*, *ccfv-threshold*) *assms*(*2*) *fst-add imageE image-eqI list.set-map*)
thus *False* **using** *F1*(*1*) **by** *simp*
qed
then show *?case* **using** *Cons.hyps* **by** *simp*
qed(*simp*)

lemma *translated-sdistinct*:
assumes *sdistinct L*
shows *sdistinct* (*map* $(\lambda p.\ p+t)$ *L*) **using** *assms distinct-fst-translated*
by (*metis prod-addright-le sorted-wrt-map-mono*)

lemma *map-sorted-list-set*:
fixes $t :: R2$
shows *map* (λ *p*. *p* + *t*) (*sorted-list-of-set S*) = *sorted-list-of-set* ((λ *p*. *p* + *t*) ‘ *S*)
proof(*induction sorted-list-of-set S arbitrary*: *S*)
case *Nil*
then show *?case* **using** *bij-tr*
by (*metis* (*no-types*, *lifting*) *bij-def card-seteq card-vimage-inj dual-order.refl empty-subsetI top.extremum inj-vimage-image-eq list.simps*(*8*) *sorted-list-of-set.fold-insort-key.infinite sorted-list-of-set.length-sorted-key-list-of-set sorted-list-of-set.sorted-key-list-of-set-empty*)
next
case (*Cons a x*)
hence *I1*:*x* = *sorted-list-of-set* (*S* − {*a*})
by (*metis list.inject neq-Nil-conv sorted-list-of-set.fold-insort-key.infinite sorted-list-of-set.sorted-key-list-of-set-eq-Nil-iff sorted-list-of-set-nonempty*)
hence *I2*:*map* (λ *p*. *p* + *t*) (*sorted-list-of-set* (*S*−{*a*})) = *sorted-list-of-set* ((λ *p*. *p* + *t*) ‘ (*S*−{*a*})) **using** *Cons.hyps*(*1*,*2*) **by** *presburger*
have *I4*: *a* = *Min S*
by (*metis Cons.hyps*(*2*) *list.inject neq-Nil-conv sorted-list-of-set.fold-insort-key.infinite sorted-list-of-set.sorted-key-list-of-set-empty sorted-list-of-set-nonempty*)
hence *a* ∈ *S*
by (*metis Cons.hyps*(*2*) *Diff-insert-absorb I1 insert-Diff1 not-Cons-self2 singleton-iff*)
hence *I5*:*a* + *t* ∈ (λ*s*. *s* + *t*) ‘ *S* **by** *simp*
have *I6*:∀ *s*∈*S*. *a*≤*s* **using** *I4*
by (*metis Cons.hyps*(*2*) *Min.coboundedI neq-Nil-conv sorted-list-of-set.fold-insort-key.infinite*)
hence ∀ *s*∈*S*. *a* + *t* ≤ *s* + *t* **using** *prod-addright-le* **by** *simp*
hence ∀ *s*∈((λ*s*. *s* + *t*) ‘ *S*). *a* + *t* ≤ *s* **by** *simp*
hence *I3*:*a* + *t* = *Min* ((λ *p*. *p* + *t*) ‘ *S*) **using** *I5*
by (*metis Cons.hyps*(*2*) *I1 Min-eqI finite-imageI infinite-remove not-Cons-self sorted-list-of-set.fold-insort-key.infinite*)
then show *?case*
by (*smt* (*z3*) *Cons.hyps*(*2*) *Diff-empty Diff-insert0 Diff-insert-absorb I2 add-right-cancel empty-not-insert finite-imageI insert-absorb list.inject list.set*(*1*,*2*) *list.set-map list.simps*(*8*,*9*) *sorted-list-of-set.fold-insort-key.infinite sorted-list-of-set.set-sorted-key-list-of-set sorted-list-of-set-nonempty*)
qed

lemma *translated-cross3*:
shows *cross3 a b c* = *cross3* (*a*+*t*) (*b*+*t*) (*c*+*t*)
unfolding *cross3-def* **by** *simp*

lemma *translated-cup*:
assumes *cup k xs*
shows *cup k* (*map* (λ*p*. *p* + *t*) *xs*) **using** *assms*
proof(*induction xs arbitrary*: *k*)
case (*Cons a xs*)

```
  hence cup (k−1) xs using cup-reduct
   by (metis Suc-eq-plus1 Suc-pred′ bot-nat-0.not-eq-extremum cup-def length-0-conv
       neq-Nil-conv)
  hence F1:cup (k−1) (map (λp. p + t) xs)
    using Cons.IH by blast
  have F2: length (a#xs) = k using Cons(2) cup-def by simp
  hence F3: length (map (λp. p + t) (a#xs)) = k by simp
  have F4: sdistinct (map (λp. p + t) (a#xs)) using translated-sdistinct Cons(2)
cup-def
    by blast
  then show ?case
  proof(cases length (map (λp. p+t) (a#xs)) ≤ 2)
    case True
    then show ?thesis using F3 lcheck-len2-T
      using F4 cup-def by blast
  next
    case False
    hence length (a#xs) ≥ 3 by simp
    hence ∃ a u v rest. (a#xs) = (a#u#v#rest)
    by (metis One-nat-def Suc-eq-plus1 list.size(3,4) neq-Nil-conv nle-le not-less-eq-eq
        numeral-3-eq-3)
    then obtain u v rest where xsp: xs = u#v#rest by blast
    hence cup3 a u v list-check cup3 (u#v#rest)
      using assms cup-def Cons.prems by auto
    hence F2:cup3 (a+t) (u+t) (v+t) using translated-cross3
      by (metis cup3-def)
    have list-check cup3 (map (λp. p+t) (u#v#rest))
      using F1 cup-def xsp by blast
    then show ?thesis using F2
      using F3 F4 cup-def list.simps(9) list-check.simps(4) xsp by force
  qed
qed(simp)

lemma translated-cup-eq:
  ⋀t. cup k xs = cup k (map (λp. p + t) xs)
proof
  define ys where ysp: ys ≡ map (λp. p − t) xs
  have cup k ys ⟹ cup k (map (λp. p + t) ys) using translated-cup by simp
  hence cup k (map (λp. p − t) xs) ⟹ cup k xs
    by (simp add: o-def ysp)
  thus ⋀t. cup k (map (λp. p + t) xs) ⟹ cup k xs
    using tr-refl-inv
    by (metis translated-cup)
qed(simp add:translated-cup)

lemma translated-cap:
  assumes cap k xs
  shows   cap k (map (λp. p + t) xs) using assms
proof(induction xs arbitrary: k)
```

case (*Cons a xs*)
hence *cap* (*k−1*) *xs* **using** *cap-reduct*
by (*metis Suc-eq-plus1 Suc-pred′ bot-nat-0.not-eq-extremum cap-def length-0-conv neq-Nil-conv*)
hence *F1*:*cap* (*k−1*) (*map* (λ*p*. *p* + *t*) *xs*)
using *Cons.IH* **by** *blast*
have *F2*: *length* (*a*#*xs*) = *k* **using** *Cons*(*2*) *cap-def* **by** *simp*
hence *F3*: *length* (*map* (λ*p*. *p* + *t*) (*a*#*xs*)) = *k* **by** *simp*
have *F4*: *sdistinct* (*map* (λ*p*. *p* + *t*) (*a*#*xs*)) **using** *translated-sdistinct Cons*(*2*) *cap-def*
by *blast*
then show *?case*
proof(*cases length* (*map* (λ*p*. *p*+*t*) (*a*#*xs*)) ≤ *2*)
case *True*
then show *?thesis* **using** *F3 lcheck-len2-T*
using *F4 cap-def* **by** *blast*
next
case *False*
hence *length* (*a*#*xs*) ≥ *3* **by** *simp*
hence ∃ *a u v rest*. (*a*#*xs*) = (*a*#*u*#*v*#*rest*)
by (*metis One-nat-def Suc-eq-plus1 list.size*(*3*,*4*) *neq-Nil-conv nle-le not-less-eq-eq numeral-3-eq-3*)
then obtain *u v rest* **where** *xsp*: *xs* = *u*#*v*#*rest* **by** *blast*
hence *cap3 a u v list-check cap3* (*u*#*v*#*rest*)
using *assms cap-def Cons.prems* **by** *auto*
hence *F2*:*cap3* (*a*+*t*) (*u*+*t*) (*v*+*t*) **using** *translated-cross3*
by (*metis cap3-def*)
have *list-check cap3* (*map* (λ*p*. *p*+*t*) (*u*#*v*#*rest*))
using *F1 cap-def xsp* **by** *blast*
then show *?thesis* **using** *F2*
using *F3 F4 cap-def list.simps*(*9*) *list-check.simps*(*4*) *xsp* **by** *force*
qed
qed(*simp*)

lemma *translated-cap-eq*:
⋀*t*. *cap k xs* = *cap k* (*map* (λ*p*. *p* + *t*) *xs*)
by (*metis tr-refl-inv translated-cap*)

lemma *card-tr*:
fixes *t* :: *R2*
assumes *card S* = *n*
shows *card* ((λ *p*. *p* + *t*) ‘ *S*) = *n*
using *bij-tr bij-def assms*
by (*metis card-vimage-inj inj-vimage-image-eq top.extremum*)

lemma *gpos-tr*:
fixes *t* :: *R2*

assumes *gpos S*
shows *gpos* ((λ *p*. *p* + *t*) ‘ *S*)
proof−
{
fix *a b c*
assume *asm*:*a*∈*S b*∈*S c*∈*S distinct* [*a*,*b*,*c*]
hence *4*:¬*collinear3 a b c* **using** *assms* **unfolding** *gpos-def* **by** *simp*
have *5*:*distinct* [*a* + *t*, *b* + *t*, *c* + *t*] **using** *asm*(*4*) **by** *auto*
have ¬*collinear3* (*a* + *t*) (*b* + *t*) (*c* + *t*)
using *4* **unfolding** *collinear3-def cross3-def* **by** *simp*
then have *distinct* [*a* + *t*, *b* + *t*, *c* + *t*] ⟶ ¬*collinear3* (*a* + *t*) (*b* + *t*) (*c* + *t*)
using *5* **by** *simp*
}
thus *?thesis* **using** *gpos-def* **by** *simp*
qed

lemma *translated-set*:
fixes *t* :: *R2*

assumes *card S* = *n*
and *general-pos S*
and *sdistinct*(*sorted-list-of-set S*)
and ∀ *xs*. *set xs* ⊆ *S* ∧ *sdistinct xs* ⟶ ¬(*cap k xs* ∨ *cup l xs*)
and *St* = (λ *p*. *p* + *t*) ‘ *S*

shows *card St* = *n* ∧ *general-pos St* ∧ *sdistinct*(*sorted-list-of-set St*) ∧
(∀ *xs*. *set xs* ⊆ *St* ∧ *sdistinct xs* ⟶ ¬(*cap k xs* ∨ *cup l xs*))
proof−
have *P1*:*card St* = *n* **using** *assms*(*1*,*5*) *card-tr* **by** *simp*

have *P2*:*general-pos St* **using** *gpos-generalpos gpos-tr assms*(*2*,*5*) **by** *blast*

have *P3*:*sdistinct*(*sorted-list-of-set St*) **using** *assms*(*3*,*5*) *translated-sdistinct*
by (*metis map-sorted-list-set*)

A change in variable, *ys* = *map* (λ*p*. *p* + *t*) *xs*, allows us to prove the following.

have ∀ *xs*. *set xs* ⊆ *S* ⟶
¬(*cap k* (*map* (λ *p*. *p* + *t*) *xs*) ∨ *cup l* (*map* (λ *p*. *p* + *t*) *xs*))
using *translated-cup-eq translated-cap-eq*
by (*meson assms*(*4*) *cap-def cup-def*)
hence *9*:∀ *xs*. ∀ *ys*. *set xs* ⊆ *S* ∧ (*ys* = *map* (λ*p*. *p*+*t*) *xs*)
⟶ ¬(*cap k ys* ∨ *cup l ys*) **by** *blast*
hence ∀ *xs*. ∀ *ys*. *set ys* ⊆ ((λ*p*. *p*+*t*) ‘ *S*) ∧ (*ys* = *map* (λ*p*. *p*+*t*) *xs*)
⟶ ¬(*cap k ys* ∨ *cup l ys*)
by (*simp add*: *image-iff image-subset-iff subsetI*)
hence ∀ *xs*. *set xs* ⊆ *St* ∧ *sdistinct xs* ⟶ ¬(*cap k xs* ∨ *cup l xs*)
using *assms*(*5*) *9*

by (*metis* (*no-types*, *lifting*) *diff-add-cancel ex-map-conv*)

thus *?thesis* **using** *P1 P2 P3* **by** *simp*
qed

lemma *shift-set-above-c2*:
fixes *S1 S2* :: *R2 set*

assumes *distinct* (*map fst* (*sorted-list-of-set S1*))
and *finite S1*
and *card S1* ≥ *2*

and *distinct* (*map fst* (*sorted-list-of-set S2*))
and *finite S2*
and *card S2* ≥ *2*

shows ∃ *t*. ∃ *S2t*.
(*S2t* = (λ*p*. *p* + *t*) ‘ *S2* ∧
(∀ *x*∈*S1*. ∀ *y*∈*S2t*. *fst x* < *fst y*) ∧
(∀ *x*∈*S1*.∀ *y*∈ *S1*. ∀ *z*∈*S2t*. *x* < *y* ⟶ *slope x y* < *slope y z*) ∧
(∀ *a*∈*S1*. ∀ *b*∈*S2t*. ∀ *c*∈*S2t*. *b* < *c* ⟶ *slope a b* > *slope b c*))
proof−
define *s1-minx* **where** *s1-minx* = *Min* (*fst* ‘ *S1*)
define *s1-maxx* **where** *s1-maxx* = *Max* (*fst* ‘ *S1*)
define *s1-maxy* **where** *s1-maxy* = *Max* (*snd* ‘ *S1*)

define *s2-miny* **where** *s2-miny* = *Min* (*snd* ‘ *S2*)
define *s2-minx* **where** *s2-minx* = *Min* (*fst* ‘ *S2*)
define *s2-maxx* **where** *s2-maxx* = *Max* (*fst* ‘ *S2*)
define *w1* **where** *w1* = *s1-maxx* − *s1-minx*
define *w2* **where** *w2* = *s2-maxx* − *s2-minx*

define *S1pr* **where** *S1pr* = {(*u*,*v*)|*u v*. *u* ∈ *S1* ∧ *v* ∈ *S1* ∧ *u* < *v*}
define *S2pr* **where** *S2pr* = {(*u*,*v*)|*u v*. *u* ∈ *S2* ∧ *v* ∈ *S2* ∧ *u* < *v*}

have *s1pr-fin*: *finite S1pr* **using** *assms*(*2*) *S1pr-def finite-set-pr* **by** *fast*
have *s2pr-fin*: *finite S2pr* **using** *assms*(*5*) *S2pr-def finite-set-pr* **by** *fast*
have *s1pr-ne*: *S1pr* ≠ {} **using** *assms*(*1*,*3*) *dst-set-card2-ne S1pr-def* **by** *blast*
have *s2pr-ne*: *S2pr* ≠ {} **using** *assms*(*4*,*6*) *dst-set-card2-ne S2pr-def* **by** *blast*

have *w1-pos* : *w1* ≥ *0* **using** *w1-def assms*(*2*,*3*) *s1-maxx-def s1-minx-def width-non-zero*
by (*metis card.empty finite-imageI image-is-empty not-numeral-le-zero*)
have *w2-pos* : *w2* ≥ *0* **using** *w2-def assms*(*5*,*6*) *s2-maxx-def s2-minx-def width-non-zero*
by (*metis card.empty finite-imageI image-is-empty not-numeral-le-zero*)

define *tx* **where** *tx* = −*s2-minx* + *s1-maxx* + *1*
define *ty1* **where**
ty1 = *max s1-maxy* (*MAX* (*a*,*b*) ∈ *S1pr*. *snd b* + (*slope a b*) ∗ ((*s1-maxx* + *1*)
+ *w2* − *fst b*))

define *ty2* **where**
ty2 = *max s1-maxy*
(*MAX* (*b*,*c*) ∈ *S2pr*. *s1-maxy* + *s2-miny* − *snd b* + (*slope* (*b*+(*tx*,*0*)) (*c*+(*tx*,*0*)))∗(*fst b*+*tx*−*s1-minx*))

define *ty* **where** *ty* = *max ty1 ty2*
define *t* **where** *t* = (−*s2-minx* + *s1-maxx* + *1*, −*s2-miny* + *ty* + *1*)
define *S2t* **where** *S2t* = (λ*p*. *p* + *t*) ‘ *S2* (**is** - = *?tr* ‘ -)

Properties of max function follow.

have *m1*: *ty* ≥ *ty1 ty* ≥ *ty2* **using** *ty-def* **by** *simp*+

The result r1 below is obtained by translating all the points of S2 along the x-axis after the point in S1 with highest x-coordinate.

have *f1r1*:∀ *x*∈*S1*. *fst x* < *s1-maxx* + *1* **using** *s1-maxx-def*
by (*smt* (*verit*, *best*) *Max-ge assms*(*2*) *finite-imageI image-eqI*)
have *f2r1*:∀ *y*∈*S2t*. *s1-maxx* + *1* ≤ *fst y* **using** *s2-minx-def S2t-def t-def*
using *assms*(*5*) **by** *auto*
hence *r1*:(∀ *x*∈*S1*. ∀ *y*∈*S2t*. *fst x* < *fst y*) **using** *f1r1 f2r1* **by** *fastforce*

We chose ty1 in such a way that the lower right corner of bounding box of S2 is above all the lines passing through point pairs in S1.

have *f0r2*:∀ *a*∈*S1*. ∀ *b*∈*S1*. *a* < *b* ⟶ *slope a b* < *slope b* (*s1-maxx*+*1* + *w2*, *ty1*+*1*)
proof(*rule*+)
fix *a b* **assume** *asm*:*a* ∈ *S1 b* ∈ *S1 a* < *b*
have *f2*:(*a*,*b*) ∈ *S1pr* **using** *asm S1pr-def* **by** *blast*
have *ty1* ≥ (*MAX* (*a*,*b*) ∈ *S1pr*. *snd b* + (*slope a b*) ∗ (*s1-maxx* + *1* + *w2* − *fst b*))
using *ty1-def* **by** *argo*
hence *ty1* + *1* > *snd b* + ((*slope a b*) ∗ (*s1-maxx* + *1* + *w2* − *fst b*))
using *f2 s1pr-ne s1pr-fin Max-ge finite-imageI pair-imageI*
by (*smt* (*verit*, *best*))
hence *slope a b* < (*ty1* + *1* − *snd b*) / (*s1-maxx* + *1* + *w2* − *fst b*)
using *f1r1 asm divide-diff-eq-iff divide-pos-pos s1-maxx-def w2-pos* **by** *smt*
thus *slope a b* < *slope b* (*s1-maxx*+*1* + *w2*, *ty1*+*1*) **using** *slope-def*
by *simp*
qed

We show that the slope of any point y in S1 to the lower left corner of bounding box of S2t is at most the slope to any other point in S2t.

have ∀ *y*∈*S1*. ∀ *z*∈*S2t*. *y* < *z* ⟶ *slope y* (*s1-maxx*+*1* + *w2*, *ty1*+*1*) ≤ *slope y z*
proof(*rule*+)
fix *y z* **assume** *asm*:*y* ∈ *S1 z*∈*S2t y*<*z*
then obtain *z2*
where *z2*: *z2* ∈ *S2 z* = (λ*p*. *p* + *t*) *z2*
using *S2t-def t-def* **by** *blast*

have *fx*:*snd* (*?tr z2*) ≥ *ty* + *1* **using** *s2-miny-def asm ty-def t-def z2*(*1*) *assms*(*5*)
by *simp*
moreover have ... ≥ *s1-maxy* **using** *m1*(*1*) *ty1-def calculation* **by** *linarith*
moreover have ... ≥ *snd y* **using** *asm*(*1*) *assms*(*2*) *s1-maxy-def*
by (*smt* (*verit*) *Max-ge calculation*(*2*) *finite-imageI imageI*)
ultimately have *0*:*snd* (*?tr z2*) − *snd y* ≥ *0* **using** *ty1-def* **by** *argo*

have *1*:*snd* (*?tr z2*) − *snd y* ≥ *ty1* + *1* − *snd y*
using *fx assms s2-miny-def z2*(*1*) *t-def m1*(*1*) **by** *argo*
have *2*:*fst* (*?tr z2*) − *fst y* ≤ *s1-maxx*+*1*+*w2* − *fst y*
by (*simp add*: *assms s2-maxx-def z2*(*1*) *w2-def t-def*)
have *3*: (*fst* (*?tr z2*) − *fst y*) > *0* **using** *asm*(*1*,*2*) *r1 z2*(*2*) **by** *fastforce*
show *slope y* (*s1-maxx* + *1* + *w2*, *ty1* + *1*) ≤ *slope y z*
using *0 1 2 3 slope-def slope-div-ord*
by (*metis frac-le fst-eqD snd-eqD z2*(*2*))
qed
hence *r2*:∀ *x*∈*S1*.∀ *y*∈ *S1*. ∀ *z*∈*S2t*. *x* < *y* ⟶ *slope x y* < *slope y z*
using *f0r2 r1* **by** (*smt* (*verit*, *best*) *prod-less-def*)

We now apply the bounding box technique on S1. We make sure the upper left corner of bounding box of S1 is below the max slope in S2 after shifting.

have *f0r3*:∀ *b*∈*S2t*. ∀ *c*∈*S2t*. *b* < *c* ⟶ *slope* (*s1-minx*, *s1-maxy*) *b* > *slope b c*
proof(*rule*+)
fix *b c* **assume** *asm*:*b* ∈ *S2t c* ∈ *S2t b* < *c*
then obtain *b2 c2* **where** *b2c2*: *b2*∈*S2 b* = *b2* + *t c2*∈*S2 c* = *c2* + *t*
using *S2t-def* **by** *blast*
have *f2*:(*b2*,*c2*) ∈ *S2pr* **using** *asm S2pr-def* **by** (*metis* (*mono-tags*, *lifting*) *b2c2*(*1*,*2*,*3*,*4*) *linorder-not-less mem-Collect-eq prod-addright-le*)
have *f3*:*slope b c* = *slope* (*b2*+(*tx*,*0*)) (*c2*+(*tx*,*0*)) **by** (*simp add*: *b2c2*(*2*,*4*) *slope-def*)
have *ty* ≥
(*MAX* (*b*,*c*) ∈ *S2pr*. *s1-maxy* + *s2-miny* − *snd b* +(*slope* (*b*+(*tx*,*0*)) (*c*+(*tx*,*0*)))∗(*fst b*+*tx*−*s1-minx*))
using *ty-def ty2-def f2 m1* **by** *fastforce*
hence *ty* ≥ *s1-maxy* + *s2-miny* − *snd b2* + (*slope* (*b2*+(*tx*,*0*)) (*c2*+(*tx*,*0*)))∗(*fst b2* + *tx*−*s1-minx*)
using *f2 s2pr-fin s2pr-ne* **by** *auto*
hence *ty* + *1* > *s1-maxy* + *s2-miny* − *snd b2* + (*slope* (*b2*+(*tx*,*0*)) (*c2*+(*tx*,*0*)))∗(*fst b2*+*tx*−*s1-minx*)
by *linarith*
hence *slope b c* < (*snd b* − *s1-maxy*) / (*fst b* − *s1-minx*)
using *f3 asm*(*1*) *b2c2*(*2*) *f2r1 less-divide-eq t-def tx-def w1-def w1-pos* **by** *fastforce*
thus *slope* (*s1-minx*, *s1-maxy*) *b* > *slope b c* **using** *slope-def* **by** *simp*
qed

Next we show that the slope across sets S1 and S2t is at least the slope of

upper left corner to any point in the set S2t.

```
  have ∀ a∈S1. ∀ b∈S2t. a < b ⟶ slope a b ≥ slope (s1-minx, s1-maxy) b
  proof(rule+)
    fix a b assume asm:a ∈ S1 b∈S2t a<b
    then obtain b2 where b2: b2 ∈ S2 b = ?tr b2 using S2t-def t-def by blast

    have fx:snd b ≥ ty + 1 using s2-miny-def t-def b2 assms(5) by simp
    moreover have ... ≥ s1-maxy using m1(1) ty1-def calculation by linarith
    ultimately have 0:snd b − s1-maxy ≥ 0 by argo

    have 1:snd b − snd a  ≥ snd b − (s1-maxy)
      using fx assms s2-miny-def b2(1) t-def m1(1)
      by (smt (verit, del-insts) Max.coboundedI asm(1)
          finite-imageI imageI s1-maxy-def)
     have 2:fst b − fst a ≤ fst b − (s1-minx)  by (simp add: asm(1) assms(2)
s1-minx-def)
    have 3: (fst b − fst a) > 0                by (simp add: asm(1,2) r1)
    show slope a b ≥ slope (s1-minx, s1-maxy) b using 0 1 2 3
      by (simp add: b2(2) frac-le slope-def)
  qed
  hence ∀ a∈S1. ∀ b∈S2t. ∀ c∈S2t. b < c ⟶ slope a b > slope b c
    using S2t-def f0r3
    by (metis order-less-le-trans prod-less-def r1)
  then show ?thesis using r1 r2 S2t-def t-def by auto
qed

lemma shift-set-above:
  fixes S1 S2 :: R2 set

  assumes distinct (map fst (sorted-list-of-set S1))
      and finite S1
      and card S1 ≥ 1

      and distinct (map fst (sorted-list-of-set S2))
      and finite S2
      and card S2 ≥ 1

    shows ∃ t. ∃ S2t.
            ( S2t = (λp. p + t) ‘ S2 ∧
              (∀ x∈S1. ∀ y∈S2t. fst x < fst y) ∧
              (∀ x∈S1.∀ y∈ S1. ∀ z∈S2t.  x < y ⟶ slope x y < slope y z) ∧
              (∀ a∈S1. ∀ b∈S2t. ∀ c∈S2t. b < c ⟶ slope a b > slope b c) )
proof(cases card S1 = 1 ∨ card S2 = 1)
  case Tout:True
  then show ?thesis
  proof(cases card S1 ≠ 1)
    case True
    hence s2-c1:card S2 = 1 using Tout by simp
    have s1-c2:card S1 ≥ 2 using assms(3) True by simp
```

then obtain *p2* **where** *p2*:*S2* = {*p2*}
using *card-1-singletonE s2-c1* **by** *blast*

define *tx* **where** *tx* = *Max* (*fst* ‘ *S1*) + *1*
define *S1pr* **where** *S1pr* = {(*u*,*v*)|*u v*. *u* ∈ *S1* ∧ *v* ∈ *S1* ∧ *u*<*v*}
then obtain *s1 s2* **where** *s1*∈*S1 s2*∈*S1 s1*<*s2* **using** *assms s1-c2*
by (*meson card-2-iff′ ex-card linorder-less-linear subset-iff*)
hence (*s1*, *s2*) ∈ *S1pr* **using** *S1pr-def* **by** *blast*
hence *S1pr-nonE*:*S1pr* ≠ {} **by** *force*
have *S1pr-fin*:*finite S1pr* **using** *S1pr-def assms*(*2*) *finite-set-pr* **by** *blast*

define *ty* **where** *ty*:
ty=(*MAX* (*a*,*b*) ∈ *S1pr*. *snd b* + (*slope a b*) ∗ (*tx* − *fst b*)) + *1*
define *S2t* **where** *S2t* = (λ*p*. *p* + (−*fst p2* + *tx*, −*snd p2* + *ty*)) ‘ *S2*

have *f1*:(∀ *x*∈*S1*. *fst x* < *Max* (*fst* ‘ *S1*) + *1*)
by (*smt* (*verit*, *best*) *Max-ge assms finite-imageI image-eqI*)
hence *r1*:(∀ *x*∈*S1*. ∀ *y*∈*S2t*. *fst x* < *fst y*) **using** *tx-def S2t-def p2* **by** *auto*

have ∀ *a*∈*S1*. ∀ *b*∈*S1*. *a* < *b* ⟶ *slope a b* < *slope b* (*tx*, *ty*)
proof(*rule*+)
fix *a b* **assume** *asm*:*a* ∈ *S1 b* ∈ *S1 a* < *b*
have *f2*:(*a*,*b*) ∈ *S1pr* **using** *asm S1pr-def* **by** *blast*
have *ty*−*1*=(*MAX* (*a*,*b*) ∈ *S1pr*. *snd b* + (*slope a b*) ∗ (*tx* − *fst b*))
using *ty* **by** *argo*
hence *ty* − *1* ≥ (*snd b* + (*slope a b*) ∗ (*tx* − *fst b*))
using *f2 S1pr-nonE S1pr-fin*
by (*metis* (*no-types*, *lifting*) *Max-ge finite-imageI*
pair-imageI)
hence *slope a b* < (*ty* − *snd b*) / (*tx* − *fst b*)
by (*smt* (*verit*, *ccfv-threshold*) *f1 asm*(*2*) *divide-diff-eq-iff divide-pos-pos*
tx-def)
thus *slope a b* < *slope b* (*tx*, *ty*) **using** *slope-def*
by *simp*
qed
hence *r2*:∀ *x*∈*S1*.∀ *y*∈ *S1*. ∀ *z*∈*S2t*. *x* < *y* ⟶ *slope x y* < *slope y z*
by (*smt* (*verit*, *del-insts*) *S2t-def add-Pair imageE p2*
prod.collapse singletonD)
have ∀ *a*∈*S1*. ∀ *b*∈*S2t*. ∀ *c*∈*S2t*. *b* < *c* ⟶ *slope a b* > *slope b c*
using *S2t-def p2* **by** *blast*
then show *?thesis* **using** *r1 r2 S2t-def* **by** *auto*
next
case *False*
then show *?thesis*
proof(*cases card S2* = *1*)
case *True*
then obtain *s1 s2* **where** *s1s2*: *S1* = {*s1*} *S2* = {*s2*} **using** *False*
by (*meson card-1-singletonE*)
define *S2t* **where** *S2t* = (λ*p*. *p* + (−*fst s2* + *fst s1* + *1*, *0*)) ‘ *S2*

have *r1*: ($\forall$ *x*∈*S1*. $\forall$ *y*∈*S2t*. *fst x* < *fst y*)
by (*simp add*: *S2t-def s1s2*(*1*,*2*))
have ($\forall$ *x*∈*S1*. $\forall$ *y*∈*S1*. $\forall$ *z*∈*S2t*. *x* < *y* ⟶ *slope x y* < *slope y z*) ∧
($\forall$ *a*∈*S1*. $\forall$ *b*∈*S2t*. $\forall$ *c*∈*S2t*. *b* < *c* ⟶ *slope b c* < *slope a b*)
using *s1s2 S2t-def* **by** *blast*
then show *?thesis* **using** *r1*
using *S2t-def* **by** *auto*
next
case *False*
hence *s2-c2*: *card S2* ≥ *2* **using** *assms*(*6*) **by** *linarith*
have *s1-c1*: *card S1* = *1* **using** *False Tout* **by** *blast*
then obtain *s1x s1y* **where** *s1*: *S1* = {(*s1x*, *s1y*)}
using *card-1-singletonE* **by** (*metis surj-pair*)

define *s2-miny* **where** *s2-miny* = *Min* (*snd* ‘ *S2*)
define *s2-minx* **where** *s2-minx* = *Min* (*fst* ‘ *S2*)
define *S2pr* **where** *S2pr* = {(*u*,*v*)|*u v*. *u* ∈ *S2* ∧ *v* ∈ *S2* ∧ *u* < *v*}

have *s2pr-fin*: *finite S2pr* **using** *assms*(*5*) *S2pr-def finite-set-pr* **by** *fast*
have *s2pr-ne*: *S2pr* ≠ {} **using** *assms*(*4*,*6*) *dst-set-card2-ne S2pr-def s2-c2*
by *presburger*

define *tx* **where** *tx* = −*s2-minx* + *s1x* + *1*
define *ty2* **where**
ty2 = *max s1y*
(*MAX* (*b*,*c*) ∈ *S2pr*. *s1y* + *s2-miny* − *snd b* + (*slope* (*b*+(*tx*,*0*)) (*c*+(*tx*,*0*)))∗(*fst*
b+*tx*−*s1x*))

define *t* **where** *t* = (−*s2-minx* + *s1x* + *1*, −*s2-miny* + *ty2* + *1*)
define *S2t* **where** *S2t* = (λ*p*. *p* + *t*) ‘ *S2* (**is** - = *?tr* ‘ -)

have *f1r1*:∀ *x*∈*S1*. *fst x* < *s1x* + *1* **using** *s1* **by** *simp*
have *f2r1*:∀ *y*∈*S2t*. *s1x* + *1* ≤ *fst y* **using** *s2-minx-def S2t-def t-def*
using *assms*(*5*) **by** *auto*
hence *r1*:($\forall$ *x*∈*S1*. $\forall$ *y*∈*S2t*. *fst x* < *fst y*) **using** *f1r1 f2r1* **by** *fastforce*

have $\forall$ *b*∈*S2t*. $\forall$ *c*∈*S2t*. *b* < *c* ⟶ *slope* (*s1x*, *s1y*) *b* > *slope b c*
proof(*rule*+)
fix *b c* **assume** *asm*:*b* ∈ *S2t c* ∈ *S2t b* < *c*
then obtain *b2 c2* **where** *b2c2*: *b2*∈*S2 b* = *b2* + *t c2*∈*S2 c* = *c2* + *t*
using *S2t-def* **by** *blast*
have *f2*:(*b2*,*c2*) ∈ *S2pr* **using** *asm S2pr-def* **by** (*metis* (*mono-tags*, *lifting*)
b2c2(*1*,*2*,*3*,*4*) *linorder-not-less mem-Collect-eq prod-addright-le*)
have *f3*:*slope b c* = *slope* (*b2*+(*tx*,*0*)) (*c2*+(*tx*,*0*)) **by** (*simp add*: *b2c2*(*2*,*4*)
slope-def)
have *ty2* ≥
(*MAX* (*b*,*c*) ∈ *S2pr*. *s1y* + *s2-miny* − *snd b* +(*slope* (*b*+(*tx*,*0*)) (*c*+(*tx*,*0*)))∗(*fst*
b+*tx*−*s1x*))
using *ty2-def* **by** *fastforce*

hence *ty2 + 1 > s1y + s2-miny − snd b2 + slope b c ∗ (fst b2+tx−s1x)*
using *f3 f2 s2pr-fin s2pr-ne* **by** *auto*
hence *slope b c < (snd b2 − s2-miny + ty2 + 1 − s1y) / (fst b2 + tx − s1x)*
using *asm(1) b2c2(2) f2r1 less-divide-eq t-def tx-def*
by *fastforce*
thus *slope (s1x, s1y) b > slope b c* **using** *slope-def t-def S2t-def*
by (*smt* (*verit*, *best*) *b2c2(2) fst-add fst-conv snd-add*
snd-eqD tx-def)
qed
hence *r3*:∀ *a*∈*S1*. ∀ *b*∈*S2t*. ∀ *c*∈*S2t*. *b < c ⟶ slope a b > slope b c*
using *s1* **by** *blast*
have *r2*: ∀ *x*∈*S1*.∀ *y*∈ *S1*. ∀ *z*∈*S2t*. *x < y ⟶ slope x y < slope y z* **using** *s1* **by** *blast*
show *?thesis* **using** *S2t-def r1 r2 r3 t-def*
by *auto*
qed
qed
next
case *False*
hence *card S1 ≥ 2 card S2 ≥ 2*
using *assms(3,6)* **by** *fastforce+*
then show *?thesis* **using** *shift-set-above-c2 assms* **by** *presburger*
qed

end
theory *EZ-bound*
imports *Four-Convex Definitions Prelims GeneralPosition3 Invariants*
begin

lemma *min-conv-set-alt-def*:
shows *min-conv-set k l*
=
{*n*. ∀ *S* . (*card S = n ∧ general-pos S ∧ sdistinct*(*sorted-list-of-set S*))
⟶ (∃ *xs*. *set xs ⊆ S ∧ sdistinct xs ∧* (*cap k xs ∨ cup l xs*))}
by (*smt* (*verit*) *min-conv-set-def Collect-cong add.commute add-diff-cancel-left' card.infinite diff-is-0-eq' dual-order.trans ex-card general-pos-subs le-add-diff-inverse linorder-not-less nless-le sdistinct-sub*)

lemma *min-conv-leImpe*:
shows *Inf* (*min-conv-set k l*)
=
Inf {*n*. ∀ *S* . (*card S = n ∧ general-pos S ∧ sdistinct*(*sorted-list-of-set S*))
⟶ (∃ *xs*. *set xs ⊆ S ∧ sdistinct xs ∧* (*cap k xs ∨ cup l xs*))}
using *min-conv-set-alt-def* **by** *presburger*

lemma *min-conv-lower-imp1o*:
assumes $n < \textit{min-conv}\ k\ l$
shows $\exists S.\ (\textit{card}\ S = n \wedge \textit{general-pos}\ S \wedge \textit{sdistinct}(\textit{sorted-list-of-set}\ S))$
$\wedge\ (\forall \textit{xs}.\ \textit{set xs} \subseteq S \wedge \textit{sdistinct xs} \longrightarrow \neg(\textit{cap}\ k\ \textit{xs} \vee \textit{cup}\ l\ \textit{xs}))$
using *assms inf-upper min-conv-def min-conv-leImpe min-conv-set-def*
by (*smt* (*verit*, *del-insts*) *linorder-not-less mem-Collect-eq sorted-list-of-set.sorted-sorted-key-list-of-set*)

lemma *min-conv-num-out-set*:
assumes $\exists S.\quad \textit{card}\ S \geq n \wedge \textit{general-pos}\ S \wedge \textit{sdistinct}(\textit{sorted-list-of-set}\ S)$
$\wedge$
$(\forall \textit{xs}.\ \textit{set xs} \subseteq S \wedge \textit{sdistinct xs} \longrightarrow \neg(\textit{cap}\ k\ \textit{xs} \vee \textit{cup}\ l\ \textit{xs}))$
shows $n \notin \textit{min-conv-set}\ k\ l$
unfolding *min-conv-set-def* **using** *assms* **by** *blast*

lemma *min-conv-0*:
$\textit{min-conv}\ 0\ l = 0\ \textit{min-conv-set}\ 0\ l \neq \{\}$
proof−
have *1*:*cap 0* []
by (*simp add*: *cap-def*)
thus $\textit{min-conv}\ 0\ l = 0$
by (*metis* (*no-types*, *opaque-lifting*) *cap-def empty-subsetI list.set*(*1*)
min-conv-lower-imp1o neq0-conv)
thus $\textit{min-conv-set}\ 0\ l \neq \{\}$
using *1 min-conv-set-def* **by** *force*
qed

lemma *extract-cap-cup*:
assumes $\textit{min-conv}\ k\ l = n$ **and** $\textit{min-conv-set}\ k\ l \neq \{\}$
and $\textit{card}\ (S :: \textit{R2 set}) = n$
and *general-pos S*
and $\textit{sdistinct}(\textit{sorted-list-of-set}\ S)$
shows $\exists \textit{xs}.\ \textit{set xs} \subseteq S \wedge \textit{sdistinct xs} \wedge (\textit{cap}\ k\ \textit{xs} \vee \textit{cup}\ l\ \textit{xs})$
using *assms Inf-nat-def1 mem-Collect-eq min-conv-def min-conv-set-def*
by (*smt* (*verit*, *best*) *linorder-not-less nless-le*)

lemma *min-conv-num-out*:
assumes $\textit{min-conv-set}\ k\ l \neq \{\}$
and $\exists S.\ \textit{card}\ S \geq n \wedge \textit{general-pos}\ S \wedge \textit{sdistinct}(\textit{sorted-list-of-set}\ S) \wedge$
$(\forall \textit{xs}.\ \textit{set xs} \subseteq S \wedge \textit{sdistinct xs} \longrightarrow \neg(\textit{cap}\ k\ \textit{xs} \vee \textit{cup}\ l\ \textit{xs}))$
shows $\textit{min-conv}\ k\ l \neq n$
using *assms min-conv-def min-conv-num-out-set min-conv-set-def*
by (*metis* (*lifting*) *Inf-nat-def1*)

lemma *min-conv-lower*:
assumes $\textit{min-conv-set}\ k\ l \neq \{\}$
and $\exists S.\ \textit{card}\ S = n \wedge \textit{general-pos}\ S \wedge$
$\textit{sdistinct}(\textit{sorted-list-of-set}\ S) \wedge$
$(\forall \textit{xs}.\ \textit{set xs} \subseteq S \wedge \textit{sdistinct xs} \longrightarrow \neg(\textit{cap}\ k\ \textit{xs} \vee \textit{cup}\ l\ \textit{xs}))$
shows $\textit{min-conv}\ k\ l > n$

proof−
obtain S **where** *card* $S \geq n$ *general-pos* S
sdistinct(*sorted-list-of-set* S)
$\forall$ *xs. set xs* $\subseteq S \wedge$ *sdistinct xs* $\longrightarrow \neg$(*cap k xs* $\vee$ *cup l xs*)
using *assms* **by** *blast*
hence $\forall t.\ t \leq n \longrightarrow$ *min-conv k l* $\neq t$ **using** *min-conv-num-out assms* **by** *metis*
thus *?thesis* **using** *min-conv-def not-le-imp-less* **by** *blast*
qed

lemma *min-conv-least-out-set*:
assumes *a*$\geq$*1 b*$\geq$*a*
shows $a - 1 \notin \{n. \forall S.\ n =$ *card* $S \wedge$ *general-pos* $S \wedge$ *sdistinct*(*sorted-list-of-set* $S) \longrightarrow$
($\exists$ *xs. set xs* $\subseteq S \wedge$ *sdistinct xs* $\wedge$ (*cap a xs* $\vee$ *cup b xs*))$\}$
proof−
{
assume *asm*:$a - 1 \in \{n. \forall S.\ n =$ *card* $S \wedge$ *general-pos* $S \wedge$ *sdistinct*(*sorted-list-of-set* $S) \longrightarrow$
($\exists$ *xs. set xs* $\subseteq S \wedge$ *sdistinct xs* $\wedge$ (*cap a xs* $\vee$ *cup b xs*))$\}$
fix S
have *F1*: *a*−*1* = *card* $S \wedge$ *general-pos* $S \wedge$ *sdistinct*(*sorted-list-of-set* S) $\longrightarrow$ ($\exists$ *xs. set xs* $\subseteq S \wedge$ *sdistinct xs* $\wedge$ (*cap a xs* $\vee$ *cup b xs*))
using *asm* **by** *simp*
have *F2*: $\forall$ *xs.* (*a*−*1* = *card* $S \wedge$ *set xs* $\subseteq S \wedge$ *general-pos*(S) $\wedge$ *sdistinct*(*sorted-list-of-set* S) $\longrightarrow \neg$(*cap a xs* $\vee$ *cup b xs*))
proof
fix *xs*
{
assume *a1*:$a - 1$ = *card* S *set xs* $\subseteq S$ *sdistinct*(*sorted-list-of-set* S)
hence *asmd*: $a - 1$ = *card* S *set xs* $\subseteq S$ **using** *a1* **by** *auto*
hence *a2*:*length xs* $>$ *a*−*1* $\Longrightarrow \neg$*sdistinct xs* **using** *a1*
by (*smt* (*verit*, *ccfv-threshold*) *Orderings.order-eq-iff asm assms cap-def card.infinite*
card-mono cup-def empty-subsetI general-pos-subs genpos-ex gpos-generalpos le-add-diff-inverse
length-0-conv linorder-not-less mem-Collect-eq nat.distinct(*1*) *plus-1-eq-Suc sdistinct-sortedStrict set-empty2 sorted-list-of-set.idem-if-sorted-distinct sorted-list-of-set.length-sorted-key-list-of-set strict-sorted-iff*)
hence $\neg$(*cap a xs* $\vee$ *cup b xs*)
using *assms a2 cap-def cup-def* **by** *force*
}
thus $a - 1$ = *card* $S \wedge$ *set xs* $\subseteq S \wedge$ *general-pos* $S \wedge$ *sdistinct*(*sorted-list-of-set* S)
$\longrightarrow \neg$ (*cap a xs* $\vee$ *cup b xs*) **by** *simp*
qed
{
assume *A1*:*a*−*1* = *card* S *general-pos* S *sdistinct*(*sorted-list-of-set* S)
hence ($\exists$ *xs. set xs* $\subseteq S \wedge$ *sdistinct xs* $\wedge$ (*cap a xs* $\vee$ *cup b xs*)) **using** *F1* **by** *simp*

```
    then obtain xs where xspp:set xs ⊆ S (cap a xs ∨ cup b xs) by blast
    hence ¬(cap a xs ∨ cup b xs) using F2 A1 by simp
    hence False using xspp by simp
   }
  }
  thus a − 1
   ∉ {n. ∀ S. n = card S ∧
            general-pos S ∧ sdistinct (sorted-list-of-set S) ⟶
            (∃ xs. set xs ⊆ S ∧ sdistinct xs ∧ (cap a xs ∨ cup b xs))}
   by (metis (no-types, lifting) genpos-ex gpos-generalpos)
qed

lemma min-conv-least:
  assumes min-conv-set a b ≠ {}
    and   a ≥ 1 and b ≥ a
  shows a ≤ min-conv a b
proof−
  have a −1 < min-conv a b using assms min-conv-lower min-conv-least-out-set
    by (smt (verit) mem-Collect-eq)
  thus a ≤ min-conv a b by linarith
qed

theorem min-conv-min:
  assumes min-conv-set a b ≠ {}
    and   a ≥ 1 and b ≥ 1
    shows min a b ≤ min-conv a b
proof(cases b≥a)
  case True
  then show ?thesis using min-conv-least assms(1,2) by simp
next
  case False
  hence 1:a≥b by simp
  hence 2:min a b = b by simp
  have min-conv-set a b = min-conv-set b a
    using min-conv-set-def min-conv-sets-eq by presburger
  hence b ≤ min-conv a b
    using min-conv-arg-swap min-conv-least[of b a] assms(1,3) 1 by simp
  then show ?thesis using 2 by simp
qed

lemma min-conv-1:
  assumes k≥1
  shows min-conv 1 k = 1 min-conv-set 1 k ≠ {}
proof−
  have min-conv 1 k ≤ 1 min-conv-set 1 k ≠ {}
  proof−
    {
      fix S
      assume asm:card S = 1 general-pos S
```

```
    have (∃ xs. set xs ⊆ S ∧ sortedStrict xs ∧ (cap 1 xs ∨ cup k xs))
    proof−
      obtain p where p1p2: S = {p} using asm using card-1-singletonE by
blast
      have 1:set [p] ⊆ S ∧ sortedStrict [p] using p1p2 by auto
      have 2: cap 1 [p] using p1p2 by (simp add: cap-def)
      thus ?thesis using 1 2 by blast
    qed
  }
  hence gg:∀ S. 1 ≤ card S ∧ general-pos S ∧ sdistinct(sorted-list-of-set S) ⟶
          (∃ xs. set xs ⊆ S ∧ sdistinct xs ∧ (cap 1 xs ∨ cup k xs))
    by (meson cap-def cup-def general-pos-subs obtain-subset-with-card-n
        subset-trans)
  thus min-conv 1 k ≤ 1 unfolding min-conv-def by (simp add: inf-upper)
  thus min-conv-set 1 k ≠ {} using gg min-conv-set-def by blast
 qed
 thus min-conv 1 k = 1 min-conv-set 1 k ≠ {}
  using assms min-conv-least order-class.order-eq-iff by (blast, simp)
qed

lemma min-conv-2:
 assumes k≥2
 shows min-conv 2 k = 2 min-conv-set 2 k ≠ {}
proof−
 have min-conv 2 k ≤ 2 min-conv-set 2 k ≠ {}
 proof−
  {
   fix S
   assume asm:card S = 2 general-pos S sdistinct(sorted-list-of-set S)
   have S-fin: finite S by (metis asm(1) card.infinite zero-neq-numeral)
   have (∃ xs. set xs ⊆ S ∧ sdistinct xs ∧ (cap 2 xs ∨ cup k xs))
   proof−
    obtain p1 p2 where p1p2: S = {p1, p2} p1 < p2
     by (metis asm(1) card-2-iff insert-commute linorder-less-linear)
    hence 1:set [p1,p2] ⊆ S ∧ sorted [p1,p2] ∧ sdistinct [p1,p2]
     by (metis One-nat-def Suc-1 asm(1,3) card-distinct dual-order.refl empty-set
         length-Cons list.simps(15) list.size(3) nless-le sorted1 sorted2
         sorted-list-of-set.idem-if-sorted-distinct)
    hence 2: cap 2 [p1,p2] using p1p2(2)
     by (simp add: cap-def)
    thus ?thesis using 1 2 by blast
   qed
  }
  hence gg:∀ S. 2 ≤ card S ∧ general-pos S ∧ sdistinct(sorted-list-of-set S) ⟶
          (∃ xs. set xs ⊆ S ∧ sdistinct xs ∧ (cap 2 xs ∨ cup k xs))
   by (smt (verit, ccfv-threshold)
       card.infinite ex-card
       general-pos-subs
       less-le-not-le less-one
```

```
      order.trans
      sdistinct-sub)
    thus min-conv 2 k ≤ 2 unfolding min-conv-def
     by (simp add: inf-upper)
    show gg1:min-conv-set 2 k ≠ {} using gg min-conv-set-def by blast
 qed
 thus min-conv 2 k = 2 min-conv-set 2 k ≠ {}
  using assms min-conv-least order-class.order-eq-iff
  by (metis Suc-1 Suc-eq-plus1 le-add2, simp)
qed

lemma min-conv-base-IH-aux:
 assumes min-conv 3 k = k  and min-conv-set 3 k ≠ {} and S-asm:  card S =
Suc k and
   S-gp: general-pos S     and sdistinct(sorted-list-of-set S)
 shows ∃ xs. set xs ⊆ S ∧ sdistinct xs ∧ (cap 3 xs ∨ cup (Suc k) xs)
 using assms
proof−
 obtain x xs where x-xs:S = set (x#xs) sdistinct (x#xs) length (x#xs) = card
S
   using genpos-ex S-asm assms list.exhaust list.size(3) nat.simps(3)
   by (metis  card.infinite sorted-list-of-set.sorted-key-list-of-set-unique)

 have x-Min:x = Min S using x-xs
  by (metis S-asm card.empty card.infinite card-distinct list.inject old.nat.distinct(2)
sorted-list-of-set.idem-if-sorted-distinct
      sorted-list-of-set-nonempty)

 have length xs = k using S-asm x-xs by auto
 hence sminus-x:card (S − {x}) = k using S-asm by (simp add: card.insert-remove
x-xs(1))
  moreover have gp-sminus-x:general-pos (S − {x}) using x-xs(1) S-gp gen-
eral-pos-subs by blast
```

Using induction hypothesis obtain the cap of size 3 or cup of size k from the set S - min(S).

```
 ultimately obtain zs where zs:set zs ⊆ S − {x} (cap 3 zs ∨ cup k zs)
   unfolding min-conv-def
   using assms genpos-ex gpos-generalpos x-xs sdistinct-sub gp-sminus-x assms(2)
extract-cap-cup
   by (metis Diff-subset List.finite-set)
 have ∃ cc. set cc ⊆ S ∧ (cap 3 cc ∨ cup (Suc k) cc)
 proof(cases cap 3 zs)
   case True
   then show ?thesis using zs(1,2) by auto
 next
   case False
   hence F1:cup k zs using zs by simp
   hence F2:length zs = k unfolding cup-def by argo
```

hence *F0*:*length* (*x*#*zs*) = *Suc k* **using** *x-Min* **by** *auto*
have *F4*:*set* (*x*#*zs*) ⊆ *S* **using** *x-xs*(*1*) *zs*(*1*) **by** *auto*
have *F3*:*sdistinct* (*x*#*zs*) **using** *zs x-Min x-xs*
by (*metis* (*no-types*, *lifting*) *Diff-insert-absorb F0 F1 F4 assms*(*3*) *card.infinite card-subset-eq cup-def distinct-card distinct-map distinct-singleton insert-absorb length-0-conv list.set*(*2*) *set-empty2 sminus-x sorted-list-of-set.idem-if-sorted-distinct sorted-list-of-set-nonempty*)
then show *?thesis*
proof(*cases cup* (*Suc k*) (*x*#*zs*))
case *True*
then show *?thesis* **using** *x-xs*(*1*) *zs*(*1*)
by (*metis Diff-empty insert-subset list.set-intros*(*1*) *list.simps*(*15*) *subset-Diff-insert*)
next
case *False*
hence ∃ *a b c*. (*sublist* [*a*,*b*,*c*] (*x*#*zs*)) ∧ ¬ *cup3 a b c*
using *F0 F3 get-cap-from-bad-cup* **by** *presburger*
then obtain *b1 b2 b3* **where** *bcup*:*sublist* [*b1*,*b2*,*b3*] (*x*#*zs*) ¬ *cup3 b1 b2 b3* **by** *blast*
have *fb1*:*b1* ∈ *S* ∧ *b2* ∈ *S* ∧ *b3* ∈ *S*
by (*meson F4 bcup list.set-intros*(*1*,*2*) *set-mono-sublist subset-code*(*1*))
have *fb2*:*sdistinct* [*b1*, *b2*, *b3*] **using** *bcup*(*1*) *F3 sdistinct-subl*
by *presburger*

have *fb3*:*cross3 b1 b3 b2* ≠ *0* **using** *S-gp fb1 cross3-commutation-23 fb2 genpos-cross0* **by** (*metis distinct-map*)
have *fb4*:*length* [*b1*, *b2*, *b3*] = *3* **using** *fb2* **by** *simp*
have *cap3 b1 b2 b3* **using** *fb3 bcup*(*2*) *cap3-def cup3-def not-less-iff-gr-or-eq*
by (*metis cross3-commutation-23*)
hence *cap 3* [*b1*, *b2*, *b3*] **unfolding** *cap-def* **using** *fb2 fb4* **by** *auto*
thus *?thesis* **by** (*meson F4 bcup*(*1*) *dual-order.trans fb2 set-mono-sublist*)
qed
qed
thus *?thesis*
using *cap-def cup-def* **by** *auto*
qed

lemma *min-conv-base-IH*:
assumes *min-conv 3 k* = *k* ∧ *min-conv-set 3 k* ≠ {}
shows *min-conv 3* (*Suc k*) ≤ *Suc k* ∧ *min-conv-set 3* (*Suc k*) ≠ {}
proof
have *1*:∀ *S*. (*card S* ≥ *Suc k*) ∧ *general-pos S* ∧
sdistinct(*sorted-list-of-set S*)
⟶ (∃ *xs*. (*set xs* ⊆ *S*) ∧ *sdistinct xs* ∧ (*cap 3 xs* ∨ *cup* (*Suc k*) *xs*))
using *min-conv-base-IH-aux assms sdistinct-sub general-pos-subs*
by (*smt* (*verit*, *ccfv-SIG*) *Suc-le-D card.infinite ex-card nat.distinct*(*1*) *subset-trans*)
thus *min-conv 3* (*Suc k*) ≤ *Suc k* **using** *min-conv-def*[*of 3 Suc k*] *assms inf-upper* **by** *simp*
hence *Suc k* ∈ *min-conv-set 3* (*Suc k*) **using** *1 min-conv-set-def* **by** *simp*

thus *min-conv-set 3 (Suc k) ≠ {}* **by** *blast*
qed

lemma *min-conv-3-k-bnd*:
assumes *min-conv-set 3 (Suc k) ≠ {}*
shows *min-conv 3 (Suc k) > k*
proof−
have ∃ *S*. (*card S = k* ∧ *general-pos S* ∧ *sdistinct*(*sorted-list-of-set S*))
∧ (∀ *xs*. *set xs* ⊆ *S* ∧ *sdistinct xs* ⟶ ¬(*cap 3 xs* ∨ *cup* (*Suc k*) *xs*))
proof−
obtain *L* **where** *Trp*:*length* (*L* :: *real list*) = *k sortedStrict L* **using** *real-set-ex set2list*
by (*metis sorted-list-of-set.length-sorted-key-list-of-set*
sorted-list-of-set.strict-sorted-key-list-of-set)
define *V2L* **where** *v2lp*:*V2L* = *map v2 L*
hence *v2l-k*: *length V2L* = *k* **using** *Trp* **by** *force*
hence *v2l-ss*: *sdistinct V2L* **using** *Trp*(*2*) *v2lp f-prop11* **by** *simp*
hence *cup k V2L* **using** *f-prop4 v2lp v2l-k f-prop12* **by** *blast*
have *v2l-gp*: *general-pos* (*set V2L*) **using** *f-prop6 Trp*(*2*)*f-prop11*
by (*simp add*: *gpos-generalpos v2lp*)
have *v2l-ssgp*:*sdistinct*(*sorted-list-of-set* (*set V2L*)) **using** *v2l-ss*
by (*simp add*: *distinct-map*)
have (∀ *xs*. (*set xs* ⊆ *set V2L*) ⟶ ¬(*cap 3 xs* ∨ *cup* (*Suc k*) *xs*))
proof−
{
fix *xs*
assume *xsp*:*set xs* ⊆ *set V2L*
hence *length xs* > *k* ⟹ ¬*sdistinct xs* **using** *v2l-k*
by (*metis List.finite-set card-mono distinct-card distinct-map linorder-not-less v2l-ss*)
hence *1*:¬ *cup* (*Suc k*) *xs* **using** *cup-def* **by** *force*
have ¬ *cap 3 xs*
proof(*cases length xs* = *3* ∧ *sdistinct xs*)
case *True*
hence *length xs* = *3* **by** *simp*
then obtain *a b c* **where** *xs3*: *xs* = [*a*,*b*,*c*] **using** *xsp*
by (*smt* (*verit*, *best*) *List.finite-set True distinct.simps*(*2*) *distinct-map length-0-conv length-Cons*
nat.inject nat.simps(*3*) *numeral-3-eq-3 set-empty sorted-list-of-set.idem-if-sorted-distinct*
sorted-list-of-set.sorted-sorted-key-list-of-set sorted-list-of-set-nonempty)
then obtain *u v w* **where** *uvw*: *a* = *v2 u b* = *v2 v c* = *v2 w* **using** *xsp v2lp* **by** *auto*
hence *sdistinct*[*v2 u*, *v2 v*, *v2 w*] **using** *xsp xs3 True* **by** *simp*
hence *cup3 a b c* **using** *f-prop4 uvw f-prop10* **by** *presburger*
thus ¬(*cap 3 xs*) **using** *True exactly-one-true cap-def xs3* **by** (*metis list-check.simps*(*4*))

When *length xs* ≠ *3* then xs can not be a 3-cap from definition trivially.

qed(*auto simp add*: *cap-def*)

```
      hence set xs ⊆ set V2L ∧ sdistinct xs ⟶ ¬ (cap 3 xs ∨ cup (Suc k) xs)
using 1
       by blast
     }
     thus ?thesis
      by (simp add: cap-def cup-def)
   qed
   hence (∀ xs. (set xs ⊆ set V2L) ⟶ ¬(cap 3 xs ∨ cup (Suc k) xs)) by simp
   thus ?thesis using v2l-gp Trp(1) v2l-ss v2l-k
    by (metis distinct-card distinct-map v2l-ssgp)
   qed
 thus ?thesis using min-conv-lower min-conv-set-def assms by simp
qed

theorem min-conv-base:
 min-conv 3 k = k ∧ min-conv-set 3 k ≠ {}
proof(induction k)
 case 0
 show min-conv 3 0 = 0 ∧ min-conv-set 3 0 ≠ {}
  using min-conv-0[of 3] min-conv-arg-swap[of 3 0] min-conv-sets-eq min-conv-set-def

   by simp
next
 case (Suc k)
 hence min-conv 3 (Suc k) ≤ Suc k ∧ min-conv-set 3 (Suc k) ≠ {}
   using min-conv-def[of 3 Suc k] Suc inf-upper min-conv-base-IH by blast
 moreover have min-conv 3 (Suc k) ≥ Suc k
   using Suc-leI min-conv-3-k-bnd calculation by presburger
 ultimately show ?case by simp
qed


end
theory SlopeCapCup
 imports Definitions Prelims Invariants

begin



lemma slope-cross3:
 assumes sdistinct [a,b,c]
 shows   cross3 a b c  = (fst b − fst a) ∗ (fst c − fst b) ∗ (slope b c − slope a b)
proof−
 have 1:fst a ≠ fst b fst b ≠ fst c using assms(1) by simp+
 have cross3 a b c = (fst b − fst a) ∗ (snd c − snd b) − (fst c − fst b) ∗ (snd b
− snd a) using cross3-def by blast
 also have ... = (fst b − fst a) ∗ (fst c − fst b) ∗ (slope b c − slope a b) using 1
assms unfolding slope-def by (simp add: diff-frac-eq mult.commute)
```

ultimately show *?thesis*
by *presburger*
qed

lemma *cup3-slope*:
assumes *sdistinct* [*x*,*y*,*z*] *cup3 x y z*
shows *slope x y < slope y z*
using *assms*
by (*smt* (*verit*) *cup3-def less-eq-prod-def mult-less-0-iff slope-cross3 sorted2 zero-less-mult-iff*)

lemma *cap3-slope*:
assumes *sdistinct* [*x*,*y*,*z*] *cap3 x y z*
shows *slope x y > slope y z*
using *assms*
by (*smt* (*verit*) *cap3-def less-eq-prod-def mult-less-0-iff slope-cross3 sorted2 zero-less-mult-iff*)

lemma *slope-cup3*:
assumes *sdistinct* [*x*,*y*,*z*] *slope x y < slope y z*
shows *cup3 x y z* **using** *slope-cross3 assms*(*1*,*2*) *cup3-def*
by (*smt* (*verit*, *del-insts*) *distinct-length-2-or-more less-eq-prod-def list.simps*(*9*) *sorted2 zero-less-mult-iff*)

lemma *slope-cap3*:
assumes *sdistinct* [*x*,*y*,*z*] *slope x y > slope y z*
shows *cap3 x y z*
proof−
have *1*:*fst x ≠ fst y fst y ≠ fst z* **using** *assms*(*1*) **by** *simp*+
have *0 > slope y z − slope x y* **using** *assms*(*2*) **by** *simp*
hence *0 > (fst y − fst x) ∗ (fst z − fst y) ∗ (slope y z − slope x y)*
by (*metis 1*(*1*,*2*) *assms*(*1*) *diff-gt-0-iff-gt less-eq-prod-def linorder-neqE-linordered-idom mult-less-cancel-right-disj mult-zero-left nle-le sorted2*)
thus *?thesis*
using *1 assms*(*2*) *cup3-def cross3-def*[*of x y z*] *slope-def*[*of x y*] *slope-def*[*of y z*]
by (*smt* (*verit*, *best*) *cap3-def diff-frac-eq divide-cancel-right mult.commute nonzero-mult-div-cancel-left right-minus-eq*)
qed

lemma *slope-coll3*:
assumes *sdistinct* [*x*,*y*,*z*] *slope x y = slope y z*
shows *collinear3 x y z* **using** *slope-cup3 slope-cap3 exactly-one-true*
by (*smt* (*verit*, *del-insts*) *assms*(*1*,*2*) *collinear3-def cross3-def diff-frac-eq distinct-length-2-or-more divide-cancel-right list.simps*(*9*) *mult.commute mult-eq-0-iff slope-def*)

lemma *slopeinc-is-cup*:
assumes *sdistinct xs* ∀ *x y z. subseq* [*x*,*y*,*z*] *xs* ⟶ *slope x y < slope y z*

shows *cup* (*length xs*) *xs*
proof−
have *sublist* [*x*, *y*, *z*] *xs* ⟶ *sdistinct*[*x*, *y*, *z*]
using *assms*(*1*) *sdistinct-subl* **by** *blast*
hence *sublist*[*x*, *y*, *z*] *xs* ⟶ *cup3 x y z* **using** *slope-cup3 assms*(*2*) **by** *blast*
hence *list-check cup3 xs* **using** *assms*(*1*,*2*) *bad3-from-bad sdistinct-subl slope-cup3*
by (*metis sublist-imp-subseq*)
thus *cup* (*length xs*) *xs* **using** *assms cup-def* **by** *simp*
qed

lemma *list-check-cup3-slope*:
assumes *sdistinct xs list-check cup3 xs*
shows ⋀*i*. *i* < *length xs* − *2* ⟶ *slope* (*xs*!*i*) (*xs*!(*i*+*1*)) < *slope* (*xs*!(*i*+*1*)) (*xs*!(*i*+*2*))
using *assms*
proof(*induction xs*)
case (*Cons a xs*)
show *?case*
proof(*cases length* (*a*#*xs*) ≥ *3*)
case *True*
have *sdistinct xs* **using** *Cons*(*2*) **by** *simp*
also have *list-check cup3 xs* **using** *Cons*(*3*)
using *list-check.elims*(*3*) **by** *fastforce*
ultimately have
⋀*i*. *i* < *length xs* − *2* ⟶
slope (*xs* ! *i*) (*xs* ! (*i* + *1*)) < *slope* (*xs* ! (*i* + *1*)) (*xs* ! (*i* + *2*))
using *Cons*(*1*) **by** *simp*
hence *1*:⋀*i*. *i* <*length xs* − *2* ⟶
slope ((*a*#*xs*) ! (*i*+*1*)) ((*a*#*xs*) ! (*i*+*2*)) < *slope* ((*a*#*xs*) ! (*i*+*2*)) ((*a*#*xs*) ! (*i* + *3*))
by *simp*

hence *2*:⋀*j i*. *j* ≥ *1* ∧ *j* < *1* + *length* (*xs*) − *2* ∧ *j* = *i*+*1* ⟶
slope ((*a*#*xs*) ! *j*) ((*a*#*xs*) ! (*j* + *1*)) < *slope* ((*a*#*xs*) ! (*j* + *1*)) ((*a*#*xs*) ! (*j* + *2*))
by *fastforce*
hence *3*:⋀*j*. *j* ≥ *1* ∧ *j* < *length* (*a*#*xs*) − *2* ⟶
slope ((*a*#*xs*) ! *j*) ((*a*#*xs*) ! (*j* + *1*)) < *slope* ((*a*#*xs*) ! (*j* + *1*)) ((*a*#*xs*) ! (*j* + *2*))
by (*metis Suc-eq-plus1 length-Cons nat-le-iff-add*
plus-1-eq-Suc)
hence *7*:*sdistinct* [(*a*#*xs*)!*0*, (*a*#*xs*)!*1*, (*a*#*xs*)!*2*] **using** *Cons*(*2*) *True ind-seq-subseq*
by (*metis Suc-1 Suc-le-eq diff-Suc-1 diff-is-0-eq le-add2*
nless-le numeral-3-eq-3 plus-1-eq-Suc)
hence *cup3* ((*a*#*xs*)!*0*) ((*a*#*xs*)!*1*) ((*a*#*xs*)!*2*) **using** *Cons*(*3*)
by (*smt* (*verit*, *ccfv-SIG*) *One-nat-def Suc-1 Suc-eq-plus1*
True diff-Suc-1 diff-add-0 diff-is-0-eq list.size(*3*,*4*)
list-check.elims(*2*) *not-less-eq-eq nth-Cons-0*
nth-Cons-Suc numeral-3-eq-3)

hence *slope* ((*a*#*xs*) ! *0*) ((*a*#*xs*) ! (*1*)) < *slope* ((*a*#*xs*) ! (*1*)) ((*a*#*xs*) ! (*2*))
using *Cons*(*3*) *cup3-slope 7* **by** *simp*

then show *?thesis* **using** *1 2 3*
by (*metis One-nat-def Suc-eq-plus1 add-2-eq-Suc' bot-nat-0.extremum-unique length-Cons not-less-eq-eq nth-Cons-Suc plus-1-eq-Suc*)
qed(*auto*)
qed(*simp*)

lemma *farey-prop1*:
fixes *n1 n2 d1 d2* :: *real*
assumes *d1* > *0* **and** *d2* > *0* **and** *n1* / *d1* < *n2* / *d2*
shows *n1* / *d1* < (*n1*+*n2*) / (*d1*+*d2*)
proof−
have *n1* ∗ *d2* < *d1* ∗ *n2* **using** *assms*
by (*smt* (*verit*) *divide-pos-pos frac-le-eq mult.commute mult-le-0-iff*)
hence *n1* ∗ *d2* + *n1* ∗ *d1* < *d1* ∗ *n2* + *d1* ∗ *n1* **by** *simp*
hence *n1* ∗ (*d1* + *d2*) < *d1* ∗ (*n1* + *n2*) **by** *argo*
thus *?thesis*
by (*smt* (*verit*, *best*) *assms*(*1*,*2*) *divide-pos-pos frac-le-eq mult.commute mult-le-0-iff*)
qed

lemma *farey-prop2*:
fixes *n1 n2 d1 d2* :: *real*
assumes *d1* > *0* **and** *d2* > *0* **and** *n1* / *d1* < *n2* / *d2*
shows (*n1*+*n2*) / (*d1*+*d2*) < *n2* / *d2*
by (*smt* (*verit*, *best*) *assms*(*1*,*2*,*3*) *divide-minus-left farey-prop1*)

lemma *farey-pos-coeff*:
fixes *β n1 n2 d1 d2* :: *real*
assumes *β* ≥ *0 d1* > *0* **and** *d2* > *0* **and** *n1* / *d1* < *n2* / *d2*
shows *n1* / *d1* ≤ (*n1* + *β*∗*n2*) / (*d1* + *β*∗*d2*) (*n1* + *β*∗*n2*) / (*d1* + *β*∗*d2*) < *n2*/*d2*
using *farey-prop1 farey-prop2 nle-le*
by (*metis mult-pos-pos mult-zero-left order.strict-iff-order add.right-neutral mult-divide-mult-cancel-left-if assms*)+

lemma *slope-trans*:
assumes *sdistinct* [*a*,*b*,*c*]
and *slope a b* < *slope b c*
shows *slope a b* < *slope a c*
slope a c < *slope b c*
using *assms farey-prop2 farey-prop1*
unfolding *slope-def*
by (*smt* (*verit*) *distinct-length-2-or-more less-eq-prod-def list.simps*(*9*) *sorted2*)+

lemma *slope-trans-fwd*:

assumes *cup* (*length xs*) *xs* **and** *i+1* < *j* ∧ *j* < *length xs*
shows *slope* (*xs*!*i*) (*xs*!(*j*−*1*)) < *slope* (*xs*!(*j*−*1*)) (*xs*!*j*)
using *assms*(*2*)
proof(*induction j*−*i arbitrary*: *j*)
case *0*
then show *?case* **by** *simp*
next
case (*Suc x*)
hence *F1*:*x* = (*j*−*1*) − *i* **by** *simp*
then show *?case*
proof(*cases i+3* ≤ *j*)
case *True*
hence *F2*:*i* + *1* < (*j*−*1*) ∧ (*j*−*1*) < *length xs* **using** *Suc*(*3*) **by** *linarith*
hence *F3*:*slope* (*xs* ! *i*) (*xs* ! (*j* − *2*)) < *slope* (*xs* ! (*j* − *2*)) (*xs* ! (*j*−*1*))
using *F1 Suc*(*1*) **by** (*metis Suc-1 diff-diff-left plus-1-eq-Suc*)
have *sdistinct* [*xs*!*i*, *xs*!(*j*−*2*), *xs*!(*j*−*1*)] **using** *assms*(*1*) *F2 cup-def ind-seq-subseq*
by (*metis* (*no-types*, *lifting*) *Suc-1 Suc-diff-Suc add-leE diff-diff-left less-diff-conv*
less-eq-Suc-le linorder-not-less plus-1-eq-Suc)
hence *F4*:*slope* (*xs* ! *i*) (*xs* ! (*j* − *1*)) < *slope* (*xs* ! (*j* − *2*)) (*xs* ! (*j*−*1*))
using *slope-trans F3* **by** *blast*
moreover have ... < *slope* (*xs* ! (*j* − *1*)) (*xs* ! *j*)
by (*smt* (*verit*, *ccfv-threshold*) *Suc.prems Suc-eq-plus1*
add-2-eq-Suc' assms(*1*) *cup-def diff-Suc-1 diff-is-0-eq*
less-diff-conv less-natE list-check-cup3-slope nless-le
plus-1-eq-Suc verit-comp-simplify1(*3*))
finally show *?thesis* **by** *simp*
next
case *False*
then show *?thesis*
using *Suc.prems assms*(*1*) *cup-def list-check-cup3-slope*
by (*smt* (*verit*, *ccfv-SIG*) *Suc-1 add.commute add-Suc-right diff-zero less-Suc-eq*

less-diff-conv not-less-eq numeral-3-eq-3 plus-1-eq-Suc verit-comp-simplify1(*3*))
qed
qed

lemma *slope-trans-bkd*:
assumes *cup* (*length xs*) *xs* **and** *j+1* < *k* ∧ *k* < *length xs*
shows *slope* (*xs*!*j*) (*xs*!(*j+1*)) < *slope* (*xs*!*j*) (*xs*!*k*)
using *assms*(*2*)
proof(*induction k*−*j arbitrary*: *k*)
case *0*
then show *?case* **by** *simp*
next
case (*Suc x*)
hence *F1*:*x* = (*k*−*1*) − *j* **by** *simp*
then show *?case*
proof(*cases j+3* ≤ *k*)
case *True*

```
    hence F2:j + 1 < (k−1) ∧ (k−1) < length xs using Suc(3) by linarith
    have F4:sdistinct [xs!j, xs!(k−1), xs!k]
      using assms(1) F2 cup-def ind-seq-subseq
      by (metis (no-types, opaque-lifting) Suc.prems Suc-pred' nless-le
          bot-nat-0.extremum-strict less-diff-conv not-less-eq-eq lessI)
    hence F3:slope (xs ! j) (xs ! (j + 1)) < slope (xs ! j) (xs ! (k−1))
      using F1 F2 by (metis diff-diff-left plus-1-eq-Suc Suc(1))
    have F5:slope (xs ! j) (xs ! (k−1)) < slope (xs!(k−1)) (xs!(k))
      using Suc.prems assms(1) slope-trans-fwd by blast
    thus ?thesis using F3 F4 slope-trans(1)
      by (meson dual-order.strict-trans)
  next
    case False
    hence j+2 = k
      using Suc.prems by linarith
    then show ?thesis
      using Suc.prems assms(1) cup-def slope-trans(1) slope-trans-fwd
      by (smt (verit, ccfv-threshold) Suc-1 add-Suc add-Suc-shift diff-Suc-numeral
          diff-zero ind-seq-subseq less-add-one numeral-One pred-numeral-simps(1))
  qed
qed


theorem cup-is-slopeinc:
  assumes cup (length xs) xs and i < j ∧ j < k ∧ k < length xs
  shows   slope (xs!i) (xs!j) < slope (xs!j) (xs!k)
proof−
  have 1:0 < j j+1 < length xs using assms(2) by simp+
  hence 2:slope (xs!(j−1)) (xs!j) < slope (xs!j) (xs!(j+1))
    by (metis (no-types, lifting) Suc-diff-Suc Suc-eq-plus1 add.commute assms(1)
diff-add-inverse less-add-one plus-nat.add-0 slope-trans-fwd)
  have 3:i+1 < j ⟹ slope (xs!i) (xs!(j−1)) < slope (xs!(j−1)) (xs!j)
    using assms slope-trans-fwd by simp
  have 4:i+1 = j ⟹ ?thesis using 2 assms slope-trans-bkd
    by (smt (verit, ccfv-threshold) Suc-eq-plus1 Suc-lessI add-diff-cancel-right')
  have 5:j+1 < k ⟹ slope (xs!j) (xs!(j+1)) < slope (xs!j) (xs!k)
    using assms slope-trans-bkd by simp
  have   j+1 = k ⟹ ?thesis
    using assms(1,2) slope-trans-fwd by fastforce
  thus ?thesis using 3 4 5 slope-trans-fwd assms Suc-eq-plus1
    by (smt (verit, best) add-diff-cancel-right' less-trans-Suc linorder-neqE-nat
not-less-eq)
qed

corollary cup-is-slopeinc-subseq:
  assumes cup (length xs) xs and subseq [a,b,c] xs
  shows   slope a b < slope b c
  using subseq-index3 cup-is-slopeinc
  by (metis assms(1,2))
```

lemma *cup-sub-cup*:
assumes *cup* (*length xs*) *xs* **and** *subseq ys xs*
shows *cup* (*length ys*) *ys*
proof−
have *1*:*sdistinct ys* **using** *sdistinct-subseq assms cup-def* **by** *simp*
have ⋀*a b c*. *sublist* [*a*,*b*,*c*] *ys* ⟹ *slope a b* < *slope b c*
proof−
fix *a b c*
assume *sublist* [*a*,*b*,*c*] *ys*
hence *subseq* [*a*,*b*,*c*] *xs*
using *assms*(*2*) *subseq-order.dual-order.trans* **by** *blast*
thus *slope a b* < *slope b c* **using** *assms*(*1*) *cup-is-slopeinc subseq-index3*
by *metis*
qed
thus *?thesis* **using** *1 slopeinc-is-cup get-cap-from-bad-cup sdistinct-subl slope-cup3*
by *meson*
qed

lemma *slopedec-is-cap*:
assumes *sdistinct xs*
and ∀*x y z*. *subseq* [*x*,*y*,*z*] *xs* ⟶ *slope x y* > *slope y z*
shows *cap* (*length xs*) *xs*
using *assms*
by (*metis get-cup-from-bad-cap sdistinct-subl slope-cap3 sublist-imp-subseq*)

lemma *cap-sub-cap*:
assumes *cap* (*length xs*) *xs* **and** *subseq ys xs*
shows *cap* (*length ys*) *ys*
by (*metis assms*(*1*,*2*) *cap-orig-refl-rev cup-sub-cup length-neg length-rev subseq-map subseq-rev*)

end
theory *MinConv*
imports *EZ-bound SlopeCapCup Invariants*

begin

lemma *cap-endpoint-subset*:
assumes *l*≥*2*
and *Y* = {*Min* (*set xs*) | *xs*. *set xs* ⊆ *X* ∧ *sdistinct xs* ∧ *cup* (*l* − *1*) (*xs*)}
shows *Y* ⊆ *X*
proof
fix *x*
assume *asm*: *x* ∈ *Y*
then obtain *xs* **where** *xsp*: *x* = *Min* (*set xs*) *set xs* ⊆ *X cup* (*l*−*1*) (*xs*)

using *assms* **by** *blast*
hence *1*: *set xs* $\neq$ {} **using** *assms* **unfolding** *cup-def* **by** *fastforce*
hence *finite* (*set xs*) **using** *xsp*(*3*) *cup-def assms*(*1*) **by** *fastforce*
thus *x*$\in$*X* **using** *1 xsp Min-in asm* **by** *fastforce*
qed

lemma *min-conv-lower-bnd*:
assumes *k*$\geq$*3* **and** *l*$\geq$*3*
and *min-conv-set* (*k*$-$*1*) *l* $\neq$ {}
and *min-conv-set k* (*l*$-$*1*) $\neq$ {}
shows *min-conv k l* $\leq$ *min-conv* (*k* $-$ *1*) *l* + *min-conv k* (*l* $-$ *1*) $-$ *1* (**is** *?a* $\leq$ *?b*)
min-conv-set k l $\neq$ {}
using *inf-upper*
proof
show *gg*:*?b* $\in$ {*n*. $\forall$ *S* . (*card S* = *n* $\wedge$ *general-pos S* $\wedge$ *sdistinct*(*sorted-list-of-set S*))
$\longrightarrow$ ($\exists$ *xs*. *set xs* $\subseteq$ *S* $\wedge$ *sdistinct xs* $\wedge$ (*cap k xs* $\vee$ *cup l xs*))}
(**is** *?b* $\in$ {*n*. $\forall$ *S*. *?GSET n S* $\longrightarrow$ ($\exists$ *xs*. *?SS xs S* $\wedge$ *?CUPCAP k xs l*)})

proof

We show that any point set with size *min-conv* (*k* $-$ *1*) *l* + *min-conv k* (*l* $-$ *1*) $-$ *1* contains a k-cap or an l-cup.

show $\forall$ *S*. *card S* = *?b* $\wedge$ *general-pos S* $\wedge$ *sdistinct*(*sorted-list-of-set S*)
$\longrightarrow$ ($\exists$ *xs*. (*set xs* $\subseteq$ *S* $\wedge$ *sdistinct xs* $\wedge$ (*cap k xs* $\vee$ *cup l xs*)))
proof$-$
{
fix *X*
assume *asm*: *?b* = *card X general-pos X sdistinct*(*sorted-list-of-set X*)
hence *A1*: *?b* $\leq$ *card X* **by** *simp*
have *X-fin*:*finite X* **using** *assms*(*1*,*2*,*3*) *asm*(*1*) *min-conv-min*
by (*smt* (*verit*) *add-leE le-add-diff-inverse card.infinite diff-is-0-eq min-def not-less-eq-eq numeral-3-eq-3 plus-1-eq-Suc*)

define *Y* **where** *ys*: *Y* = {*Min* (*set xs*) | *xs*. *set xs* $\subseteq$ *X* $\wedge$ *sdistinct xs* $\wedge$ *cup* (*l* $-$ *1*) *xs*}
hence *ysx*: *Y*$\subseteq$*X* **using** *cap-endpoint-subset* **using** *asm assms* **by** *auto*
hence *ygen*: *general-pos Y* **using** *asm*(*2*) *general-pos-subs* **by** *presburger*
have *Y-fin*: *finite Y* **using** *X-fin rev-finite-subset ysx* **by** *blast*

have $\exists$ *xs*. *?SS xs X* $\wedge$ *?CUPCAP k xs l*
proof(*cases* $\exists$ *xs*. *?SS xs X* $\wedge$ (*cap k xs* $\vee$ *cup l xs*))
case *True*
then show *?thesis*
using *cap-def cup-def* **by** *auto*
next
case *outerFalse*:*False*
then show *?thesis*

proof(*cases card* $(X-Y) \geq$ *min-conv k* $(l-1)$)

case *True*
have *xy1*: *general-pos* $(X-Y)$ **using** *general-pos-subs ysx asm(2)* **by** *blast*
have *XmY-fin*: *finite* $(X-Y)$ **using** *X-fin* **by** *blast*

The following is not possible as X-Y can not have a (l-1)-cup as their left points have been put in Y.

hence $\exists$ *xs. set xs* $\subseteq (X-Y) \wedge$ *sdistinct xs* $\wedge$ *cup* $(l-1)$ *xs*
using *outerFalse True genpos-ex gpos-generalpos min-conv-num-out sdistinct-sub X-fin asm(3) xy1 assms(4)*
by (*smt* (*verit*, *del-insts*) *Diff-subset dual-order.trans*)
then obtain *lcs* **where** *lcs*: *set lcs* $\subseteq (X-Y)$ *cup* $(l-1)$ *lcs* **by** *blast*
hence *C1*: *Min* (*set lcs*) $\in (X-Y)$
by (*smt* (*verit*, *ccfv-SIG*) *List.finite-set Min-in One-nat-def assms(2) cup-def diff-Suc-Suc diff-less-mono in-mono less-Suc-eq less-nat-zero-code list.size(3) numeral-3-eq-3 set-empty2*)
have *C2*: *Min* (*set lcs*) $\in$ *Y*
using *lcs ys cup-def* **by** *auto*
show *?thesis* **using** *C1 C2* **by** *simp*

next

case *False*
hence *2*:*min-conv* (k) $(l-1) \geq$ *card* $(X - Y) + 1$ **by** *simp*

have *Y-sd*:*sdistinct*(*sorted-list-of-set Y*)
using *asm(3) sdistinct-sub ysx X-fin*
by *presburger*

have *card* $(X - Y) =$ *card X* $-$ *card Y* **using** *ysx card-def 2 asm(1)*
by (*smt* (*verit*, *ccfv-SIG*) *Suc-eq-plus1 add.commute card.infinite card-Diff-subset diff-0-eq-0 diff-Suc-1 diff-is-0-eq double-diff finite-Diff le-antisym subset-refl trans-le-add2*)
hence *card Y* $\geq$ *min-conv* $(k-1)$ *l* **using** *2 asm(1)* **by** *linarith*

hence *3*:$\exists$ *xs. set xs* $\subseteq$ *Y* $\wedge$ *sdistinct xs* $\wedge$ (*cap* $(k-1)$ *xs*)
using *ygen outerFalse genpos-ex gpos-generalpos ysx min-conv-num-out sdistinct-sub X-fin Y-sd extract-cap-cup assms(2,3)*
by (*metis* (*full-types*) *dual-order.trans*)

then obtain *kap* **where** *kap*: *set kap* $\subseteq$ *Y* (*cap* $(k-1)$ *kap*) **by** *blast*

hence *k0*:*sdistinct kap sorted kap distinct* (*map fst kap*) **using** *cap-def* **by** *simp+*
have *k1*:*length kap* $= k-1$ **using** *kap cap-def* **by** *auto*

hence *k2*:*kap*!(*k*−*2*) ∈ *Y* **using** *kap*
by (*metis One-nat-def Suc-1 Suc-diff-Suc Suc-le-lessD add-leE assms*(*1*) *lessI nth-mem numeral-3-eq-3 plus-1-eq-Suc subset-iff*)
then obtain *lup* **where** *lup*:*kap*!(*k*−*2*) = *Min* (*set lup*) *set lup* ⊆ *X cup* (*l* − *1*) (*lup*)
using *ys* **by** *auto*
hence *lup-sd*:*sdistinct lup* **using** *cup-def* **by** *simp*
have *k3*:*Min* (*set lup*) = *lup*!*0* **using** *assms*(*2*) *k0*(*2*) *lup*
by (*metis List.finite-set One-nat-def add-leE card.empty cup-def distinct-card distinct-map le-add-diff-inverse nth-Cons-0 numeral-3-eq-3 numeral-le-one-iff plus-1-eq-Suc semiring-norm*(*70*) *sorted-list-of-set.idem-if-sorted-distinct sorted-list-of-set-nonempty*)
have *k7*:*lup*!*1* ∈ *X* **using** *lup assms*(*2*) *cup-def*
by (*metis One-nat-def Suc-le-eq less-diff-conv nth-mem numeral-3-eq-3 plus-1-eq-Suc subsetD*)

have *k4*:(*kap*!(*k*−*3*)) < (*kap*!(*k*−*2*)) **using** *assms kap cap-def k0*(*2*)
by (*metis* (*no-types*, *lifting*) *One-nat-def Suc-1 Suc-diff-Suc Suc-le-lessD add-leE distinct-map lessI numeral-3-eq-3 plus-1-eq-Suc sorted-wrt-nth-less strict-sorted-iff*)
have *k5*:(*kap*!(*k*−*2*)) < (*lup*!*1*) **using** *assms lup cup-def k0*(*2*)
by (*metis One-nat-def Suc-le-lessD distinct-map k3 lessI less-diff-conv numeral-3-eq-3 plus-1-eq-Suc sorted-wrt-nth-less strict-sorted-iff*)
hence *k61*:*sorted* [(*kap*!(*k*−*3*)),(*kap*!(*k*−*2*)),(*lup*!*1*)] **using** *lup kap asm*
by (*meson k4 nless-le sorted1 sorted2*)
have *aa1*:*distinct* [(*kap*!(*k*−*3*)),(*kap*!(*k*−*2*)),(*lup*!*1*)] **using** *k4 k5* **by** *auto*
have *k62*:*distinct* (*map fst* [(*kap*!(*k*−*3*)),(*kap*!(*k*−*2*)),(*lup*!*1*)])
proof−
have {(*kap*!(*k*−*3*)),(*kap*!(*k*−*2*)),(*lup*!*1*)} ⊆ *X*
using *assms*(*1*) *k1 k7 kap*(*1*) *ysx* **by** *force*
thus *?thesis* **using** *k61 aa1 asm*(*3*) *list.simps*(*15*) *sdistinct-sub X-fin*
by (*metis empty-set sorted-list-of-set.idem-if-sorted-distinct*)

qed
hence *sdistinct* [(*kap*!(*k*−*3*)),(*kap*!(*k*−*2*)),(*lup*!*1*)] **using** *k61* **by** *simp*
hence *k-noncoll*:¬ *collinear3* (*kap*!(*k*−*3*)) (*kap*!(*k*−*2*)) (*lup*!*1*) **using** *aa1*
by (*smt* (*verit*, *best*) *Suc-diff-le Suc-eq-plus1 Suc-le-eq asm*(*2*) *assms*(*1*) *diff-Suc-Suc diff-diff-left diff-le-self diff-zero gpos-def gpos-generalpos k1 k2 k7 kap*(*1*) *nth-mem numeral-3-eq-3 subset-iff ysx*)

thus *?thesis*
proof(*cases cap3* (*kap*!(*k*−*3*)) (*kap*!(*k*−*2*)) (*lup*!*1*))

case *True*
hence *k8*:*cap k* (*kap*@[*lup*!*1*]) **using** *kap cap-def*
proof−

have *k-rev*: *rev* ((*lup*!*1*) # (*rev kap*)) = (*kap*@[*lup*!*1*]) **by** *force*
have *k-lup-len*: *length* (*kap*@[*lup*!*1*]) = *k* **using** *k1 k5*
using *assms*(*1*) **by** *fastforce*
have *kl-s*:*sortedStrict* (*kap*@[*lup*!*1*])
using *k1 k3 lup*(*1*) *k5 kap*(*2*) *assms append-last-sortedStrict*
by (*metis Suc-1 diff-diff-left diff-is-0-eq distinct-map k0*(*1*) *k-lup-len last-conv-nth list.size*(*3*) *plus-1-eq-Suc sorted-wrt01 strict-sorted-iff*)
have *kl-inX*: *set* (*kap*@[*lup*!*1*]) ⊆ *X*
using *k7 kap*(*1*) *ysx* **by** *auto*
hence *k29*:*sdistinct* (*kap*@[*lup*!*1*])
using *kl-s asm*(*3*) *sdistinct-sub X-fin*
by (*metis*
sorted-list-of-set.idem-if-sorted-distinct
strict-sorted-iff)
thus *?thesis*
using *k29 k-lup-len True cap-def extend-cap-cup k-rev kap*
by (*metis* (*no-types*, *lifting*) *One-nat-def Suc-1 diff-diff-left distinct-map numeral-3-eq-3 plus-1-eq-Suc strict-sorted-iff*)
qed
have *set kap* ⊆ *X* **using** *kap*(*1*) *ysx* **by** *blast*
hence *set* (*kap*@[*lup*!*1*]) ⊆ *X* **using** *k7* **by** *force*
then show *?thesis* **using** *k1 assms*(*1*) *cap-def lup kap ysx order-trans k8* **by** *blast*
next
case *False*
hence *k9*:*cup l* (*kap*!(*k*−*3*)#*lup*)
proof−
have *k-kap-len*: *length* (*kap*!(*k*−*3*)#*lup*) = *l* **using** *k4 lup*(*1*) *k3 lup cup-def*
using *assms*(*2*) **by** *auto*
have *k-kap-d*: *sortedStrict* (*kap*!(*k*−*3*)#*lup*)
by (*metis* (*no-types*, *lifting*) *append-first-sortedStrict cup-def diff-is-0-eq distinct-map k3 k4*
k-kap-len list.sel(*1*) *list.size*(*3*) *lup*(*1*,*3*) *nth-Cons-0 sorted-wrt.elims*(*2*) *sorted-wrt01*
strict-sorted-iff)
have *k-kap-inX*: *set* (*kap*!(*k*−*3*)#*lup*) ⊆ *X*
using *assms*(*1*) *asm*(*3*) *cap-def kap*(*1*,*2*) *lup*(*2*) *ysx* **by** *force*
hence *sdistinct* (*kap*!(*k*−*3*)#*lup*) **using** *k-kap-d asm*(*3*) *sdistinct-sub X-fin*
by (*metis*
sorted-list-of-set.idem-if-sorted-distinct
strict-sorted-iff)
thus *?thesis* **using** *False k-noncoll cup-def list-check.simps exactly-one-true k3 k4 k-kap-len lup*
by (*smt* (*verit*) *diff-is-0-eq*′ *verit-comp-simplify1*(*1*) *nth-Cons-numeral numeral-One le-numeral-extra*(*4*) *list-check.elims*(*1*) *nth-Cons-0*)
qed

```
            have set (kap!(k−3)#lup) ⊆ X
              using kap(1) ysx assms(1) k1 lup(2) by force
            then show ?thesis using cup-def k9
              by blast
          qed
        qed
      qed
    }
    thus ?thesis by presburger
  qed
 qed
 thus min-conv k l ≤ min-conv (k − 1) l + min-conv k (l − 1) − 1
  using min-conv-def inf-upper min-conv-leImpe min-conv-set-def by force
 show min-conv-set k l ≠ {} using gg min-conv-set-def min-conv-set-alt-def by
blast
qed

thm min-conv-lower[of a−1 a b]

lemma min-conv-lower-bnd-Suc:
 assumes min-conv-set (k+2) (Suc(l+2)) ≠ {}
  and   min-conv-set (Suc(k+2)) (l+2) ≠ {}

 shows   min-conv (Suc(k+2)) (Suc(l+2))
      ≤ min-conv (k + 2) (Suc(l+2)) + min-conv (Suc(k+2)) (l + 2) − 1
        min-conv-set (Suc(k+2)) (Suc(l+2)) ≠ {}
 using min-conv-lower-bnd(1,2) assms(1,2)
 by (metis (no-types, lifting)
    One-nat-def Suc-1 add-Suc-right diff-Suc-1 le-add2 numeral-3-eq-3, simp)



lemma sorted-union:
 fixes   S1 S2 :: R2 set
 assumes finite S1 and finite S2 and ∀ x∈S1. ∀ y∈S2. fst x < fst y
  and   sdistinct(sorted-list-of-set S1) and sdistinct(sorted-list-of-set S2)
 shows   sorted-list-of-set (S1∪S2) = (sorted-list-of-set S1) @ (sorted-list-of-set
S2)
      ∧ sdistinct(sorted-list-of-set (S1∪S2))
 using assms
proof(induction sorted-list-of-set S1 arbitrary: S1)
 case Nil
 then show ?case
  using sorted-list-of-set.set-sorted-key-list-of-set
  by force
next
 case (Cons a x)
 hence 1:a = Min S1
  using sorted-list-of-set-nonempty by fastforce
```

obtain *S1a* **where** *S1a*:*S1a = S1 − {a}* **using** *Cons(2)* **by** *blast*
hence *2*:*x = sorted-list-of-set S1a* **using** *Cons(2,3) sorted-list-of-set-nonempty*
by *fastforce*

have *3*: *sdistinct* (*sorted-list-of-set S1a*) **using** *S1a Cons.prems(1,4) sdistinct-sub*
by (*meson Diff-subset*)
hence *4*:*sorted-list-of-set* (*S1a∪S2*) = (*sorted-list-of-set S1a*) @ (*sorted-list-of-set S2*)
sdistinct (*sorted-list-of-set* (*S1a∪S2*))
using *Cons S1a 2* **by** *blast+*

have *6*:*sorted-list-of-set* (*S1*) = *a # sorted-list-of-set*(*S1a*) **using** *Cons(2) 2* **by** *simp*
hence *5*:*a#sorted-list-of-set* (*S1a∪S2*) = *a#*(*sorted-list-of-set S1a*) @ (*sorted-list-of-set S2*)
using *4* **by** *blast*

have *7*:∀ *b*∈*S1a*. *fst a < fst b* **using** *1 Cons.prems(1,4) S1a assms(4) 6*
by (*smt* (*verit*, *ccfv-SIG*) *Min-le list.simps(15,15,9)*
distinct.simps(2) insert-Diff-single insert-absorb
insert-iff insert-iff less-eq-prod-def list.set-map
sorted-list-of-set.set-sorted-key-list-of-set)

hence *8*:∀ *b*∈*S2*. *fst a < fst b* **using** *assms(1,2,3)*
by (*metis Cons.hyps(2) Cons.prems(1,3) insertI1 list.simps(15)*
sorted-list-of-set.set-sorted-key-list-of-set)
hence *9*:∀ *b*∈(*S1a∪S2*). *fst a < fst b* **using** *7 8 assms(3)* **by** *fastforce*
hence ∀ *b*∈(*S1a∪S2*). *a ≤ b*
by (*meson 7 UnE 8 less-eq-prod-def*)
hence *10*:*a#sorted-list-of-set* (*S1a∪S2*) = *sorted-list-of-set* (*S1∪S2*)
by (*smt* (*verit*, *del-insts*) *1 4 Cons.prems(1,3) 6*
Diff-insert-absorb Min-in Min-le S1a Un-iff
append-Cons assms(2) empty-not-insert finite-Diff
finite-Un finite-has-minimal insert-absorb
list.simps(15) set-append
sorted-list-of-set.set-sorted-key-list-of-set
sorted-list-of-set-nonempty)
hence *11*:*sorted-list-of-set* (*S1∪S2*) = (*sorted-list-of-set S1*) @ (*sorted-list-of-set S2*)
using *5 6* **by** *simp*

have *fst a* ∉ (*fst* ‘(*S1a* ∪ *S2*)) **using** *9*
by (*metis imageE order-less-irrefl*)
hence *sdistinct*(*sorted-list-of-set* (*S1∪S2*))
using *Cons.prems(1) S1a 10 3 4 9 distinct-def assms(2)*
by (*smt* (*verit*, *ccfv-SIG*) *sorted-list-of-set.sorted-sorted-key-list-of-set set-append*
distinct.simps(2) finite-Diff list.set-map list.simps(9) sorted-list-of-set.set-sorted-key-list-of-set)

```
  then show ?case using 11 by simp
qed

lemma general-pos-union:
  assumes general-pos S1
      and general-pos S2
      and ∀ x∈S1. ∀ y∈S2. fst x < fst y
      and ∀ x∈S1.∀ y∈ S1. ∀ z∈S2. x < y ⟶ slope x y < slope y z
      and ∀ a∈S1. ∀ b∈S2. ∀ c∈S2. b < c ⟶ slope a b > slope b c
      and sdistinct (sorted-list-of-set S1)
      and sdistinct (sorted-list-of-set S2)
      and finite (S1∪S2)
    shows general-pos (S1 ∪ S2)
proof−
  have gp12:  gpos S1 gpos S2 using assms(1,2) gpos-generalpos by simp+
  have gpint: S1 ∩ S2 = {} using assms(3) by blast
  have sd: sdistinct (sorted-list-of-set (S1∪S2)) using assms(3,6,7) sorted-union
    by (metis infinite-Un sorted-list-of-set.fold-insert-key.infinite)
  have gpos (S1∪S2) unfolding gpos-def
  proof(rule+)
    fix a b c
    assume  asm:a ∈ S1 ∪ S2 b ∈ S1 ∪ S2 c ∈ S1 ∪ S2
            distinct [a, b, c] collinear3 a b c
```

This gives us $3! * 2^3 = 48$ cases to verify, we reduce it to 4 cases.

```
    then obtain u v w where uvw:u<v v<w {u,v,w} = {a,b,c}
    by (metis (full-types) distinct-length-2-or-more insert-commute linorder-less-linear)
    hence 0:u < w by simp
    hence 1:sorted-list-of-set {a,b,c} = [u,v,w] using uvw
      by (metis distinct-length-2-or-more distinct-singleton empty-set nless-le
          list.simps(15) sorted1 sorted2 sorted-list-of-set.idem-if-sorted-distinct)
   hence 2:distinct (map fst [u,v,w]) using uvw sdistinct-sub asm(1,2,3) assms(8)
sd
      by (metis bot.extremum insert-subset)
    hence 3:sdistinct [u,v,w] using uvw(1,2) by fastforce

    have 4: collinear3 a b c = collinear3 u v w
      by (metis asm(4,5) coll-is-affDep collinear3-def cross-affine-dependent-conv
uvw(3))
    have 5: S1∩S2 = {} using assms(3) by blast
    have 6: u ∈ S1∪S2 v ∈ S1 ∪ S2 w ∈ S1 ∪ S2
      by (metis asm(1,2,3,4) distinct-is-triple uvw(3))+

    show False
    proof(cases v ∈ S1)
      case True
      have C1:u ∈ S1 using 5 6(1) uvw(1) assms(3)
        by (metis True Un-iff not-less-iff-gr-or-eq prod-less-def)
     have C3:w∈S1 ⟹ False using 4 asm True C1 2 assms(1) distinctFst-distinct
```

```
      by (meson collinear3-def genpos-cross0)
    have w∈S2 ⟹ False using 4 asm True C1 assms(4)
      by (metis 3 collinear3-def cup3-def order-less-irrefl slope-cup3 uvw(1))
    then show ?thesis using C3
      by (metis UnE asm(1,2,3,4) distinct-is-triple uvw(3))
  next
    case False
    hence C1:v ∈ S2 using 6(2) 5 by fast
    hence w ∈ S2 using uvw assms(3) 5 6(3)
      by (meson Un-iff not-less-iff-gr-or-eq prod-less-def)
    then show ?thesis
      by (smt (verit, ccfv-SIG) 3 4 6(1) UnE C1 asm(5) assms(2,5)
          collinear3-def distinctFst-distinct exactly-one-true genpos-cross0
          slope-cap3 uvw(2))
  qed
 qed
 thus general-pos (S1∪S2) using gpos-generalpos by simp
qed

lemma min-conv-upper-bnd:
 assumes min-conv-set (k+2) (Suc(l+2)) ≠ {}
   and   min-conv-set (Suc(k+2)) (l+2) ≠ {}
   and   min-conv-set (Suc(k+2)) (Suc(l+2)) ≠ {}

 shows   min-conv (Suc (k+2)) (Suc (l+2))
       > (min-conv (k+2) (Suc (l+2)) − 1)
       + (min-conv (Suc (k+2)) (l+2) − 1)
proof−
 have min-conv (k + 2) (Suc (l + 2)) > min-conv (k + 2) (Suc (l + 2)) − 1
   using assms(1) min-conv-min add.assoc add-diff-inverse-nat le-add1 by fast-
force
 then obtain S2t where S2t: card S2t = min-conv (k + 2) (Suc (l + 2)) − 1
   general-pos S2t  sdistinct(sorted-list-of-set S2t)
   ∀ xs. set xs ⊆ S2t ∧ (sdistinct xs) ⟶ ¬(cap (k+2) xs ∨ cup (Suc (l+2)) xs)
   by (meson min-conv-lower-imp1o)
 hence S2tf: finite S2t using assms(1) min-conv-min min-conv-lower
     by (smt (verit, ccfv-threshold) One-nat-def Suc-1 Suc-diff-1 add-2-eq-Suc'
card.infinite le-add2 min-def not-less-eq-eq)
 have S2td: distinct (map fst (sorted-list-of-set S2t)) using S2t
   by meson
 have S2t1:card S2t ≥ 1 using S2t(1) assms(1) min-conv-min
   by (smt (verit, best) Suc-1 le-add1 le-add2 min-def
       le-add-diff-inverse nle-le not-less-eq-eq plus-1-eq-Suc)

 have min-conv (Suc (k + 2)) (l + 2) > min-conv (Suc (k + 2)) (l + 2) − 1
   using assms(2) min-conv-min add.assoc add-diff-inverse-nat le-add1 by fast-
force
 then obtain S1 where S1: card S1 = min-conv (Suc (k + 2)) (l + 2) − 1
   general-pos S1 sdistinct(sorted-list-of-set S1)
```

$\forall xs.\ set\ xs \subseteq S1 \wedge (sdistinct\ xs) \longrightarrow \neg(cap\ (Suc\ (k+2))\ xs \vee cup\ (l+2)\ xs)$
by (*meson min-conv-lower-imp1o*)
hence *S1f*: *finite S1* **using** *assms*(*2*)
by (*smt* (*verit*, *ccfv-threshold*) *One-nat-def Suc-1 Suc-diff-1 add-2-eq-Suc' card.infinite le-add2 min-conv-lower min-conv-min min-def not-less-eq-eq*)
have *S11*:*card S1* $\geq$ *1* **using** *S1*(*1*) *assms*(*2*)
by (*smt* (*verit*, *best*) *Suc-1 le-add1 le-add2 min-conv-min min-def le-add-diff-inverse nle-le not-less-eq-eq plus-1-eq-Suc*)
have *S1d*: *distinct* (*map fst* (*sorted-list-of-set S1*)) **using** *S1*
by *meson*

obtain *t S2* **where** *S2-prop*: $S2 = (\lambda p.\ p + t)$ ' *S2t*
$\forall x{\in}S1.\ \forall y{\in}S2.\ fst\ x < fst\ y$
$\forall x{\in}S1.\forall y{\in}\ S1.\ \forall z{\in}S2.\ x < y \longrightarrow slope\ x\ y < slope\ y\ z$
$\forall a{\in}S1.\ \forall b{\in}S2.\ \forall c{\in}S2.\ b < c \longrightarrow slope\ a\ b > slope\ b\ c$
using *shift-set-above S1*(*3*) *S1f S2t*(*3*) *S2tf S2t1 S1d S2td S11*
by (*smt* (*verit*, *best*))

have *cupExtendS1FromS2*: $\forall x{\in}S1.\ \forall y{\in}S1.\ \forall z{\in}S2.$ (*sdistinct* [*x*,*y*,*z*] $\longrightarrow$ *cup3 x y z*)
using *slope-cup3 S2-prop*
by (*metis distinct-length-2-or-more distinct-map order-le-imp-less-or-eq sorted2*)
have *capExtendS2FromS1*: $\forall a{\in}S1.\ \forall b{\in}S2.\ \forall c{\in}S2.$ (*sdistinct*[*a*,*b*,*c*] $\longrightarrow$ *cap3 a b c*)
using *slope-cap3 S2-prop*
by (*metis distinct-length-2-or-more distinct-map order-le-imp-less-or-eq sorted2*)

have *S2*:*card S2* = *min-conv* (*k* + *2*) (*Suc* (*l* + *2*)) − *1*
general-pos S2 *sdistinct*(*sorted-list-of-set S2*)
$\forall xs.\ set\ xs \subseteq S2 \wedge (sdistinct\ xs) \longrightarrow \neg(cap\ (k+2)\ xs \vee cup\ (Suc(l+2))\ xs)$
using *translated-set*[*of S2t -] S2t S2-prop*(*1*) **by** *blast*+

have *f12-1*:$S1 \cap S2 = \{\}$ **using** *S2-prop*(*2*) **by** *fast*
hence *f12-2*:*card* (*S1*∪*S2*) = *card S1* + *card S2* **using** *S1*(*1*) *S2*(*1*) *S1f S2tf S2-prop*(*1*)
by (*metis card-Un-disjoint finite-imageI*)
hence *f12-3*:*sorted-list-of-set* (*S1*∪*S2*) = (*sorted-list-of-set S1*) @ (*sorted-list-of-set S2*)
sdistinct (*sorted-list-of-set* (*S1* ∪ *S2*))
using *S2-prop*(*2*) *S2*(*3*) *S1*(*3*) *S1f S2tf sorted-union S2-prop*(*1*)
by (*metis finite-imageI*)+
hence *f12-0*: *general-pos* (*S1*∪*S2*)
using *S2-prop general-pos-union S1*(*2*,*3*) *S2*(*2*,*3*) *S1f S2tf* **by** *simp*

have *gg*:$\forall xs.\ set\ xs \subseteq (S1{\cup}S2){\wedge}(sdistinct\ xs) \longrightarrow \neg(cap\ (Suc\ (k+2))\ xs \vee cup\ (Suc\ (l+2))\ xs)$

proof(*rule+*)
fix *xs*
assume *asm*:*set xs* ⊆ *S1* ∪ *S2* ∧ *sdistinct xs cap* (*Suc* (*k* + *2*)) *xs* ∨ *cup* (*Suc* (*l* + *2*)) *xs*
define *XS1* **where** *XS1* = *S1* ∩ *set xs*
define *XS2* **where** *XS2* = *S2* ∩ *set xs*
define *xs1* **where** *xs1* = *sorted-list-of-set XS1*
define *xs2* **where** *xs2* = *sorted-list-of-set XS2*

have *xs1p1*:*set xs1* ⊆ *S1* **using** *xs1-def XS1-def* **by** *fastforce*
have *xs1p2*:*sdistinct xs1* **using** *xs1-def XS1-def S1*(*3*) *sdistinct-sub S1f*
by (*meson Int-lower1 finite-imageI*)

have *xs2p1*:*set xs2* ⊆ *S2* **using** *xs2-def XS2-def* **by** *fastforce*
have *xs2p2*:*sdistinct xs2* **using** *xs2-def XS2-def sdistinct-sub S2*(*3*) *S2-prop*(*1*) *S2tf*
by (*metis Int-lower1 finite-imageI*)

have *XS1* ∩ *XS2* = {} **using** *f12-1 asm XS1-def XS2-def* **by** *auto*
hence *xs-cat*: *xs* = *xs1* @ *xs2*
using *xs1-def xs2-def S2-prop*(*2*) *XS1-def XS2-def asm*(*1*) *S1f f12-3 xs1p2 xs2p2*
S2-prop(*1*) *S2tf*
by (*smt* (*verit*, *best*) *Int-Un-distrib2 Int-iff Un-Int-eq*(*1*)
distinct-append distinct-map finite-Int
finite-imageI set-append sorted-append
sorted-distinct-set-unique
sorted-list-of-set.set-sorted-key-list-of-set
subset-Un-eq)
hence *xs-len*: *length xs* = *length xs1* + *length xs2* **by** *simp*

The sets xs1 and xs2 satisfy the properties ¬ (*cap* (*k* + *2*) *xs1* ∨ *cup* (*Suc* (*l* + *2*)) *xs1*) and ¬ (*cap* (*Suc* (*k* + *2*)) *xs2* ∨ *cup* (*l* + *2*) *xs2*)

have *xs1p3*:¬(*cap* (*Suc*(*k*+*2*)) *xs1* ∨ *cup* (*l*+*2*) *xs1*) **using** *S1*(*4*) *xs1p1 xs1p2* **by** *simp*
have *xs2p3*:¬(*cap* (*k*+*2*) *xs2* ∨ *cup* (*Suc*(*l*+*2*)) *xs2*) **using** *S2*(*4*) *xs2p1 xs2p2* **by** *simp*

have *xs1-len-ne-0*: *length xs1* ≠ *0*
proof(*rule ccontr*)
assume *xs1asm*:¬*length xs1* ≠ *0*
hence *C1*:*cap* (*Suc* (*k*+*2*)) *xs* ⟹ *cap* (*Suc* (*k*+*2*)) *xs2*
using *xs-cat* **by** *fastforce*
have *C2*: *cap* (*Suc* (*k*+*2*)) *xs2* ⟹ *cap* (*k*+*2*) (*tl xs2*)
by (*metis Suc-eq-plus1 cap-reduct length-0-conv*
list.collapse xs1asm xs1p3)

have *C3*:*sdistinct* (*tl xs2*) **using** *xs2p2 sdistinct-subl* **by** *blast*
have *C4*:*set* (*tl xs2*) ⊆ *S2* **using** *xs2p1*

```
      by (metis list.sel(2) list.set-sel(2) subset-code(1))
    have C5:cap (k+2) (tl xs2) ⟹ False using S2(4) C3 C4 by blast

    have length xs1 = 0 using xs1asm by simp
    hence xs2lexs:length xs2 = length xs using xs-len by simp

    have cup (Suc (l+2)) xs ⟹ False using xs2p3 xs-cat xs1asm by auto
    thus False using C1 C2 C5 asm(2) by blast
  qed

  have xs2-len-ne-0: length xs2 ≠ 0
  proof(rule ccontr)
    assume xs2asm: ¬length xs2 ≠ 0
    hence C1:cap (Suc (k+2)) xs ⟹ False using xs-cat xs1p3 by auto

    have length xs2 = 0 using xs2asm by simp
    hence xs2lexs:length xs1 = length xs using xs-len by simp
    have C2: cup (Suc (l+2)) xs ⟹ cup (Suc (l+2)) xs1 using xs2asm xs-cat
by auto
    have C3: cup (Suc (l+2)) xs1 ⟹ cup (l+2) (tl xs1)
      by (metis cup-def cup-sub-cup diff-Suc-1 length-tl
          sublist-imp-subseq sublist-tl)

    have C4:sdistinct (tl xs1) using xs1p2 sdistinct-subl by blast
    have C5:set (tl xs1) ⊆ S1 using xs1p1
      by (metis list.sel(2) list.set-sel(2) subset-code(1))

    have cup (l+2) (tl xs1) ⟹ False using S1(4) C4 C5 by blast
    thus False using C2 C3 C1 asm(2) by blast
  qed

  have xs2-len1: cup (Suc (l + 2)) xs ⟹ length xs2 = 1
  proof(rule ccontr)
    assume asmF:cup (Suc (l + 2)) xs length xs2 ≠ 1
    have F1:length xs1 ≥ 1 length xs2 ≥ 2
      using xs1-len-ne-0 xs2-len-ne-0 asmF(2) by linarith+

    then obtain l2 prexs1 where l1l2:xs1 = prexs1 @ [l2]
      by (metis One-nat-def not-less-eq-eq rev-exhaust list.size(3) nle-le)

    have F3:l2 = xs!(length xs1 − 1) using F1(1) xs-cat l1l2
      by (simp add: nth-append-right)
    have F7:l2 ∈ S1 using l1l2 xs1p1 by auto

    obtain l3 l4 sufxs2 where l3l4:xs2 = l3#l4#sufxs2 using F1(2)
      by (metis One-nat-def Suc-1 Suc-le-length-iff)
    hence F4:l3 = xs!(length xs1)          by (simp add: xs-cat)
    have F41: xs = prexs1 @ [l2,l3,l4] @ sufxs2 using l3l4 l1l2 xs-cat
      by auto
```

```
  hence F42:sublist [l2, l3, l4] xs by fast
  have F62: sdistinct [l2, l3, l4] using F42 asm(1) sdistinct-subl by fastforce
  have F8:l3 ∈ S2 ∧ l4 ∈ S2 using l3l4 xs2p1 XS2-def xs2-def by simp
  have F10: cap3 l2 l3 l4 using F62 F7 F8 capExtendS2FromS1 by blast

  have cup 3 [l2,l3,l4] using cup-sub-cup F42 asmF sublist-imp-subseq
  by (smt (verit, ccfv-threshold) cup-def length-Cons list.size(3) numeral-3-eq-3)

  thus False using asmF(1) F10 exactly-one-true cup-def
    by (metis list-check.simps(4))
qed

have xs1-len1: cap (Suc (k + 2)) xs ⟹ length xs1 = 1
proof(rule ccontr)
  assume asmF: cap (Suc (k + 2)) xs ¬ length xs1 = 1
  have F1:length xs2 ≥ 1 length xs1 ≥ 2
    using xs1-len-ne-0 xs2-len-ne-0 asmF(2) by linarith+

  then obtain l1 l2 prexs1 where l1l2:xs1 = prexs1 @ [l1,l2]
    by (metis One-nat-def append.assoc append-Cons
        append-Nil asmF(2) length-Cons list.size(3)
        rev-exhaust xs1-len-ne-0)

  have F2:l1 = xs!(length xs1 − 2) using F1(1) xs-cat l1l2 by simp
  have F3:l2 = xs!(length xs1 − 1) using F1(1) xs-cat l1l2
    by (simp add: nth-append-right)
  have F7:l1 ∈ S1 ∧ l2 ∈ S1 using l1l2 xs1p1 by auto

  obtain l3 sufxs2 where l3l4:xs2 = l3#sufxs2
    by (metis length-0-conv list.exhaust xs2-len-ne-0)

  hence F4:l3 = xs!(length xs1)          by (simp add: xs-cat)
  have F41: xs = prexs1 @ [l1,l2,l3] @ sufxs2 using l3l4 l1l2 xs-cat
    by auto
  hence F42:sublist [l1, l2, l3] xs by fast
  hence F61:sdistinct [l1, l2, l3] using asm(1) sdistinct-subl by fastforce
  have F8:l3 ∈ S2 using l3l4 xs2p1 XS2-def xs2-def by simp
  have F9:  cup3 l1 l2 l3 using F61 F7 F8 cupExtendS1FromS2 by blast

  have F11: cap 3 [l1, l2, l3]
    using cap-sub-cap F42 cap-def sublist-imp-subseq asmF(1)
    by (smt (verit, del-insts) length-Cons list.size(3) numeral-3-eq-3)

  thus False using F9 exactly-one-true cap-def
    by (metis list-check.simps(4))
qed

have FF2:cup (Suc (l + 2)) xs ⟹ False
  using xs-len xs2-len1 xs1p3 cup-sub-cup xs-cat cup-def
```

by (*metis Suc-eq-plus1 subseq-rev-drop-many nat.inject subseq-order.order-refl*)

have *FF1*:*cap* (*Suc* (*k* + *2*)) *xs* ⟹ *False*
using *xs-len xs1-len1 xs2p3 cap-sub-cap xs-cat cap-def*
by (*metis list-emb-append2 old.nat.inject plus-1-eq-Suc subseq-order.order-refl*)

show *False* **using** *FF1 FF2 asm*(*2*) **by** *auto*

qed
have *asmf*:*min-conv-set* (*Suc* (*k* + *2*)) (*Suc* (*l* + *2*)) ≠ {}
using *assms*(*1*) *assms*(*2*) *min-conv-lower-bnd*(*2*) **by** *auto*
then have *min-conv* (*k* + *2*) (*Suc* (*l* + *2*)) − *1* + (*min-conv* (*Suc* (*k* + *2*)) (*l* + *2*) − *1*)
< *min-conv* (*Suc* (*k* + *2*)) (*Suc* (*l* + *2*))
using *S1*(*1*) *S2*(*1*) *add.commute f12-0 f12-2 min-conv-lower*[*of Suc* (*k*+*2*) *Suc* (*l*+*2*)] *min-conv-set-alt-def assms gg f12-3*(*2*)
by (*metis* (*lifting*))
thus *?thesis* **using** *asmf* **by** *simp*
qed

lemma *min-conv-k-l-eq*:
min-conv (*Suc* (*k*+*2*)) (*Suc* (*l*+*2*)) = *min-conv* (*k*+*2*) (*Suc* (*l*+*2*)) + *min-conv* (*Suc* (*k*+*2*)) (*l*+*2*) − *1*
∧ *min-conv-set* (*Suc* (*k*+*2*)) (*Suc* (*l*+*2*)) ≠ {}
proof(*induction k*+*l arbitrary*: *k l*)
case *0*
then show *?case* **using** *min-conv-arg-swap min-conv-base numeral-3-eq-3* **by** *fastforce*
next
case (*Suc x*)
then show *?case*
proof(*cases k*=*0* ∨ *l*=*0*)
case *True*
have *1*: *min-conv* (*k* + *2*) (*Suc* (*l* + *2*)) + (*min-conv* (*Suc* (*k* + *2*)) (*l* + *2*) − *1*)
= *min-conv* (*Suc* (*k* + *2*)) (*Suc* (*l* + *2*))
using *True min-conv-2*(*1*) *min-conv-arg-swap min-conv-base*
by (*metis add-2-eq-Suc' add-Suc-shift diff-Suc-1 le-add1 numeral-3-eq-3 plus-nat.add-0*)
have *2*: *min-conv-set 3* (*Suc* (*l* + *2*)) ≠ {} **using** *min-conv-base* **by** *simp*
have *3*: *min-conv-set* (*Suc* (*k* + *2*)) *3* ≠ {}
using *min-conv-base min-conv-sets-eq min-conv-set-def* **by** *simp*
thus *?thesis* **using** *1 2 3 True numeral-3-eq-3 min-conv-2*(*1*) *min-conv-arg-swap min-conv-base*
by (*smt* (*z3*) *One-nat-def Suc-eq-plus1 add.commute diff-Suc-1 le-add2 nat-1-add-1 nat-arith.suc1*)
next
case *False*

```
    hence FC1:k≥1  l≥1 by simp+
    have F1:(k)+(l−1) = x using FC1 Suc by simp
    have S1:min-conv-set (Suc (k + 2)) (l + 2) ≠ {} using Suc F1
      by (metis FC1(2) add-2-eq-Suc' le-add-diff-inverse plus-1-eq-Suc)
    have F2:(k−1) + l = x using FC1 Suc by simp
    have S2:min-conv-set (k + 2) (Suc (l + 2)) ≠ {} using Suc F2
      by (metis FC1(1) add-2-eq-Suc' le-add-diff-inverse plus-1-eq-Suc)

    have LT:  min-conv (Suc (k + 2)) (Suc (l + 2))
          ≤ min-conv (k + 2) (Suc (l + 2)) + min-conv (Suc (k + 2)) (l + 2) −
1
            min-conv-set (Suc (k + 2)) (Suc (l + 2)) ≠ {}
      using min-conv-lower-bnd-Suc S1 S2 by simp+
    have GT:  min-conv (Suc (k + 2)) (Suc (l + 2))
          > min-conv (k + 2) (Suc (l + 2)) − 1 + (min-conv (Suc (k + 2)) (l +
2) − 1)
      using LT(2) S1 S2 min-conv-upper-bnd by simp
    show ?thesis using LT GT by linarith
  qed
qed

lemma min-conv-bin:
  min-conv (k+3) (l+3) = ((k + l + 2) choose (k + 1)) + 1
proof(induction k+l arbitrary: l k)
  case (Suc x)
  then show ?case
  proof(cases k = 0)
    case False
    have 1:k≥1 using False by simp
    show ?thesis
    proof(cases l = 0)
      case True
      hence min-conv (k + 3) (l + 3) = k + 3
        using min-conv-arg-swap min-conv-base[of k+3] by simp
        moreover have (k + l + 2 choose k + 1) + 1 = k + 3 using True
binomial-Suc-n by simp
      ultimately show ?thesis using min-conv-base by simp
    next
      case False
      hence 2:l≥1 by simp
      have    x = (k−1) + l using 1 Suc by linarith
      hence 3:min-conv (k + 2) (l + 3) = (k + l + 1 choose k) + 1 using Suc
by fastforce
      have    x = k + (l−1) using 2 Suc by linarith
      hence   min-conv (k + 3) (l + 2) = (k + l + 1 choose k + 1) + 1 using
Suc by fastforce
      hence   min-conv (k+3) (l+3) = (k + l + 1 choose k) + (k + l + 1 choose
k + 1) + 1
        using 3 min-conv-k-l-eq
```

by (*simp add*: *numeral-3-eq-3*)
then show *?thesis* **using** *binomial-Suc-Suc* **by** *simp*
qed
qed(*simp add*: *min-conv-base*)
qed(*simp add*: *min-conv-base*)

We prove that cups and caps form a convex polygon.

lemma *cup-genpos*:
assumes *cup* (*length xs*) *xs*
shows *general-pos* (*set xs*)
proof−
have *gpos* (*set xs*) **unfolding** *gpos-def*
proof(*rule*+)
fix *a b c*
assume *asm*:*a* ∈ *set xs b* ∈ *set xs c* ∈ *set xs distinct* [*a*,*b*,*c*] *collinear3 a b c*
then obtain *u v w* **where** *uvw*:[*u*,*v*,*w*] = *sorted-list-of-set* {*a*,*b*,*c*}
by (*smt* (*verit*, *best*) *distinct-length-2-or-more*
empty-set insert-commute linorder-not-less
list.simps(*15*) *order-less-imp-le sorted1 sorted2*
sorted-list-of-set.idem-if-sorted-distinct)
have *1*:*subseq* [*u*,*v*,*w*] *xs* **using** *assms asm uvw*
by (*metis* (*no-types*, *lifting*) *bot.extremum cup-def distinctFst-distinct insert-subset*
subseq-sorted-subset)
hence *2*:*sdistinct* [*u*,*v*,*w*] **using** *assms uvw*
by (*simp add*: *cup-def sdistinct-subseq*)
hence ¬*collinear3 u v w* **using** *assms 1 slope-cup3 collinear3-def cup-is-slopeinc-subseq*
by (*metis cup3-def order-less-le*)
then show *False*
using *2 asm*(*4*,*5*) *uvw*
by (*metis coll-is-affDep sorted-list-of-set.set-sorted-key-list-of-set strict-sorted-iff*
finite.emptyI finite.insertI list.set(*1*) *list.simps*(*15*) *sdistinct-sortedStrict*)
qed
thus *?thesis* **using** *gpos-generalpos* **by** *simp*
qed

lemma *cup-abc-cup-bcd-implies-cup-acd*:
assumes *cup3 a b c*
and *cup3 b c d*
and *sdistinct* [*a*,*b*,*c*,*d*]
shows *cup3 a c d*
using *assms* **unfolding** *cup3-def*
by (*smt* (*verit*) *cup3-def cup3-slope distinct-length-2-or-more list.simps*(*9*)
order.trans slope-cup3 slope-trans(*2*) *sorted2 sorted-wrt1*)

lemma *cup4Impliescup3Last*:
assumes *cup 4* [*a*,*b*,*c*,*d*]
shows *cup3 a c d*

proof−
have *cup-abc*: *cup3 a b c* **using** *assms* **unfolding** *cup-def* **by** *simp*
have *cup-bcd*: *cup3 b c d* **using** *assms* **unfolding** *cup-def* **by** *simp*
show *?thesis* **using** *cup-abc-cup-bcd-implies-cup-acd cup-abc cup-bcd*
unfolding *cup3-def cross3-def*
using *assms cup-def* **by** *blast*
qed

definition *halfspace-pos* :: *real* × *real* ⇒ *real* × *real* ⇒ (*real* × *real*) *set* **where**
halfspace-pos p q ≡
let (*x1*, *y1*) = *p*;
(*x2*, *y2*) = *q*;
dpq = *sqrt*((*x2*−*x1*)^2 + (*y2*−*y1*)^2);
v = ((*y1* − *y2*)/*dpq*, (*x2* − *x1*)/*dpq*);
D = (*x2*∗*y1* − *x1*∗*y2*)/*dpq*
in {*s*. *inner v s* ≥ *D*}

definition *halfspace-neg* :: *real* × *real* ⇒ *real* × *real* ⇒ (*real* × *real*) *set* **where**
halfspace-neg p q ≡
let (*x1*, *y1*) = *p*;
(*x2*, *y2*) = *q*;
dpq = *sqrt*((*x2*−*x1*)^2 + (*y2*−*y1*)^2);
v = ((*y1* − *y2*)/*dpq*, (*x2* − *x1*)/*dpq*);
D = (*x2*∗*y1* − *x1*∗*y2*)/*dpq*
in {*s*. *inner v s* ≤ *D*}

lemma *a-c-belongs-halfspace-a-c*:
assumes *a* ≠ *c*
shows *a* ∈ *halfspace-pos a c*
and *c* ∈ *halfspace-pos a c*
proof−
define *x1* **where** *x1* ≡ *fst a*
define *y1* **where** *y1* ≡ *snd a*
define *x2* **where** *x2* ≡ *fst c*
define *y2* **where** *y2* ≡ *snd c*
define *dpq* **where** *dpq* ≡ *sqrt* ((*x2* − *x1*)^2 + (*y2* − *y1*)^2)
define *v* **where** *v* ≡ ((*y1* − *y2*) / *dpq*, (*x2* − *x1*) / *dpq*)
define *D* **where** *D* ≡ (*x2* ∗ *y1* − *x1* ∗ *y2*)/ *dpq*
hence *dpqNE0*: *dpq* > *0* **unfolding** *dpq-def x1-def x2-def y2-def y1-def*
using *assms*
by (*metis add-less-same-cancel1 add-less-same-cancel2 eq-iff-diff-eq-0*
linorder-neqE-linordered-idom not-sum-power2-lt-zero power2-less-0 prod.exhaust-sel
real-sqrt-eq-zero-cancel-iff real-sqrt-lt-0-iff zero-less-power2)

have *s-1*: *fst v* = (*snd a* − *snd c*) / *dpq* **using** *v-def y1-def x1-def x2-def y2-def*
by *simp*

have *s-2*: *snd v* = (*fst c* − *fst a*) / *dpq* **using** *v-def y1-def x1-def x2-def y2-def*
by *simp*
have *s-3*: *v* ∙ *a* = *D* **using** *s-1 s-2* **unfolding** *inner-prod-def D-def x1-def x2-def y1-def y2-def*
by (*smt* (*verit*, *del-insts*) *diff-diff-eq2 diff-divide-distrib diff-eq-diff-eq distrib-left divide-divide-eq-right group-cancel.sub1 inner-commute inner-real-def times-divide-eq-left times-divide-eq-right x1-def y2-def*)
have *s-4*: *v* ∙ *c* = *D* **using** *s-1 s-2* **unfolding** *inner-prod-def D-def x1-def x2-def y1-def y2-def*
by (*smt* (*verit*, *ccfv-SIG*) *add.assoc add.commute add-diff-cancel-left' add-diff-eq cancel-comm-monoid-add-class.diff-cancel diff-diff-eq diff-diff-eq2 diff-divide-distrib diff-eq-eq diff-zero distrib-right divide-divide-eq-left divide-divide-eq-right fst-swap inner-commute inner-real-def minus-diff-eq mult-minus-left mult-minus-right power2-eq-square real-divide-square-eq ring-class.ring-distribs*(1) *times-divide-eq-left times-divide-eq-right uminus-add-conv-diff x1-def x2-def y1-def y2-def*)
show *a* ∈ *halfspace-pos a c* **using** *s-3* **unfolding** *halfspace-pos-def*
by (*smt* (*verit*) *D-def dpq-def mem-Collect-eq split-beta v-def x1-def x2-def y1-def y2-def*)
show *c* ∈ *halfspace-pos a c* **using** *s-4 s-3* **unfolding** *halfspace-pos-def*
by (*metis* (*mono-tags*, *lifting*) ‹*a* ∈ *halfspace-pos a c*› *dpq-def halfspace-pos-def mem-Collect-eq split-beta v-def x1-def x2-def y1-def y2-def*)
qed

lemma *halfspace-pos-convex*: *convex* (*halfspace-pos a c*)
proof −
let *?x1* = *fst a*
let *?y1* = *snd a*
let *?x2* = *fst c*
let *?y2* = *snd c*
let *?dpq* = *sqrt* ((*?x2* − *?x1*)^2 + (*?y2* − *?y1*)^2)
let *?D* = (*?x2* ∗ *?y1* − *?x1* ∗ *?y2*) / *?dpq*
let *?v* = ((*?y1* − *?y2*)/*?dpq*, (*?x2* − *?x1*)/*?dpq*)
have *halfspace-pos a c* = {*x*. *?D* ≤ *inner x ?v*}
unfolding *halfspace-pos-def* **by** (*metis* (*no-types*, *lifting*) *Collect-cong inner-commute split-beta*)
then show *?thesis*
by (*metis* (*no-types*, *lifting*) *ext convex-halfspace-ge halfspace-pos-def split-beta*)
qed

lemma *cup4-last-halfspace-pos*:
assumes *cup 4* [*a*,*b*,*c*,*d*]
shows *d* ∈ *halfspace-pos a c*
proof−
define *x1* **where** *x1* ≡ *fst a*
define *y1* **where** *y1* ≡ *snd a*
define *x2* **where** *x2* ≡ *fst c*
define *y2* **where** *y2* ≡ *snd c*

define *dpq* **where** *dpq* ≡ *sqrt* ((*x2* − *x1*)^2 + (*y2* − *y1*)^2)
define *v* **where** *v* ≡ ((*y1* − *y2*) / *dpq*, (*x2* − *x1*) / *dpq*)
define *D* **where** *D* ≡ (*x2* ∗ *y1* − *x1* ∗ *y2*)/ *dpq*
have *distinct* [*a*,*b*,*c*,*d*] **using** *assms* **unfolding** *cup-def* **by** *auto*
hence *dpqNE0*: *dpq* > *0* **unfolding** *dpq-def x1-def x2-def y2-def y1-def*
by (*smt* (*verit*) *distinct-length-2-or-more prod.exhaust-sel real-less-rsqrt sum-power2-gt-zero-iff*)
have *cup-acd*: *cup3 a c d* **using** *assms cup4Impliescup3Last* **by** *simp*
have *inner-d1*: *d* ⋅ *v* = (*fst d* ∗ ((*y1* − *y2*) / *dpq*)) + (*snd d* ∗ (*x2* − *x1*) / *dpq*)
by (*simp add*: *inner-prod-def v-def*)
have *inner-d2*: *d* ⋅ *v* = (*fst d* ∗ ((*snd a* − *snd c*) / *dpq*)) + (*snd d* ∗ (*fst c* − *fst a*) / *dpq*)
by (*simp add*: *inner-prod-def v-def x1-def x2-def y1-def y2-def*)
have *inner-d3*: *d* ⋅ *v* = (*fst d* ∗ (*snd a* − *snd c*) + *snd d* ∗ (*fst c* − *fst a*)) / *dpq*
by (*simp add*: *add-divide-distrib inner-d2*)
have *1*: *fst d* ∗ (*snd a* − *snd c*) + *snd d* ∗ (*fst c* − *fst a*) > *fst c* ∗ *snd a* − *snd c* ∗ *fst a*
by (*smt* (*verit*, *ccfv-threshold*) *cross3-def cup3-def cup-acd left-diff-distrib mult.commute mult-diff-mult*)
have *4*: *v* ⋅ *d* ≥ *D* **using** *cup-acd inner-d3 dpqNE0*
unfolding *D-def x1-def x2-def y1-def y2-def cup3-def cross3-def*
by (*metis 1 divide-strict-right-mono inner-commute mult.commute nless-le*)
show *?thesis* **using** *4* **unfolding** *halfspace-pos-def*
by (*metis* (*mono-tags*, *lifting*) *D-def dpq-def mem-Collect-eq split-beta v-def x1-def x2-def y1-def y2-def*)
qed

lemma *convex-hull-subset-halfspace*:
assumes *cup 4* [*a*,*b*,*c*,*d*]
assumes *b* ∈ *convex hull* {*a*,*c*,*d*}
shows *convex hull* {*a*,*b*,*c*,*d*}⊆ *halfspace-pos a c*
proof−
have *s-1*:*a* ≠ *c* **using** *assms*(*1*) **unfolding** *cup-def* **by** *fastforce*
thus *?thesis* **using** *assms*(*2*) *a-c-belongs-halfspace-a-c*[*OF s-1*] *cup4-last-halfspace-pos*[*OF assms*(*1*)]
by (*smt* (*verit*, *del-insts*) *bot-least convex-hull-eq convex-hull-subset halfspace-pos-convex hull-subset insert-subset list.set*(*1*,*2*) *subset-antisym*)
qed

lemma *bNotInhalfspace-pos*:
assumes *cup 4* [*a*,*b*,*c*,*d*]
shows *b* ∉ *halfspace-pos a c*
proof−
define *x1* **where** *x1* ≡ *fst a*
define *y1* **where** *y1* ≡ *snd a*
define *x2* **where** *x2* ≡ *fst c*
define *y2* **where** *y2* ≡ *snd c*
define *dpq* **where** *dpq* ≡ *sqrt* ((*x2* − *x1*)^2 + (*y2* − *y1*)^2)

define *v* **where** *v* ≡ ((*y1* − *y2*) / *dpq*, (*x2* − *x1*) / *dpq*)
define *D* **where** *D* ≡ (*x2* ∗ *y1* − *x1* ∗ *y2*)/ *dpq*

have *cup-abc*: *cup3 a b c* **using** *assms* **unfolding** *cup-def* **by** *auto*
have *1*: *fst v* = (*snd a* − *snd c*) / *dpq* **using** *v-def y1-def x1-def x2-def y2-def* **by** *simp*
have *2*: *snd v* = (*fst c* − *fst a*) / *dpq* **using** *v-def y1-def x1-def x2-def y2-def* **by** *simp*
have *distinct* [*a*,*b*,*c*,*d*] **using** *assms* **unfolding** *cup-def* **by** *auto*
hence *dpqGT0*: *dpq* > *0* **unfolding** *dpq-def x1-def x2-def y2-def y1-def*
by (*smt* (*verit*) *distinct-length-2-or-more prod.exhaust-sel real-less-rsqrt sum-power2-gt-zero-iff*)
have *3*: *fst v* · *fst b* + *snd v* · *snd b* = ((*snd a* − *snd c*) ∗ *fst b* + (*fst c* − *fst a*) ∗ *snd b*)/*dpq*
using *1 2* **by** (*simp add*: *add-divide-distrib*)
have *4*: (*snd a* − *snd c*) ∗ *fst b* + (*fst c* − *fst a*) ∗ *snd b* < *fst c* ∗ *snd a* − *fst a* ∗ *snd c*
using *cup-abc* **unfolding** *cup3-def cross3-def* **by** *argo*
have *5*: *v* · *b* < *D* **using** *1 2 3 4 dpqGT0*
by (*simp add*: *D-def divide-strict-right-mono inner-prod-def x1-def x2-def y1-def y2-def*)
show *?thesis* **using** *5*
by (*smt* (*verit*) *D-def dpq-def halfspace-pos-def mem-Collect-eq split-beta v-def x1-def x2-def y1-def y2-def*)
qed

lemma *bNotInCHacd*:
assumes *cup 4* [*a*,*b*,*c*,*d*]
shows *b* ∉ *convex hull* {*a*,*c*,*d*}
proof(*rule ccontr*)
assume ¬ *b* ∉ *convex hull* {*a*, *c*, *d*}
hence *b* ∈ *convex hull* {*a*, *c*, *d*} **by** *simp*
hence *b* ∈ *halfspace-pos a c* **using** *convex-hull-subset-halfspace*[*OF assms*]
by (*simp add*: *halfspace-pos-convex subset-hull*)
moreover have *b* ∉ *halfspace-pos a c* **using** *bNotInhalfspace-pos*[*OF assms*] .
ultimately show *False* **by** *blast*
qed

lemma *cup-abc-cup-bcd-implies-cup-abd*:
assumes *cup3 a b c*
and *cup3 b c d*
and *sdistinct* [*a*,*b*,*c*,*d*]
shows *cup3 a b d*
using *assms* **unfolding** *cup3-def*

by (*smt* (*verit*, *del-insts*) *cup3-def cup3-slope distinct-length-2-or-more list.simps*(*9*) *order.trans*
slope-cup3 slope-trans(*1*) *sorted2 sorted-wrt1*)

lemma *cup4Impliescup3abd*:
assumes *cup 4* [*a*,*b*,*c*,*d*]
shows *cup3 a b d*
proof−
have *cup-abc*: *cup3 a b c* **using** *assms* **unfolding** *cup-def* **by** *simp*
have *cup-bcd*: *cup3 b c d* **using** *assms* **unfolding** *cup-def* **by** *simp*
show *?thesis* **using** *cup-abc-cup-bcd-implies-cup-abd cup-abc cup-bcd*
unfolding *cup3-def cross3-def*
using *assms cup-def* **by** *blast*
qed

lemma *cup4-first-halfspace-pos*:
assumes *cup 4* [*a*,*b*,*c*,*d*]
shows *a* ∈ *halfspace-pos b d*
proof−
define *x1* **where** *x1* ≡ *fst b*
define *y1* **where** *y1* ≡ *snd b*
define *x2* **where** *x2* ≡ *fst d*
define *y2* **where** *y2* ≡ *snd d*
define *dpq* **where** *dpq* ≡ *sqrt* ((*x2* − *x1*)^2 + (*y2* − *y1*)^2)
define *v* **where** *v* ≡ ((*y1* − *y2*) / *dpq*, (*x2* − *x1*) / *dpq*)
define *D* **where** *D* ≡ (*x2* ∗ *y1* − *x1* ∗ *y2*)/ *dpq*
have *distinct* [*a*,*b*,*c*,*d*] **using** *assms* **unfolding** *cup-def* **by** *auto*
hence *dpqNE0*: *dpq* > *0* **unfolding** *dpq-def x1-def x2-def y2-def y1-def*
by (*smt* (*verit*) *distinct-length-2-or-more prod.exhaust-sel real-less-rsqrt sum-power2-gt-zero-iff*)
have *cup-abd*: *cup3 a b d* **using** *assms cup4Impliescup3abd* **by** *simp*
have *inner-d1*: *a* • *v* = (*fst a* ∗ ((*y1* − *y2*) / *dpq*)) + (*snd a* ∗ (*x2* − *x1*) / *dpq*)
by (*simp add*: *inner-prod-def v-def*)
have *inner-d2*: *a* • *v* = (*fst a* ∗ ((*snd b* − *snd d*) / *dpq*)) + (*snd a* ∗ (*fst d* − *fst b*) / *dpq*)
by (*simp add*: *inner-prod-def v-def x1-def x2-def y1-def y2-def*)
have *inner-d3*: *a* • *v* = (*fst a* ∗ (*snd b* − *snd d*) + *snd a* ∗ (*fst d* − *fst b*)) / *dpq*
by (*simp add*: *add-divide-distrib inner-d2*)
have *1*: *fst a* ∗ (*snd b* − *snd d*) + *snd a* ∗ (*fst d* − *fst b*) > *fst d* ∗ *snd b* − *snd d* ∗ *fst b*
by (*smt* (*verit*, *del-insts*) *cross3-def cup3-def cup-abd left-diff-distrib mult.commute right-diff-distrib′*)
have *4*: *v* • *a* ≥ *D* **using** *cup-abd inner-d3 dpqNE0*
unfolding *D-def x1-def x2-def y1-def y2-def cup3-def cross3-def*
by (*metis 1 divide-right-mono dpqNE0 inner-commute inner-d3 mult.commute nless-le*)
show *?thesis* **using** *4* **unfolding** *halfspace-pos-def*
by (*metis* (*mono-tags*, *lifting*) *D-def dpq-def mem-Collect-eq split-beta v-def x1-def*

x2-def y1-def y2-def)
qed

lemma *convex-hull-subset-halfspace2*:
assumes *cup 4* [*a*,*b*,*c*,*d*]
assumes *c* ∈ *convex hull* {*a*,*b*,*d*}
shows *convex hull* {*a*,*b*,*c*,*d*}⊆ *halfspace-pos b d*
proof−
have *s-1*:*b* ≠ *d* **using** *assms*(*1*) **unfolding** *cup-def* **by** *fastforce*
thus *?thesis* **using** *assms*(*2*) *a-c-belongs-halfspace-a-c*[*OF s-1*]
cup4-last-halfspace-pos[*OF assms*(*1*)]
by (*metis halfspace-pos-convex assms*(*1*) *cup4-first-halfspace-pos hull-redundant*
insert-commute insert-subsetI less-by-empty subset-hull)
qed

lemma *cNotInhalfspace-pos-bd*:
assumes *cup 4* [*a*,*b*,*c*,*d*]
shows *c* ∉ *halfspace-pos b d*
proof−
define *x1* **where** *x1* ≡ *fst b*
define *y1* **where** *y1* ≡ *snd b*
define *x2* **where** *x2* ≡ *fst d*
define *y2* **where** *y2* ≡ *snd d*
define *dpq* **where** *dpq* ≡ *sqrt* ((*x2* − *x1*)^2 + (*y2* − *y1*)^2)
define *v* **where** *v* ≡ ((*y1* − *y2*) / *dpq*, (*x2* − *x1*) / *dpq*)
define *D* **where** *D* ≡ (*x2* ∗ *y1* − *x1* ∗ *y2*)/ *dpq*

have *cup-bcd*: *cup3 b c d* **using** *assms* **unfolding** *cup-def* **by** *auto*
have *1*: *fst v* = (*snd b* − *snd d*) / *dpq* **using** *v-def y1-def x1-def x2-def y2-def*
by *simp*
have *2*: *snd v* = (*fst d* − *fst b*) / *dpq* **using** *v-def y1-def x1-def x2-def y2-def* **by**
simp
have *distinct* [*a*,*b*,*c*,*d*] **using** *assms* **unfolding** *cup-def* **by** *auto*
hence *dpqGT0*: *dpq* > *0* **unfolding** *dpq-def x1-def x2-def y2-def y1-def*
by (*smt* (*verit*) *distinct-length-2-or-more prod.exhaust-sel real-less-rsqrt*
sum-power2-gt-zero-iff)
have *3*: *fst v* · *fst c* + *snd v* · *snd c* = ((*snd b* − *snd d*) ∗ *fst c* + (*fst d* − *fst*
b) ∗ *snd c*)/*dpq*
using *1 2* **by** (*simp add*: *add-divide-distrib*)
have *4*: (*snd b* − *snd d*) ∗ *fst c* + (*fst d* − *fst b*) ∗ *snd c* < *fst d* ∗ *snd b* − *fst*
b ∗ *snd d*
using *cup-bcd* **unfolding** *cup3-def cross3-def* **by** *argo*
have *5*: *v* · *c* < *D* **using** *1 2 3 4 dpqGT0*
by (*simp add*: *D-def divide-strict-right-mono inner-prod-def x1-def x2-def y1-def*
y2-def)
show *?thesis* **using** *5*
by (*smt* (*verit*) *D-def dpq-def halfspace-pos-def mem-Collect-eq split-beta v-def*
x1-def x2-def
y1-def y2-def)

qed

lemma *cNotInCHabd*:
assumes *cup 4* [*a*,*b*,*c*,*d*]
shows *c* ∉ *convex hull* {*a*,*b*,*d*}
proof(*rule ccontr*)
assume ¬ *c* ∉ *convex hull* {*a*, *b*, *d*}
hence *c* ∈ *convex hull* {*a*, *b*, *d*} **by** *simp*
hence *c* ∈ *halfspace-pos b d* **using** *convex-hull-subset-halfspace2*[*OF assms*]
by (*simp add*: *hull-inc in-mono*)
moreover have *c* ∉ *halfspace-pos b d* **using** *cNotInhalfspace-pos-bd*[*OF assms*] **.**
ultimately show *False* **by** *blast*
qed

lemma *cup-poly*:
assumes *cup* (*length xs*) *xs*
shows *convex-poly* (*length xs*) *xs*
proof(*rule ccontr*)
assume *asm*:¬*convex-poly* (*length xs*) *xs*
have *0*:*sdistinct xs* **using** *assms cup-def* **by** *simp*
have *1*:*subseq ys xs* ⟹ *length ys* = *card* (*set ys*) **for** *ys*
by (*metis assms cup-def distinct-card distinct-map sdistinct-subseq*)
have *2*:*sorted xs* **using** *0* **by** *simp*
have ¬*convex-pos* (*set xs*) **using** *asm* **by** *simp*
then obtain *Y* **where** *Y*: *Y* ⊆ (*set xs*) *card Y* ≤ *4* ¬*convex-pos Y* **using** *fourconvex* **by** *meson*
define *ys* **where** *ys*:*ys* = *sorted-list-of-set Y*
hence *23*: *distinct ys* **by** *simp*
hence *3*:*sdistinct ys* **using** *0*
by (*metis Y*(*1*) *distinct-map sdistinct-subseq subseq-sorted-subset ys*)
hence *34*:*sorted-wrt* (<) *ys*
using *sdistinct-sortedStrict* **by** *blast*
hence *4*:*subseq ys xs* **using** *Y*(*1*) *0 2 subseq-sorted-subset distinct-map ys* **by** *blast*
have *5*:*affine-dependent Y* **using** *Y*(*2*,*3*) *affine-hull-convex-hull hull-inc*
by (*metis affine-dependent-def convex-pos-def*)

show *False*

Since one of the points is contained within the triangle formed by other three: it forms a cap with some two of these points.

proof(*cases card Y* = *4*)
case *True*
have *num-4*:*4* = *Suc* (*Suc* (*Suc* (*Suc 0*))) **using** *numeral-3-eq-3* **by** *simp*
have *length* (*sorted-list-of-set Y*) = *4*
using *True* **by** *simp*
then obtain *a b c d* **where** *abcd*: *a*#*b*#*c*#*d*#*Nil* = *sorted-list-of-set Y*

using *num-4 length-Suc-conv Suc-length-conv length-0-conv* **by** *smt*
hence *y-set*: $Y = \{a,b,c,d\}$
by (*metis List.finite-set Y(1) infinite-super*
list.set(1) list.simps(15)
sorted-list-of-set.set-sorted-key-list-of-set)
hence *y-fin*: $\forall\, e{\in}Y.$ *finite* $(Y{-}\{e\})$ **by** *blast*
have *ys-abcd*:*ys* = [*a*,*b*,*c*,*d*] **using** *abcd ys* **by** *argo*
hence *y-a*:$\forall\, e{\in}Y.$ *card* $(Y{-}\{e\}) = 3$ **using** *True 23 Y(1) num-4 ys*
by (*metis add-diff-cancel-left' card-Diff-singleton numeral-3-eq-3 plus-1-eq-Suc*)
hence *y-a-len*:$\forall\, e{\in}Y.$ *length* (*sorted-list-of-set* $(Y{-}\{e\})) = 3$ **by** *simp*
have *y-sub-ind*:$\forall\, e{\in}Y.$ $\neg$*affine-dependent* $(Y{-}\{e\})$
using *general-pos-subs genpos-cross0 assms general-pos-def y-a*
cross-affine-dependent-conv cup-genpos Y(1) nsubset-def
by (*metis* (*mono-tags*, *lifting*) *Diff-subset mem-Collect-eq*)
hence *y-sub-tri*:$\forall\, e{\in}Y.$ *convex-pos* $(Y{-}\{e\})$
by (*metis affine-dependent-def affine-hull-convex-hull convex-pos-def hull-inc*)
have *coeffs*:$\forall\, e{\in}Y.$ $e \in$ *convex hull* $(Y{-}\{e\}) \longrightarrow$
$(\exists\, u.$ (*sum* u $(Y{-}\{e\}) = 1) \wedge (\forall\, x{\in}(Y{-}\{e\}).\ 0 < u\ x) \wedge$ *sum* $(\lambda x.\ u\ x *_R x)$ $(Y{-}\{e\}) = e)$
proof(*rule*, *rule*)
fix *e*
assume *asm*:$e \in Y$ $e \in$ *convex hull* $(Y - \{e\})$
then obtain *u* **where** *uy*: (*sum* u $(Y{-}\{e\}) = 1)$ $(\forall\, x{\in}(Y{-}\{e\}).\ 0 \le u\ x)$
sum $(\lambda x.\ u\ x *_R x)$ $(Y{-}\{e\}) = e$
using *convex-hull-finite y-fin mem-Collect-eq*
by (*smt* (*verit*, *best*))
have $e \in$ *convex hull* $(Y{-}\{x,\ e\})$ **if** *asm*:$x \in (Y{-}\{e\}) \wedge u\ x = 0$ **for** *x*
proof−
have *1*:*sum* u $(Y - \{x,\ e\}) = 1$
using *uy asm*
by (*smt* (*verit*, *ccfv-SIG*) *DiffD2 Diff-insert*
List.finite-set Y(1) finite.emptyI finite-Diff2
finite-insert insertI1 insert-Diff rev-finite-subset
sum.insert)
have *2*:$(\forall\, x{\in}Y - \{x,\ e\}.\ 0 \le u\ x) \wedge (\sum z{\in}Y - \{x,\ e\}.\ u\ z *_R z) = e$
proof(*rule*)
show $\forall\, x{\in}Y - \{x,\ e\}.\ 0 \le u\ x$ **using** *uy* **by** *blast*
have $(\sum z{\in}Y - \{x,\ e\}.\ u\ z *_R z) + (u\ x *_R x) = e$
using *uy add.commute asm*
by (*metis* (*no-types*, *lifting*) *Diff-insert2*
insert-absorb insert-commute sum.infinite
sum.insert-remove zero-neq-one)
thus $(\sum z{\in}Y - \{x,\ e\}.\ u\ z *_R z) = e$ **using** *asm* **by** *simp*
qed
hence $e \in \{y.\ \exists\, u.\ (\forall\, x{\in}Y{-}\{x,e\}.\ 0 \le u\ x) \wedge$ *sum* u $(Y{-}\{x,e\}) = 1 \wedge (\sum x{\in}Y{-}\{x,e\}.\ u\ x *_R x) = y\}$
using *1* **by** *blast*
thus *?thesis* **using** *convex-hull-finite Y(1) finite-subset*
by (*smt* (*verit*, *del-insts*) *mem-Collect-eq*

```
        sum.infinite)
    qed

    hence x ∈ (Y−{e}) ∧ u x = 0 ⟹ False for x
      using y-sub-ind Diff-insert2 Diff-insert-absorb convex-pos-def asm y-sub-tri
          hull-redundant-eq insert-absorb insert-commute mk-disjoint-insert asm
      by (smt (verit, best))

    thus (∃ u. (sum u (Y−{e}) = 1) ∧ (∀ x∈(Y−{e}). 0 < u x) ∧ sum (λx. u x
*R x) (Y−{e}) = e)
      by (metis (lifting) antisym-conv2 uy(1,2,3))
  qed

  have e∈{a,b,c,d} ⟹ e ∉ convex hull (Y−{e}) for e
  proof−
    have ca:¬ a ∈ convex hull (Y−{a})
    proof(rule ccontr)
      assume ¬ a ∉ convex hull (Y − {a})
      hence a∈convex hull (Y−{a}) by simp
      then obtain u where uY: (∀ x∈(Y−{a}). 0 < u x) sum u (Y−{a}) = 1
                          sum (λx. u x *R x) (Y−{a}) = a
        using coeffs Y(1) y-set by auto
      then have c-sum:u b + u c + u d = 1 using y-set coeffs
        by (smt (verit, best) 23 Diff-insert-absorb abcd
            distinct.simps(2) empty-set finite.emptyI
            finite-insert list.simps(15) sum.empty sum.insert ys)
      have fst-a-min:fst a < fst b fst a < fst c fst a < fst d using ys-abcd 3
        by (smt (verit)  distinct-length-2-or-more
            less-eq-prod-def list.simps(9) sorted2)+
      have fst a = sum (λx. u x * (fst x)) (Y−{a}) using uY
        by (metis (mono-tags, lifting) fst-scaleR fst-sum
            real-scaleR-def sum.cong)
      also have ... = u b * (fst b) + u c * (fst c) + u d * (fst d)
        using uY
        by (smt (verit) 23 Diff-insert-absorb abcd
            distinct.simps(2) empty-set finite.emptyI
            finite-insert list.simps(15) sum.empty sum.insert
            y-set ys)
      also have  ... >  u b * (fst a) + u c * (fst a) + u d * (fst a)
        using fst-a-min 23 uY(1) y-set ys-abcd
        by (smt (verit, best) DiffI insert-absorb insert-iff
            insert-not-empty mult-less-cancel-left-pos)
      ultimately show False
        by (metis c-sum distrib-left
            mult.commute mult.right-neutral
            order-less-irrefl)
    qed
    have cb:¬ b ∈ convex hull (Y−{b})
    proof(rule ccontr)
```

assume *asm*:¬ *b* ∉ *convex hull* (*Y* − {*b*})
hence *b*∈*convex hull* (*Y*−{*b*}) **by** *simp*
have *y-d*: *Y*−{*b*} = {*a*,*c*,*d*} **using** *y-set 23 ys-abcd* **by** *auto*
have *cup-ls*:*cup 4* [*a*,*b*,*c*,*d*]
by (*metis 4* ‹*length* (*sorted-list-of-set Y*) = *4*› *abcd assms cup-sub-cup ys-abcd*)
show *False* **using** *bNotInCHacd*[*OF cup-ls*] *asm y-d* **by** *auto*
qed
have *cc*:¬ *c* ∈ *convex hull* (*Y*−{*c*})
proof(*rule ccontr*)
assume *asm*:¬ *c* ∉ *convex hull* (*Y* − {*c*})
hence *c*∈*convex hull* (*Y*−{*c*}) **by** *simp*
have *y-d*: *Y*−{*c*} = {*a*,*b*,*d*} **using** *y-set 23 ys-abcd* **by** *auto*
have *cup-ls*:*cup 4* [*a*,*b*,*c*,*d*]
by (*metis 4* ‹*length* (*sorted-list-of-set Y*) = *4*› *abcd assms cup-sub-cup ys-abcd*)
show *False* **using** *cNotInCHabd*[*OF cup-ls*] *asm y-d* **by** *auto*
qed
have *cd*:¬ *d* ∈ *convex hull* (*Y*−{*d*})
proof(*rule ccontr*)
assume ¬ *d* ∉ *convex hull* (*Y* − {*d*})
hence *d*∈*convex hull* (*Y*−{*d*}) **by** *simp*
then obtain *u* **where** *uY*: (∀ *x*∈(*Y*−{*d*}). *0* < *u x*) *sum u* (*Y*−{*d*}) = *1*
sum (λ*x*. *u x* ∗$_R$ *x*) (*Y*−{*d*}) = *d*
using *coeffs Y*(*1*) *y-set* **by** *auto*
then have *c-sum*: *u a* + *u b* + *u c* = *1* **using** *y-set coeffs*
by (*smt* (*z3*) *23 Diff-insert-absorb abcd*
distinct.simps(*2*) *empty-set finite.emptyI*
finite-insert insertCI insert-absorb insert-commute
list.simps(*15*) *sum.empty sum.insert ys*)
have *y-d*: *Y*−{*d*} = {*a*,*b*,*c*} **using** *y-set 23 ys-abcd* **by** *auto*
have *fst-a-min*:*fst d* > *fst a fst d* > *fst b fst d* > *fst c* **using** *ys-abcd 3*
by (*smt* (*verit*) *distinct-length-2-or-more less-eq-prod-def list.simps*(*9*) *sorted2*)+

have *fst d* = *sum* (λ*x*. *u x* ∗ (*fst x*)) (*Y*−{*d*}) **using** *uY*
by (*metis* (*mono-tags*, *lifting*) *fst-scaleR fst-sum real-scaleR-def sum.cong*)

also have ... = *u a* ∗ (*fst a*) + *u b* ∗ (*fst b*) + *u c* ∗ (*fst c*)
using *uY*(*3*) *y-d 23 abcd ys*
by (*smt* (*verit*, *ccfv-threshold*) *distinct.simps*(*2*) *empty-set finite.emptyI*
finite-insert insert-iff list.simps(*15*) *sum.empty sum.insert*)

also have ... < *u a* ∗ (*fst d*) + *u b* ∗ (*fst d*) + *u c* ∗ (*fst d*)
using *fst-a-min 23 uY*(*1*) *y-set ys-abcd*
by (*smt* (*verit*, *best*) *DiffI insert-absorb insert-iff*
insert-not-empty mult-less-cancel-left-pos)

ultimately show *False* **using** *c-sum*

```
      by (metis distrib-left mult.commute mult.right-neutral order-less-irrefl)
    qed

    show ?thesis using ca cb cc cd Y(3) convex-pos-def y-set
      by (smt (verit, ccfv-SIG) hull-redundant-eq insertE insert-Diff singletonD)
  qed
  then show ?thesis using Y(3) convex-pos-def y-set
    by (metis hull-inc)
 next
  case False
  hence y-3:card Y ≤ 3
    using Y(2) by linarith
  then show ?thesis
  proof(cases card Y = 3)
    case True
    hence ¬affine-dependent Y using general-pos-subs genpos-cross0 assms gen-
eral-pos-def cross-affine-dependent-conv cup-genpos Y(1) nsubset-def
      by (metis (mono-tags, lifting) mem-Collect-eq)
    thus ?thesis using 5 by simp
  next
    case False
    hence y-2:card Y ≤ 2 using y-3 by simp
    have C0:card Y = 0 ⟹ False
      using affine-independent-0 5 Y(1) finite-subset by fastforce
    have C1:card Y = 1 ⟹ False
      using affine-independent-1 by (metis 5 card-1-singletonE)
    have   card Y = 2 ⟹ False
      using 5 affine-independent-2 by (metis card-2-iff)
    thus ?thesis using y-2 C0 C1
      by fastforce
  qed
 qed
qed

lemma cap-poly:
 assumes cap (length xs) xs
 shows   convex-poly (length xs) xs
 using cup-poly cap-orig-refl-rev[of length xs xs] length-neg
 by (metis assms convex-pos-list-refl implm-rev-reflect-inv length-rev set-rev)

lemma min-conv-set-non-empty:
 min-conv-set n n ≠ {}
proof(cases n ≤ 2)
 case True
 then show ?thesis using min-conv-0 min-conv-1 min-conv-2
   by (metis One-nat-def Suc-1 bot-nat-0.extremum-uniqueI le-SucE)
next
 case False
 hence n≥3 by simp
```

then show *?thesis* **using** *min-conv-k-l-eq*
by (*metis One-nat-def Suc-1 add-Suc-right le-add-diff-inverse2 numeral-3-eq-3*)
qed

lemma *EZ-min-conv*:*EZ n* ≤ *min-conv n n*
proof−
have *cap a xs* ∨ *cup a xs* ⟶ *convex-poly a xs* **for** *a xs*
using *cup-poly cap-poly cap-def cup-def* **by** *presburger*
hence *min-conv-set n n* ⊆
{*N* :: *nat*. (∀ *S* :: *R2 set*. (*card S* ≥ *N* ∧ *general-pos S* ∧ *sdistinct*(*sorted-list-of-set S*))
⟶ (∃ *xs*. *set xs* ⊆ *S* ∧ *sdistinct xs* ∧ *convex-poly n xs*))} (**is** - ⊆ *?EZXS n*)
by (*smt* (*verit*) *mem-Collect-eq min-conv-num-out-set subset-eq*)
thus *EZ n* ≤ *min-conv n n*
using *EZ-def inf-subset-bounds min-conv-def min-conv-set-def min-conv-set-non-empty*
by *fastforce*
qed

lemma *EZ-bound-final*:
assumes *n* ≥ *3*
shows *EZ n* ≤ (((*2*n* −*4*) *choose* (*n* − *2*)) + *1*)
using *assms min-conv-bin*[*of n*−*3 n*−*3*] *EZ-min-conv*[*of n*]
by (*smt* (*verit*, *ccfv-SIG*) *One-nat-def Suc-1 Suc-eq-plus1 add-mult-distrib2 diff-Suc-numeral eq-numeral-Suc le-add-diff-inverse2 mult-2 numeral-3-eq-3 numeral-Bit0-eq-double ordered-cancel-comm-monoid-diff-class.add-diff-assoc plus-1-eq-Suc right-diff-distrib'*)

end